Kubernetes as a Virtual Machine Management System

A Comparative Study of KubeVirt and Traditional Approaches

EMIL KARLSSON

Master in Computer Science
Date: May 9, 2024
Supervisors: Han Fu
Examiner: Cyrille Artho
School of Electrical Engineering and Computer Science
Host company: Net Insight AB
Swedish title: Swedish title TBD

Abstract

Virtual Machines (VMs) and containers are technologies that offer virtual computing environments, with their respective benefits of having strong isolation and being lightweight. For this reason, they are often used in similar, if not the same, contexts. However, due to them being created at different points in time, they have traditionally been managed in separate management environments. This has led to a fragmented infrastructure for organizations that use both VMs and containers, but also to certain technologies and mechanisms having duplicate implementations, even though their implementations could have been shared. To remedy this issue, KubeVirt was created to unify the management of VMs and containers by integrating VMs into Kubernetes, a widely adopted platform originally designed for container orchestration.

This project aims to evaluate the applicability of managing VMs in Kubernetes, how it compares with traditional approaches and the implications of moving into a Kubernetes-based environment for VM management. A mixed-methods approach is used, including a literature study, a qualitative comparison in the form of a usability analysis and a quantitative comparison in the form of a performance analysis.

The results suggest that KubeVirt with Kubernetes offers a scalable alternative to traditional approaches, but that in certain use cases, KubeVirt might not be suitable. The results also show that the adoption and perceived usability of KubeVirt are tied to the experience of Kubernetes and its concepts.

Sammanfattning

Contents

1 Introduction **3**
1.1 Research Questions 6
1.2 Problem 7
1.3 Purpose 7
1.4 Goals 7
1.5 Research Method 7
1.6 Limitations 8
1.7 Structure of the thesis 8

2 Background **10**
2.1 Virtualization Fundamentals 10
2.2 Understanding Hypervisors 13
2.3 VM Management Essentials 17
2.3.1 Comparison of VM Management Systems 20
2.4 Containers and Kubernetes: A Paradigm Shift 24
2.4.1 Containers Explained 24
2.4.2 Orchestration Using Kubernetes 27
2.5 Containers vs. Virtual Machines 29
2.5.1 Comparative Overview 30
2.5.2 Decision Factors 32
2.5.3 Towards Stronger Container Isolation 34
2.6 Extending Kubernetes with KubeVirt 34
2.6.1 KubeVirt Technical Overview 36
2.6.2 KubeVirt and GitOps 36
2.6.3 Comparison with Traditional Approaches 36
2.7 Related Work 37
2.7.1 Current challenges 37
2.7.2 KubeVirt challenges 38
2.7.3 Usability of Kubernetes and KubeVirt 38

2.7.4 Performance Evaluations 40

3 Performance Analysis 42
3.1 Setup 42
3.2 Building a Framework 44
3.2.1 Design 44
3.2.2 Implementation 49

4 Usability Analysis 53
4.1 Setup 53
4.2 Task Design 54
4.3 Conducting the Usability Analysis 55

5 Results 56
5.1 Performance Analysis 57
5.2 Usability Analysis 68

6 Discussion 71
6.1 Limited Features and Result Intepretation 71
6.2 Kubernetes Overhead and Induced Load 73
6.3 Jagged Disk Utilization 75
6.4 Mixed Opinions Using KubeVirt 76
6.5 Concluding Research Questions 77

7 Conclusions 79

8 Future Work 80

Bibliography 81

A Performance Analysis Components 93
A.1 VM Management System Interface 93
A.2 Metrics Collector 93
A.3 Metrics Collector Daemon 95

B Performance Analysis Results 96

C Usability Tasks 121

D Usability Analysis Answers **131**
D.1 Participants 131
D.2 Task 1/3 131
D.3 Task 2/3 134
D.4 Task 3/3 135
D.5 Summary 137

List of Abbreviations

API Application Programming Interface.

AWS Amazon Web Services.

AZ Availability Zone.

CLI Command Line Interface.

CNCF Cloud Native Computing Foundation.

CRD Custom Resource Definition.

CRUI Checkpoint-restore in Userspace.

EC2 Elastic Compute.

GCE Google Compute Engine.

GUI Graphical User Interface.

HPC High-Performance Computing.

IaaS Infrastructure as a Service.

JVM Java Virtual Machine.

KVM Kernel-based Virtual Machine.

OS Operating System.

PaaS Platform as a Service.

QEMU Quick EMUlator.

SaaS Software as a Service.

SAN Storage Area Network.

SSH Secure Shell.

vCPU Virtual CPU.

VM Virtual Machine.

VMM Virtual Machine Monitor.

VNC Virtual Network Computing.

Chapter 1

Introduction

The earliest usage of virtualization goes back several decades when larger computers were split into smaller units. This gave multiple users simultaneous access to a computer and allowed for more efficient use of hardware resources. However, it would not be until the late 1990s that virtualization would take off when commodity servers grew in popularity. This time, virtualization allowed companies to run a variety of applications on a wide range of Operating Systems (OSs) [1]. This image of virtualization has led to its widespread adoption that is seen today, and nowadays virtualization forms the base for cloud computing, providing virtual environments that can be used for various workloads, including compute, network and storage [2, 3].

The key enabler for virtualization is the hypervisor, which is a software layer that operates between the hardware and the OS and is responsible for creating virtual computing environments called virtual machines (VMs) [3, 4, 5]. VMs are emulations of physical computers that provide a complete system with its own OS. This means a VM is self-sufficient and can be isolated from its outside environment. This is particularly useful when running legacy applications and applications that require a high level of isolation. For these reasons, VMs have become the core part of IT infrastructure [1, 2].

However, with the evolution of virtualization, an alternative to VMs was created called containers. By raising the virtualization level to reside above an OS, containers become lightweight and portable [6]. For this reason, containers have become a popular choice in scenarios where scalability is important, such as in a microservice architecture, which are core components in modern cloud-native applications [7]. Figure 1.1 visualizes the differences between how VMs and containers are virtualized, and how the virtualization layer is placed on a lower level for VMs than for containers.

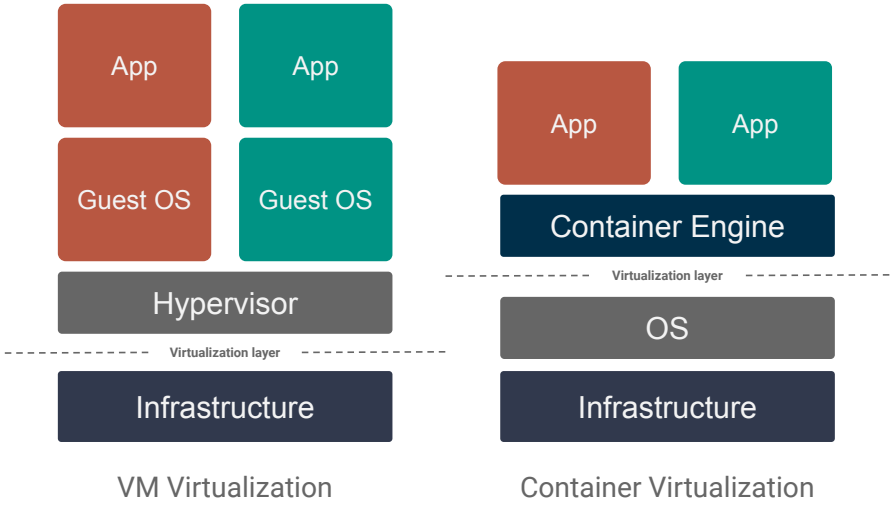

Figure 1.1: Overview of VMs and containers. The virtualization layer is placed on a lower level for VMs than for containers, illustrating the difference in their respective virtualization techniques.

However, the benefits of a container does not come without a cost. Due to the fact containers share the underlying OS kernel, it opens up a larger attack surface, which increases the risk of a security vulnerability. Some technologies have been created to mitigate this issue, such as Firecracker, Kata Containers and gVisor, but none of them has gained significant traction due to the trade-off they impose with additional overhead. This means the decision of using a VM or a container remains a use-case question [8, 9].

Due to their different use cases, VMs and containers have traditionally been managed separately. There exists a wider range of management systems for VMs than for containers since they have existed for a longer time. VMs are often managed by a VM management system that provides tools to aid in life-cycle management. Many such systems are designed with a set of control nodes and worker nodes, where control nodes instruct the worker nodes with VM-related tasks, such as creating a VM. A VM management system can thus be seen as a hypervisor distributed across several nodes, which is illustrated in Figure 1.2. Examples of such systems include OpenStack [10], CloudStack [11], and OpenNebula [12]. Furthermore, a management system for a container is similar to that of a VM. It also provides a set of tools for life-cycle management and is commonly structured with control and worker nodes. However, a system that manages containers also includes certain operations that are not as applicable to VMs, such as auto-scaling. This has led

to these systems being referred to as an orchestration system, rather than just management. Examples include Kubernetes [13], Nomad [14] and Docker Swarm [15], where the former is the most widely adopted in the industry [16]. As such, while VMs and containers have their respective use cases, they also share many aspects regarding how they provide virtual computing environments. For this reason, there has been an ongoing convergence in how VMs and containers are managed [17].

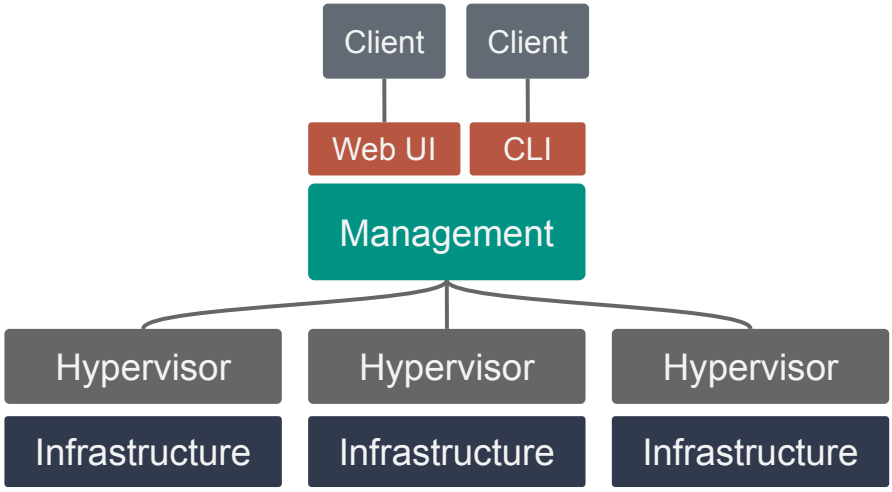

Figure 1.2: A brief overview of a VM management system built in a monolithic fashion, where clients can send VM-related instructions to a centralized location.

Recently, Kubernetes was extended to accommodate VMs through KubeVirt. It is a novel project that aims to fill a gap in how virtualized resources are managed by unifying the management of VMs and container into a single platform. Running both VMs and container in the same platform enables traffic to flow natively between the two resources, which is illustrated in Figure 1.3. How this is achieved on a technical level is discussed later in 2.6.1.

The approach of managing VMs as a Kubernetes resource comes with many benefits. A VM can now be managed in a more cloud-native manner through Kubernetes workflows, but it can also integrate with tools that previously only supported containers [7]. Additionally, managing both VMs and containers in the same platform reduces architectural complexity since only one platform is needed, as well as reducing the required skillset among the personnel [18, 19].

The integration of VMs into Kubernetes using KubeVirt represents a significant milestone toward a more efficient and unified IT infrastructure [19].

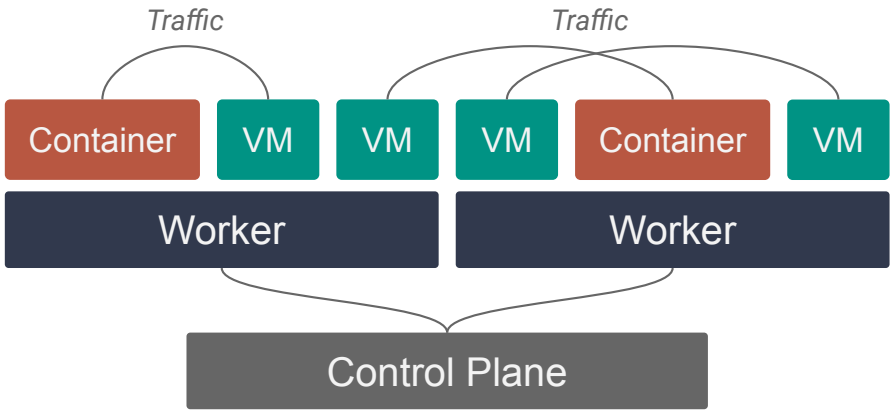

Figure 1.3: Overview of VMs and containers managed by using KubeVirt. The figure illustrates how VMs and containers communicate natively in a single environment.

However, KubeVirt's practicality in the real world is not yet fully explored. As such, this project aims to evaluate Kubernetes with KubeVirt as a VM management system and compare it with traditional approaches to understand its practicality and limitations.

1.1 Research Questions

What performance, scalability, and operational efficiencies can be gained by integrating VMs in Kubernetes?

This could stem from using a single resource scheduler. Several metrics will be used to evaluate this, as described in the background and related work.

What are the current challenges and limitations in managing VMs and containerized applications in separate environments?

This is concerned with scenarios in which a traditional VM management system is operated alongside Kubernetes.

How can system administration be simplified by running VMs in Kubernetes using KubeVirt?

This is focused on the practical implications of using KubeVirt and Kubernetes

instead of traditional VM management systems. For instance, the number of steps required to deploy a VM.

1.2 Problem

VMs was the first technology that offered virtual computing environments that could be used to host various applications. This led to a widespread adoption of VMs and many management systems were since developed. With the evolution of VMs, containers were created as a lightweight alternative. However, this led to separate management systems being created for the two technologies, even though they are often used in a similar, if not the same, context. This means that duplicate implementations of mechanisms have been required, even though they could have been shared, but also that environments are more architecturally complex.

1.3 Purpose

The purpose of this project is to evaluate whether Kubernetes with KubeVirt is a viable option for VM management. This evaluation is important in the field of cloud computing since it can provide insight into how an existing system could benefit from integrating VMs management in Kubernetes, as well as by consolidating the management of VMs and containers to a single platform.

1.4 Goals

The goal of this project is to provide a transparent evaluation with quantifiable results regarding the applicability of managing VMs inside the Kubernetes ecosystem. The results could be used as a fundamental argument for or against Kubernetes as a management platform. Results that argue for the use of KubeVirt in terms of performance could be used as an argument for future research regarding sustainability.

1.5 Research Method

This project will use a mixed-methods approach to investigate the research questions. A literature study will initially be conducted to thoroughly explore

what led up to KubeVirt being created, as well as the current challenges regarding VM management. To evaluate KubeVirt itself, both a quantitative and qualitative analysis will be conducted. The quantitative analysis will be based on a performance analysis that compares KubeVirt against a traditional approach and benchmarks the performance of the system when performing tasks such as creating VMs. The qualitative aspects will be based on a usability analysis that involves having system administrators perform certain common tasks using KubeVirt. Together, these methods aim to capture an overall image of KubeVirt for managing VMs.

1.6 Limitations

1. This project only investigates what is available in KubeVirt and does not account for possible features that exist in a system that it is being compared to. This means that this project acknowledges that KubeVirt itself might not be a complete replacement for a traditional VM management system, but rather form a complete alternative in conjunction with Kubernetes tools and integrations.

2. This project primarily focuses on the practical implications of moving into a KubeVirt-based system. However, it does not investigate or explore processes regarding the installation and configuration of a KubeVirt-based system.

1.7 Structure of the thesis

This report is divided into three main parts. The first part is concerned with background information and the relevant concepts and technologies that are necessary to understand the problem presented in this project. It aims to provide the fundamentals to understand why VM management and container orchestration have traditionally been separated as well as why KubeVirt was created. This part also includes related work that is used as a base for the methods used in the project.

The second part describes the research method used in the project and is divided into two branches, performance analysis and usability analysis. These branches cover both the quantitative and qualitative aspects of the project.

The third and last part involves result evaluation and discussions regarding the interpretation of the results. It combines the quantitative and qualitative

aspects to provide a more comprehensive understanding of the results, as well as answer the research questions and conclude the project as a whole.

Chapter 2

Background

This chapter provides all the fundamentals to understand the problem this project aims to solve. It includes fundamentals for what virtualization, hypervisors and containers are but also practical information regarding the currently used technologies. As such, this chapter provides the necessary information to understand why KubeVirt was created.

2.1 Virtualization Fundamentals

Virtualization is the technology that enables hardware to be split or grouped into logical units. Each unit runs with its own allocated resources and is separated from other units running on the same host, which means virtualization can be applied to networks, storage and entire servers [2]. The two former are simpler kinds of virtualization that involve creating an abstraction layer on top of existing physical resources to provide a logical interface defined in software. By defining storage and network in software, a more flexible environment is created, where an application interacts with interfaces that indirectly point to a group of hardware, instead of directly communicating with the hardware. Server virtualization, on the other hand, is the more common form of virtualization and is also known as hypervisor-based virtualization. It involves splitting a server into multiple virtual computing environments [1].

A virtual computing environment, more commonly referred to as a VM, is a complete system that operates as an isolated environment. The environment is perceived as if it were an ordinary computer from the perspective of the application running inside it, even though the physical resources are shared with other VMs running on the same host. Being a complete system is particularly useful when running legacy applications that are dependent on a certain OS,

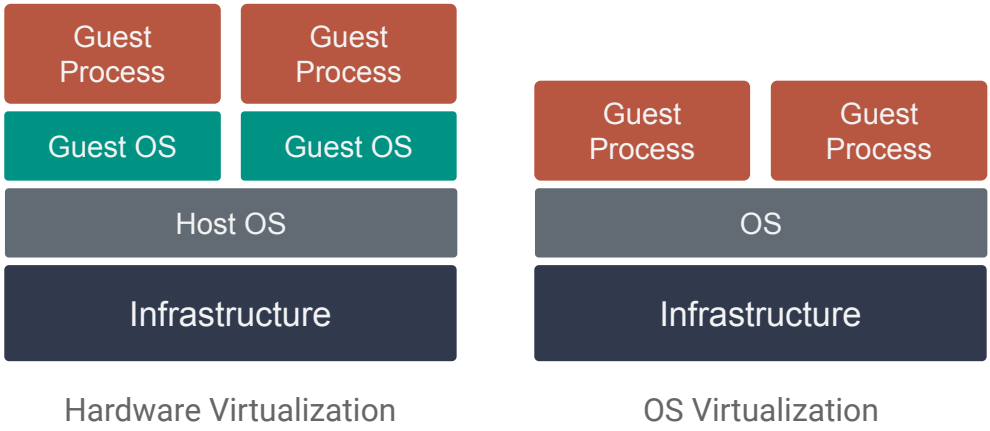

Figure 2.1: Overview of the different layers of virtualization for hardware-level and OS-level virtualization. The figure illustrates how hardware virtualization inhibits several internal OSs from running on the same hardware, while OS virtualization keeps a single OS for each process.

driver version or piece of hardware. Additionally, the isolated environments are suitable in scenarios with strict security requirements regarding the application running inside it [2, 3].

The three major levels for virtualizations are hardware level, OS-level and application level. Storage and network virtualization both fall under the hardware level category since they create an abstraction layer on top of the physical resources. However, virtualization is not required to be low-level. Application virtualization is concerned with encapsulating an application in a more lightweight environment, such that the application thinks it is executed on a specific platform. A common example is Java Virtual Machine (JVM) that enables programs written in the programming language Java to be run on multiple operating systems [20]. Figure 2.1 illustrates this as an overview of the differences between the two more common layers of virtualization; hardware and OS virtualization. It is important to note that the virtualization layers are not mutually exclusive and that using them in combination could lead to unique use cases. For instance, a guest process (OS level) can run in a guest OS (hardware level). This specific use case is referred to as nested virtualization [2].

So far, virtualization has been described as being able to contain an entire environment virtually, but this is not always a feasible solution. While maintaining a low-level abstraction layer enables great flexibility, it could also lead to degrading performance [2]. For this reason, virtualization was divided into

two categories, full virtualization and paravirtualization. Full virtualization is what has been described so far and is the older and traditional approach where the entire underlying hardware is virtualized to its core. This allows a VM to portray any specifications to the application running inside it and the application has no way of knowing it is being run in a virtual environment [2, 21, 22]. Paravirtualization, however, reduces the number of resources that are virtualized by raising the virtualization layer slightly. With paravirtualization, a VM is presented with an image that is similar, but not identical, to the underlying hardware. A consequence of this is that the guest OS is aware it is running inside a VM [21, 22].

Benefits of Virtualization

Many of the benefits of virtualization stem from the fact that there exists an abstraction layer above the physical hardware. The abstraction layer decouples software from hardware, allowing for the use of vendors with different I/O in the same environment. This creates a more flexible and scalable computing environment as a whole where hardware is turned into a simple pool of computing resources [3]. As such, there are numerous benefits of virtualization, a few of which are listed below.

- **Isolation**
  A VM provides isolation by being self-sufficient and containing its own OS, including the filesystem and application stack. As previously described, being able to provide isolation is particularly useful in scenarios with high-security requirements [23].

- **Flexibility**
  Another consequence of being self-sufficient is portability, which leads to more flexible management of VMs. This is useful in many management-related tasks, such as live migration and load balancing, but also when performing maintenance on the physical hardware.

- **Resource utilization**
  A problem with larger servers is that hardware resources might not be fully utilized. VMs alleviates this issue by placing isolated workloads on the same physical hardware. If hardware resources are more utilized, it reduces the need to acquire additional hardware, which in turn could make server management simpler [2].

- **Management**
  A VM conceptually represents computing resources defined in software, which means they are by design manageable. This has led to VM management systems being created that allow remote control of VMs, such as creating and deleting them [2].

- **Virtual hardware**
  A VM is built upon virtualized resources, meaning resources that are not necessarily directly tied to a physical resource. For instance, a VM might be allocated 4 Virtual CPU (vCPU) cores even though a server only had 2 physical CPU cores available [2]. The benefits of this scenario specifically are described later in 2.5.1.

2.2 Understanding Hypervisors

To allow a computer to be split into several virtual computing environments, a software layer is injected. This software layer is referred to as a hypervisor and is the key technology that enables the creation of VMs. Hypervisors are also referred to as Virtual Machine Monitor (VMM) [1, 5].

Hypervisors can be used in primarily two ways, either by having it manage its VMs using its own scheduler and memory management or by placing it as an abstraction layer on top of an underlying OS. These are called *type 1* and *type 2* respectively. Type 1 hypervisors manage VMs by directly interacting with the underlying hardware. This means they are self-sufficient, but also that they are required to implement many features on their own. However, having no other middleware means type 1 hypervisor are performant and scalable. On the other hand, a type 2 hypervisor is situated above and OS that enables its creation of VMs. This means a type 2 hypervisor delegates many mechanisms such as scheduling and resource management to an underlying OS [24]. While a type 2 hypervisor represents a simpler approach to creating VMs, it also inhibits a larger latency and overhead than type 1 hypervisors. For this reason, type 2 hypervisors have found their use cases in development and testing, rather than in production [5]. Figure 2.2 illustrates a high-level overview of the difference between a type 1 and type 2 hypervisor [5, 24].

Hypervisors can be further categorized into *monolithic* and *microkernelized* hypervisors [25]. Monolithic hypervisors are built as a single large software stack, managing all the VMs and their resources, as well as hosting drivers for all hardware access. Having all drivers reside within the hypervisor means that the hypervisor will only run on certain hardware. However,

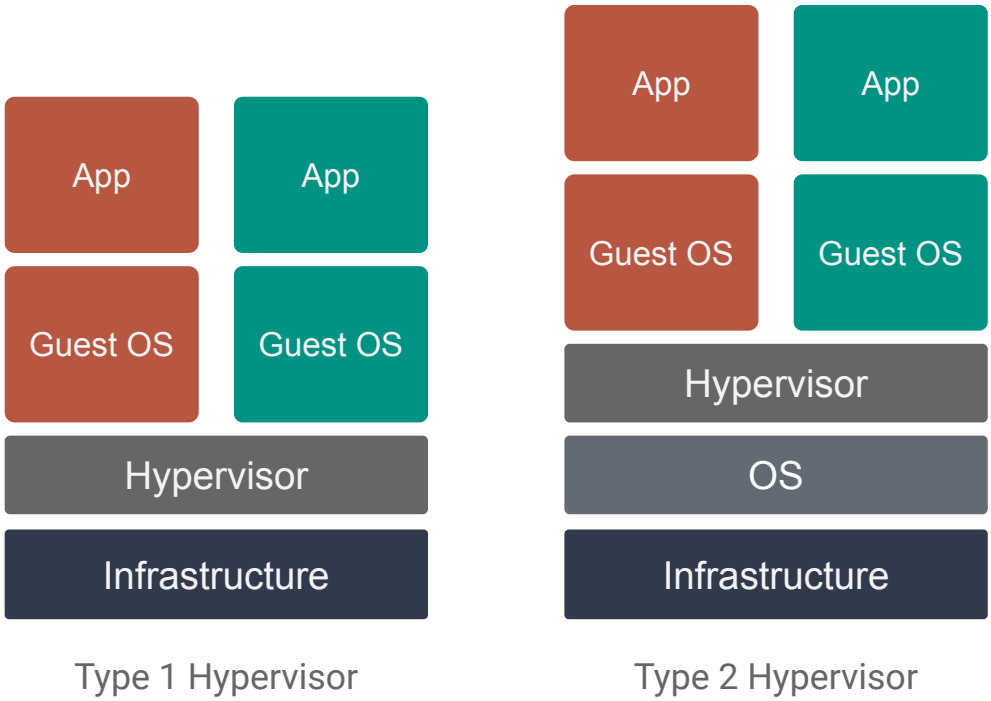

Figure 2.2: An overview of type 1 and type 2 hypervisors illustrating how a type 2 hypervisor requires an underlying OS, while a type 1 runs directly on top of the hardware.

it also means no operating system is required at all, as the hypervisor acts as an operating system by supporting all the operations of its VMs. Microkernelized hypervisors, on the other hand, do not require device drivers to be present in the hypervisor. Instead, the drivers are installed in the form of a manager VM, often referred to as parent VM. Each VM created by such a hypervisor is thus referred to as a child VM since it uses the parent VM's drivers to access the hardware. A microkernelized approach is more flexible than a monolithic, as it can allow the hypervisor to run on any hardware that supports the parent VM's operating system. This distinction is similar to type 1 vs type 2 hypervisor, with one being more self-sufficient and the other more modular. However, it is important to note that monolithic and micro kernelized hypervisors are more concerned with the inner technical aspects of a hypervisor rather than the usability for the end-user.

The concept of *security rings* is often used to describe the level of privilege different part of a hypervisor has. A security ring represents a hierarchical protection domain, where the innermost ring (ring 0), usually reserved for the OS kernel, has the highest privilege, and the outermost ring (ring 3), usually reserved for user applications, has the lowest privilege. The hypervisor itself is often placed in ring 0, its VM kernel space in ring 1, and the VM user space in ring 3. Security rings are thus used to define execution level to provide a system to limit the execution capabilities of VMs to regulate access to shared memory. The abstraction leads to more transparency regarding the execution level of certain applications and is key for ensuring that a VM maintains its isolation [25, 26]. Figure 2.3 illustrates this concept as a layered group of rings.

Key Hypervisors

There are several hypervisors available, each with its own set of features and capabilities, some of which are listed below.

- **VMware ESXi**
  A type 1 monolithic hypervisor developed by VMware. It is often used in enterprise environments and is designed to use minimal system resources [25]. Being a type 1 hypervisor, it does not require an underlying OS to run.

- **Microsoft Hyper-V**
  A type 1 microkernelized hypervisor is used in the Window Servers suite. Its microkernelized properties stem from having a parent and child

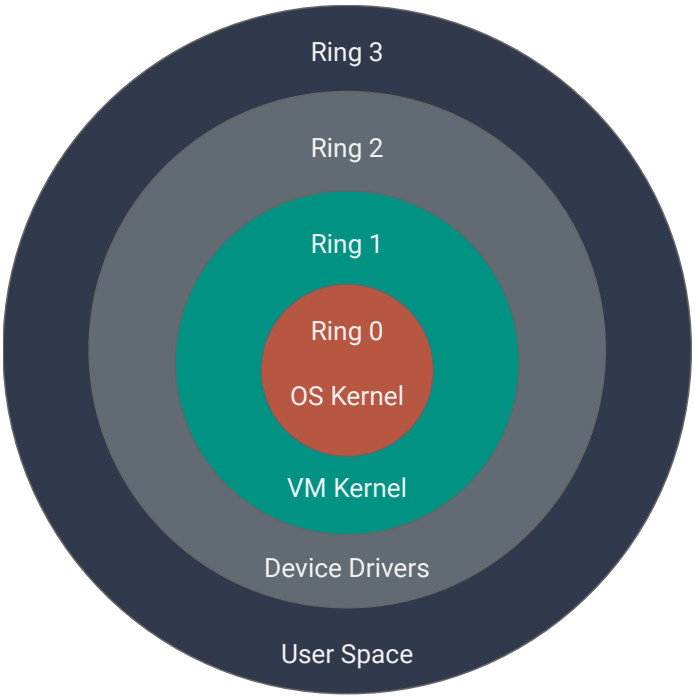

Figure 2.3: Overview of security rings in a hypervisor. The figure illustrates how the inner rings have higher privileges than the outer rings.

relationship between its components [25, 27]. Being a type 1 hypervisor, it does not require an underlying OS to run.

- **Citrix Hypervisor**
  A type 1 microkernelized hypervisor that was previously known as XenServer. Its centralized parts consist of a modified Linux kernel called Dom0, which takes care of I/O operations and underlying hardware access, as well as creating other , called DomU guests [22, 25]. Being a type 1 hypervisor, it does not require an underlying OS to run.

- **Kernel-based Virtual Machine (KVM)**
  A type 1 hypervisor that is neither microkernelized nor monolithic. Instead, KVM is built into the Linux kernel and uses many of its functionalities such as the scheduler and memory management. KVM is also free and open-source, which has made it a popular alternative to proprietary hypervisors [25, 28].

2.3 VM Management Essentials

To provide a control plane when managing VMs, many VM management systems were created. These systems comprise tools and mechanisms for managing the entire life-cycle of VMs, including creation, deletion and access. This makes a VM management system comparable to a hypervisor, in that they both manage a set of VMs. However, a VM management system operates both on a high level and on a larger scale by spanning over multiple nodes. As such, it often supports a wide variety of hypervisors among its nodes [29]. However, certain hypervisor features, such as live migration, could have unique implementations for different hypervisors. This creates a possible trade-off for a VM management system where it can choose to implement fewer but homogeneous operations, or more tunable operations that implement hypervisor-specific features [30].

Concepts and Features

With a higher abstraction layer, a VM management system brings a set of concepts to simplify the management of VMs. Many such features and concepts exist and can be broadly grouped as clustering, availability and scalability [29].

Clustering is a common mechanism found in VM management system that involves grouping a set of resources such that they are perceived as a bigger

unit. This is useful in scenarios where a workload is too large for a unit. A VM management system can provide tools for grouping VMs in various forms. A common example of clustering computing resources is High-Performance Computing (HPC), which is often used to process larger datasets by dividing larger computations into fewer but smaller units. However, clustering can also refer to placing a group of VMs behind a load balancer that distributes the load evenly [31, 32].

Availability is another concept tied to a system managing VMs and is concerned with how well a service can be kept running without disruption, even in the case of malfunctioning components. A common approach for ensuring availability is by providing Availability Zones (AZs), which is a type of design pattern where services are replicated in separate environments such that an environment can be kept running despite another one failing. For this reason, AZs can span servers or even entire data centers [29, 33]. Figure 2.4 illustrates a scenario in which AZs provides redundant access to the same service.

Scalability is a concept that is not directly tied to the VMs themselves, but rather to how well the system scales up or down. Fundamentally, a system can scale either by adding additional nodes or by increasing the resource capacity of the nodes, such as the number of CPU cores. These methods are called scaling out and scaling up respectively. Scaling out is commonly referred to as the more scalable of the two. This is because, in practice, there is a stricter upper limit on the performance that can be gained by adding hardware resources to a single node. A scale-out operation also causes less downtime since the responsibility is not as large per server. However, a possible argument against scaling out is that certain applications might not be designed to operate in several instances, such as a single-node database [34, 35].

Cloud Service Models

In IT infrastructure, VMs are not the only component required, and VM management is instead part of a larger infrastructure that provides additional features such as networking and storage. These are referred to as an Infrastructure as a Service (IaaS) platform, and they form the foundation for managing virtual infrastructure [36].

IaaS is a subset of a larger group called cloud service models, which also defines models such as Platform as a Service (PaaS) and Software as a Service (SaaS). Compared to IaaS, they provide a higher level of abstraction that is used either to build or distribute applications Together, the three cloud service models form a hierarchical relationship with each other, which means a SaaS

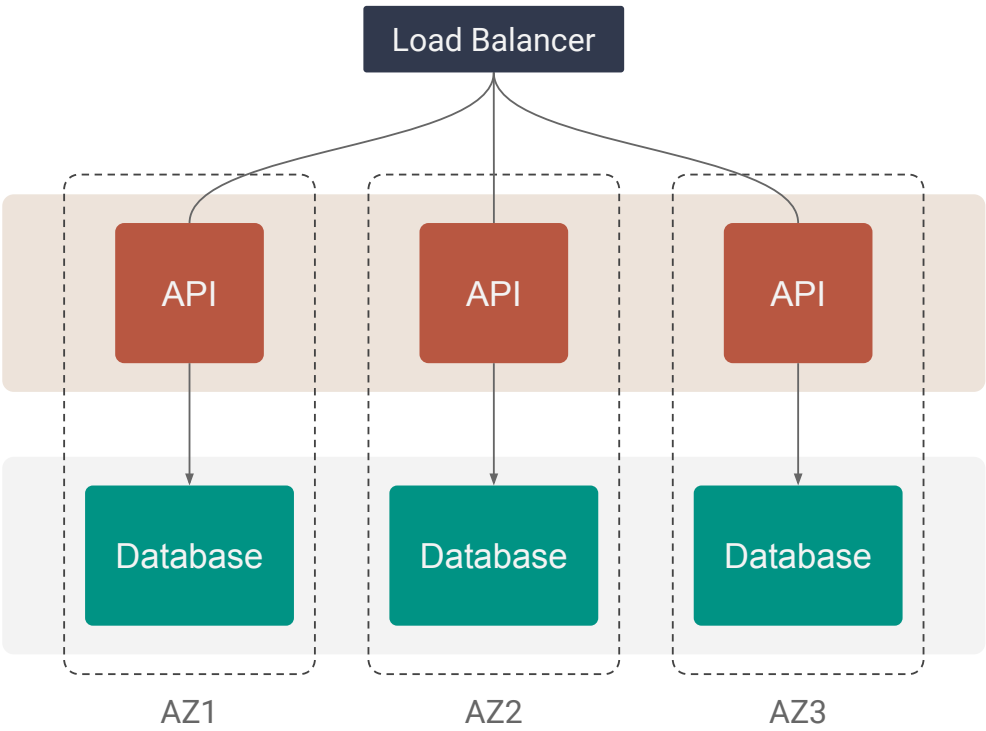

Figure 2.4: Overview of availability zones with several API gateways and databases operating in the same logical environment but in different availability zones to provide redundancy and fault tolerance.

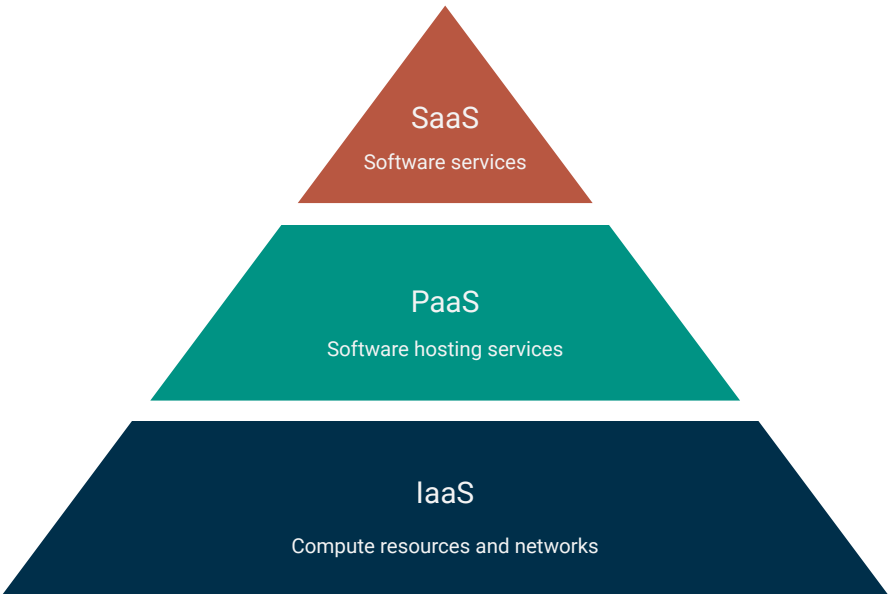

Figure 2.5: An overview of the relationship between cloud server models, where a SaaS platform can be built on top of a PaaS platform, which in turn can be built on top of an IaaS platform

service can be built on top of a PaaS service, which in turn can be built on top of an IaaS service. Figure 2.5 illustrates this relationship. Both PaaS and SaaS are out of the scope of this project [37].

2.3.1 Comparison of VM Management Systems

This far, managing VMs has been described as being multifaceted, where a management system should provide availability and scalability. However, there are additional factors when deciding on which management system to use, such as its cost or required skillset to manage.

In practice, there are mainly two types of systems offering VM management: public cloud and private cloud. A public cloud is a commercial platform entirely managed by a third-party company. These are characterized by being highly available and scalable, as well as having little to no up-front costs. A common denominator among public clouds is their pay-as-you-go model. A private cloud, on the other hand, refers to a piece of software that can be installed on a set of servers within an organization. This creates a clear trade-off where while an organization is responsible for managing servers and any data stored on them, it could lead to less expensive usage. Some of the most

prevalent private cloud platforms are OpenStack [38], CloudStack [39] and OpenNebula [30], and some of the most popular public cloud platforms are Amazon, Azure, and Google Cloud. A public cloud offers far more than just VM management. However, to keep the focus of the comparison, only the VM service of each public cloud will be brought up. These are Amazon EC2 [40], Azure Virtual Machines [41] and Google Compute Engine [42].

OpenStack

OpenStack is an open-source IaaS platform that provides a large set of tools for managing virtualized resources. Its architecture is built to be modular using a component-based approach with each of the services it provides running as a separate component. The components communicate in a distributed manner using technologies such as message queues [30] [38].

While OpenStack includes a large set of components, only the three major and most relevant to this project are brought up below.

- **Nova**
  The VM service that provides a set of tools for managing the life-cycle of VMs, such as creating, deleting and migrating VMs. Nova is a relatively large component that is split into multiple parts each with a *shared-nothing* and *messaging-based* policy, meaning they are independent and communicate with other components using message queues.

- **Swift**
  The service that manages static data in a scalable and redundant manner. This is used by *Nova* to store images and backups.

- **Glance**
  The image service that provides a catalog of VM images that can be used when creating new VMs.

In summary, OpenStack supports many features regarding VM management as well as a wide range of hypervisors. It is built to be efficient and scalable and has thus gained enormous traction in the industry [30].

CloudStack

CloudStack is an open-source IaaS platform and cloud computing software that manages virtualized resources including VMs, virtual networks and storage resources. It is used primarily to build and manage private cloud environments [39]. Architecture-wise, CloudStack is designed in a centralized

fashion with a management server and agents. The management server can be viewed as a controller node and is responsible for sending out instructions regarding the life-cycle of VMs, such as creating, scaling and deleting them. The agent, on the other hand, is installed on each node designated to running VMs and takes care of low-level management tasks related to the VMs. The agent can thus be viewed as an intermediator between the hypervisor and the management server that enables CloudStack to be hypervisor agnostic. While CloudStack is not designed to be as modular as OpenStack, it is designed to be horizontally scalable and can be deployed in a distributed manner with multiple management servers [39] [43].

CloudStack supports a large variety of hypervisors, such as VMware, KVM, and XenServer, as well as storage services [39]. In summary, CloudStack provides an efficient public cloud environment similar to OpenStack. However, CloudStack does not scale to the same size as OpenStack due to its simple design [30].

OpenNebula

OpenNebula is an open-source IaaS platform that is mainly used to build private clouds. Contrary to OpenStack, OpenNebula is built to be simpler with as little overhead as possible, which has made OpenNebula a common choice for larger-scale datacenters [30]. Its architecture is designed to be modular and is split into three parts as described below. [44, 45].

- **Tools layer**
  The high-level layer that provides a set of interfaces such as a Command Line Interface (CLI) and web-based user interface to interact with the system.

- **Core layer**
  Provides core functionalities to the system, such as host management, VM management and virtual network management.

- **Driver layer**
  The low-level layer that provides an abstraction on top of the underlying infrastructure, including networks, storage and hypervisors. Having this layer allows OpenNebula to support a wide range of providers, as well as adapt quickly to new technologies.

In summary, OpenNebula provides a simple yet efficient approach to VM management. Its simplicity and low overhead have made it a popular choice in the industry [12].

Amazon EC2

Amazon Elastic Compute (EC2) is a service that provides VMs management in the public cloud platform Amazon Web Services (AWS), and it is used to host general-purpose applications [40, 46]. Since AWS is a public cloud platform, the technical specifications regarding hypervisor technology and such are not visible to the end user. AWS provides a large set of templates that can be used when creating VMs, called Amazon Machine Images (AMIs) [40].

Azure Virtual Machines

Azure Virtual Machines is a service provided by the public cloud platform Azure to manage VMs [41, 47]. Similar to AWS, Azure provides a set of preconfigured templates that can be used when creating VMs are chosen depending on the use case, such as general-purpose or HPC [48]. Any technical specifications on the architecture of Azure Virtual Machines are not visible to the end user.

Google Compute Engine

Google Compute Engine (GCE) is a service provided by Google Cloud to manage in the cloud [49]. Similar to both AWS and Azure, Google Cloud provides a set of templates when creating VMs [42].

Comparison

OpenStack, OpenNebula, and CloudStack are all examples of software that can be used to build both private and public cloud platforms. A private cloud is an on-premise cloud that is hosted within an organization and is, more often than not, dedicated to internal use. These systems are also often free and open-source, making them a go-to tool for many organization that seeks to host internal services. The systems are also highly configurable to work on most existing infrastructure. However, being highly configurable could be seen as a double-edged sword since it leaves much of the configuration of the system to the user, such as preconfigured VM templates. [50, 51].

Amazon EC2, Azure Virtual Machines and Google Compute Engine are examples of VM management systems in public cloud platforms. They are designed with the end user in mind by providing a complete abstraction layer over the underlying hardware and hypervisors. This means that any configuration regarding the hypervisor and its settings is taken out of the equation. All three cloud platforms can be viewed as complete implementations of the previously

mentioned private cloud platforms, including hardware. For this reason, the public cloud platform has seen enormous traction in the industry. However, this also creates a trade-off where the user has less control and customization but instead gains a more user-friendly and complete system [50].

Private and public clouds both provide solutions for managing virtual resources and have their respective advantages. Public clouds scale better performance-wise since they make use of large data centers. However, it inevitably involves a third party which might not be suitable for certain use cases, such as for applications managing sensitive data. Private cloud, on the other hand, comes with a larger initial cost, both in terms of purchasing hardware but also to acquire the required skillset among the personnel managing the system. Which solution to choose depends on the specific needs of the organization, such as the level of control required, customization needs and security requirements [51].

2.4 Containers and Kubernetes: A Paradigm Shift

To provide a lightweight alternative to VMs while still being able to create virtual computing environments, containers were invented. They provide environments with less overhead, which makes them excellent candidates for applications employing a microservice architecture. Microservices is a software design pattern that uses a set of small, independent and loosely coupled applications to provide a scalable service with minimal downtime [52]. Moreover, similar to VMs, containers are often managed using a management platform. Depending on the use case, many such platforms exist including Kubernetes [13], Nomad [14] and Docker Swarm [15], where Kubernetes is the most popular due its ability scale and integrate with other services.

2.4.1 Containers Explained

Containers are a form of virtualization that offers a lightweight alternative to VMs. Unlike VMs, containers do not require a hypervisor to run and instead run on top of an OS and share the OS kernel with other containers. By not requiring a hypervisor, containers can run with less overhead than VMs, making their life cycle relatively fast and efficient compared to VMs [6, 53]. The differences between container and hypervisor virtualization are visualized in figure 2.6.

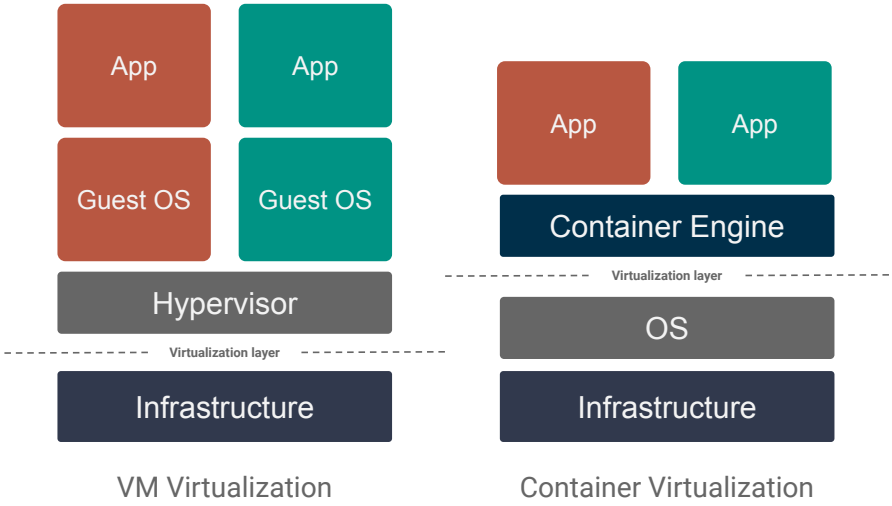

Figure 2.6: Overview of VMs and containers. The virtualization layer is placed on a lower level for VMs than for containers, illustrating the difference in their respective virtualization techniques.

A container is created from a container image, which is a lightweight package that is used when configuring the virtual environment for the container. It encapsulates the application together with any dependencies such that it runs on any platform, which makes the container itself portable and consistent. The image itself is created using a configuration file called *Dockerfile**, which describes a parent image, such as an OS image, together with several steps required to produce the container image. Once the container image is created, it can be used as a parent image in any other configuration files, creating a chain of dependencies.

To enable caching, a container image can be divided into a series of layers. Each layer represents a change to the parent image, such as adding a file or installing a package, and thus acts as a checkpoint. This enables container images to be built incrementally, where layers are only rebuilt when a change is made [54, 55, 56]. Figure 2.7 illustrates the concept of container image layers, where each operation represents a change to the parent image.

Moreover, container images are often stored in container registries, which provide central storage and are used to distribute the images. A container registry often comes with tools to manage its images, such as tools for version control and scanning for malicious images. Another common event-based feature is webhook. It acts as a message broker for predefined events, such as notifying an external system that a new image was pushed. Together these features

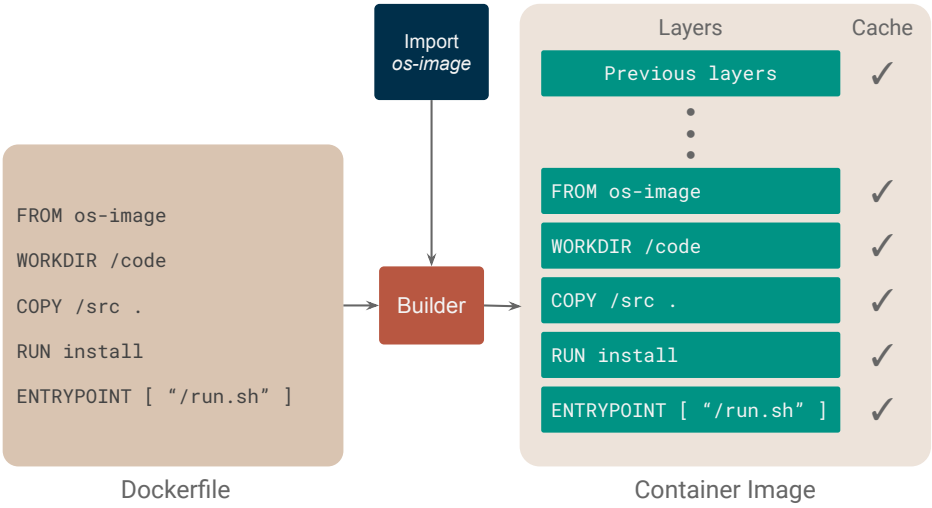

Figure 2.7: Overview of container image layers where each operation in a Dockerfile represents a layer that can be cached and reused. The layers are stacked on top of each other and appended to the parent image to form the final container image.

enable a container registry to integrate well in a cloud-native environment and development pipelines [57, 58].

Originally, containers were designed to run on top of a technology developed by the company that invented containers called the *Docker Engine*. Docker Engine is a container runtime that creates and manages containers similar to a hypervisor. However, to standardize an interface for container communication, the Open Container Initiative (OCI) was created, which defines a group of standards that aims to make containers portable across different platforms [59]. It is split into three standards called the runtime specification, image specification and distribution specification. From these specifications, the *containerd* runtime was created, which aims to provide a more transparent and open-source alternative to the Docker Engine. Since it adheres to the standards, it allows for simpler integration with tools and services built on top of it. Today, many large systems use containerd under the hood, such as Kubernetes [60]

* Recently, some movement has been made to change the name to *Containerfile*, as this would describe the content better than the company that invented the technology. However, *Dockerfile* is still the most widely used term.

2.4.2 Orchestration Using Kubernetes

Kubernetes is an open-source container orchestration platform that manages containers. It uses a set of abstraction layers to manage the life cycle of containers and all the communication between them. It provides a complete environment for the containers to run in, including scheduling, scaling and networking [6].

Kubernetes defines many abstractions to simplify container orchestration and communication, where the most basic ones are *Pods* and *Services*. A Pod is the smallest deployable unit in Kubernetes, which groups one or more containers into a single unit that can be run on nodes. This means that Kubernetes ensures all containers in a Pod run on the same host. The encapsulation allows coupled containers to reside on the same host and enables efficient communication between them. A common design pattern for containers in a Pod is the sidecar design pattern. It involves designating one or more containers to provide some additional functionality to a main container, such as logging or monitoring [61, 62]. A Service represents a communication entry point that is placed in front of one or more Pods. This means that a Service targetting a group of nodes acts as a load balancer, while a service that targets a single Pod is comparable to port-forwarding. Separating the communication into a separate layer is especially useful in scenarios where the Pods can serve client requests independently, and that the requesting client does not care which Pod it targets. Such Pods are referred to as *fungible* Pods and are more common for cloud-native applications. Additionally, using Services in front of a group of Pods can lead to simpler deployment of updates, as the Pods can be updated one at a time using a rolling update strategy [63].

To enable a more consistent deployment of Pods, Kubernetes provides an abstraction on top of the Pods called *Deployments*. A Deployment is given a desired state in which Pods should be and actively operates towards it. This means that if a Pod fails, the number of running Pods no longer matches the desired number of Pods, and the Deployment should create a new Pod. By following a desired state, Deployments are commonly used when the load of an application is irregular. For instance, a website might face heavier traffic during the day than during the night. In this scenario, the Deployment could be adjusted to scale down the number of Pods [64, 60].

So far, none of the scenarios have included a state that should be kept between or within Pods. Containers are inherently stateless by design, meaning that any data inside a container is lost upon deletion. Kubernetes primarily provides three abstractions to manage the state of an application: *Per-*

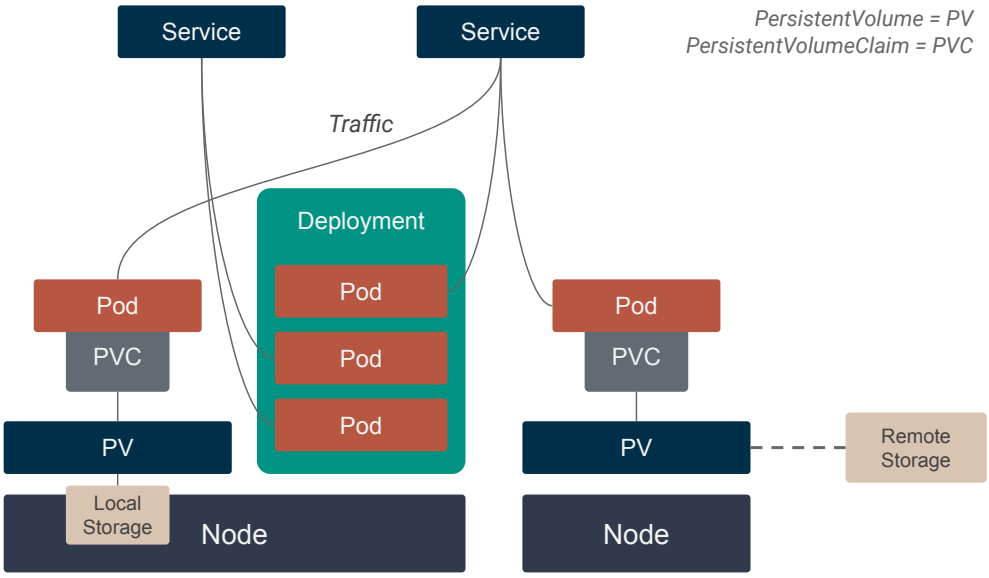

Figure 2.8: Brief overview of Kubernetes abstractions including many of the core components. The figure illustrates the communication between the different components and how storage is managed in Kubernetes.

sistentVolume, *PersistentVolumeClaim* and *StatefulSet*. The former two are used in conjunction, where a PersistentVolume defines a storage target, such as the file path in a network storage server or on the local node, while a PersistentVolumeClaim acts as the intermediator between a PersistentVolume and a Pod. There is a one-to-one relationship between a PersistentVolume and PersistentVolumeClaim to ensure consistent access to the underlying storage, but also to provide a mechanism to notify Kubernetes that storage is no longer used [65]. A StatefulSet, on the other hand, is more comparable to a Deployment with a specific order for the Pods to start in. As previously described, a Deployment ensures that a desired number of Pods are running, but it does not guarantee the order in which they start or stop. A StatefulSet solves this by providing a more predictable strategy where each Pod has its own identity that is preserved between restarts. For this reason, a StatefulSet is often used for applications that define roles for a group of nodes, such as a database that must start a primary node before any secondary nodes [66, 60].

Figure 2.8 presents a brief overview of the Kubernetes abstractions and how they interact with one another.

In some more sophisticated scenarios, these built-in abstractions might not suffice since they might not be able to model more complex types. For such

scenarios, Kubernetes provides Custom Resource Definitions (CRDs), a form of extension, that allows user-defined types with logic and validation. For instance, a CRD can define an abstraction similar to a Deployment, but it enforces certain sidecar containers to exist using custom validation rules. With CRDs, the Kubernetes ecosystem can be tailored for specific use cases forming a more feature-rich environment [67, 60, 68]. Kubernetes includes many more abstractions, each with its use case. However, presenting an exhaustive list falls out of the scope of this project.

The design and workflow of Kubernetes resources are inherently declarative, meaning that instead of providing a set of instructions to move a resource into a specific state, a complete state is given in the form of configuration files called manifests. The manifest defines metadata, relationships and dependencies for a resource, and Kubernetes controllers are responsible for ensuring this state is applied and up to date [69]. Depending on the kind of abstraction, different logic is applied to ensure the resource specification matches the manifest. For instance, a Pod might be rescheduled to ensure it matches the manifest [6, 60].

Architecture-wise, Kubernetes uses a client-server approach. The server, often referred to as the control plane, manages the state of the cluster, including scheduling and scaling, as well as networking. Since it consists of the API server, it can be viewed as the gateway to cluster, processing and acting upon the request of clients [70]. The client, often referred to as the worker node, receives Pod-related instructions from the control plane. Contrary to the control plane, a worker node consists of a container runtime, which allows it to run containers [60, 71, 72].

From an end user's perspective, Kubernetes resources are managed through the API server, which in turn notifies its workers. An end user can send requests using a CLI tool such as *kubectl* or using a programmatic interface through a REST API [60].

Through its abstractions, life cycle management and declarative client-server architecture, Kubernetes provides a complete and feature-rich platform for container orchestration. This has led to enormous traction in the industry and has Kubernetes a popular choice for container infrastructure [16].

2.5 Containers vs. Virtual Machines

Virtualization has brought many benefits within the realm of cloud computing by allowing hardware to be split or grouped into virtual hardware. It has been the key in enabling what is today referred to as cloud computing and cloud

platforms. There are mainly two technologies that were made possible due to virtualization: VM and containers. VMs are complete and isolated systems with their own OS and containers are a lightweight alternative to VMs. This creates a trade-off where depending on the use case, a VM might be more suitable or vice versa [6, 73].

2.5.1 Comparative Overview

The main difference between VMs and containers revolves around isolation and how independently they can run. As previously described VMs provides a complete system while containers share the OS kernel with other containers. This can lead to more efficient resource sharing among the containers, but it also increases the attack surface of containers. A vulnerability found in the underlying OS might compromise the containers running on the host. A VM reduces this risk by removing the OS as a dependency [74].

Isolation is also important when considering the scenario of co-location, meaning containers and VMs running on the same host. Since containers are allocated resources from the OS, they are allocated as if they were ordinary applications. In many scenarios, this leads to efficient resource utilization, but sharing resources this way can be a double-edged sword. For instance, containers are more vulnerable than VMs to the noisy neighbor problem, since a co-located container could consume an unproportionate amount of the host's resources, which could lead to resource starvation among the other containers running on the host. Some solutions exist for this, such as the OS-level feature *cgroups*. A cgroup is a mechanism that limits, prioritizes and isolates resource usage for a process [75, 76, 77]. VMs, on the other hand, are not as susceptible to the noisy neighbor problem since they are allocated a static amount of resources from the host. However, it is important to note that a VM does not reduce the noisy neighbor problem for all resource types. For instance, unproportionate consumption of network traffic is still an issue for both VMs and containers [73].

Overcommitment refers to a mechanism that allocates more virtual resources than what is physically available. This is a common practice when using virtualization, as hardware resources are often not fully utilized. Containers use mechanisms built into the OSto achieve resource overcommitment, which involves multiplexing and virtual memory. However, resource overcommitment is more challenging for VMs. While CPU overcommitment is achieved using VCPU, memory overcommitment is solved by more sophisticated methods since a hypervisor allocates a static amount of memory to each

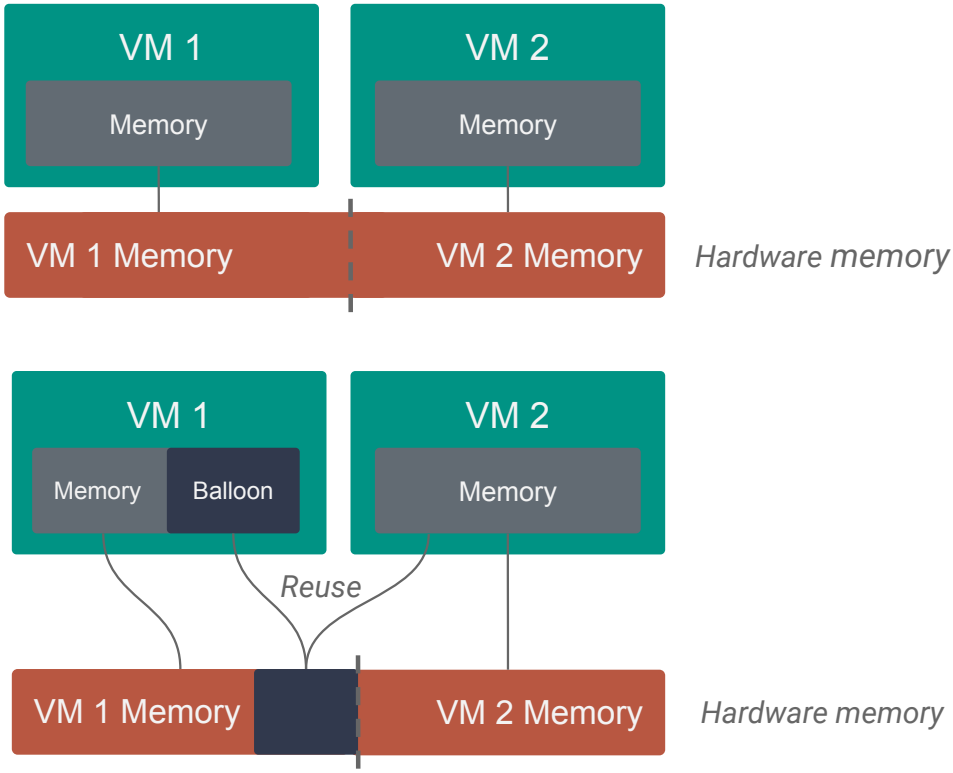

Figure 2.9: Overview of memory ballooning in two stages. The figure illustrates the memory of a VM is shrunk by the hypervisor (deflation) and then reused by another VM (inflation).

VM. One such method is called memory ballooning and it involves sharing a subset of each VM's memory with other VMs. It works by reclaiming memory that is not used inside a VM and allocating it to another VM. Then, when the first VM requires the now reclaimed memory, the hypervisor ensures the VM is either given back its memory or allocated a new chunk. As such, memory ballooning operates under the assumption that certain parts of the allocated memory are not used [78]. However, it is important to note that a technique like memory ballooning does not directly solve memory overcommitment, but instead reduces the need for it [73, 79]. Figure 2.9 illustrates the processes of deflating the memory of a VM and how it is used in another VM.

Live migration is a technique used to move a running resource to a new host without stopping and starting it. The technique is often used when balancing load over hosts or when performing maintenance. For VMs, live migration is a mature and widely used technique and works by copying memory pages from the source host to the destination host and then transferring the VM from

the source hypervisor to the destination hypervisor. Since VM disks are often stored on shared storage, such as a Storage Area Network (SAN), the VM disk is not transferred but instead reattached after the live migration is finished. This usually requires some form of time-freezing to reduce the risk of data corruption. However, a live migration operation is usually fast enough to even notice [80]. Live migration for containers, however, is not as mature as for VMs. A container requires OS-level process migration, which is less reliable. However, several projects aim to provide live migration for containers, such as Checkpoint-restore in Userspace (CRUI). CRUI works by checkpointing the state of a container, including its memory and file descriptors, and then restoring the state on another host. While this is considered as the defacto container live migration, it is more comparable to a snapshot operation than a live migration operation [81, 82, 83]. Another approach for live migrating containers involves terminating them on one host and restarting them on another. This strategy assumes the container is stateless or utilizes shared storage for its state, as is often used in Kubernetes since it does not provide container live migration. Instead, Kubernetes facilitates persistent storage through its persistent storage abstractions, such as PersistentVolumes and PersistentVolumeClaims. This means a user maps portions of the container's state to shared storage and assumes the rest can be discarded when a container is terminated [60].

2.5.2 Decision Factors

The choice between containers and VMs depends on the specific use case, and there are several factors to consider when deciding which technology to use. Some of the core factors are security, scalability and legacy support [73].

As previously described, using a container implies a larger attack surface than for a VM due to sharing the OS kernel. For this reason, VM have commonly been associated with stronger security since they are less prone to security vulnerabilities. This makes VMs an excellent candidate for application with stricter security requirements. Some examples include applications managing sensitive or confidential data [84]. Another use case that is made possible due to a VM's strong isolation is running unsafe or untrusted code. This use case is often referred to as sandbox and is useful when evaluating potentially malicious code that could otherwise risk harming the host system. Untrusted code could also involve user-submitted code, such as in GitHub pipelines that use VMs to execute actions based on some predefined event. This could be tests or some other development-related tasks created by a user [85]. Using a VM in these scenarios ensures that the untrusted code does not affect the

hosting system [86, 87]

Configuration drift is another aspect to consider when deciding between using a VM or a container. It is a phenomenon that occurs when the configuration of a system changes over time, such that it could potentially lead to security risks. This is usually caused by continuous manual intervention in a system, where the summarized effects of all the changes made could be an unstable system [88, 89]. This is more common for VMs than for containers, as a VM's state is saved indefinitely. A container mitigates this problem by having its entire state defined in the Dockerfile (see 2.4.1), where its dependencies and configuration are more transparent to the person managing it. A container state is thus discarded once it is stopped, which allows containers to have a more consistent environment than VMs. This is further supported by the fact container registries (a central location for container images, see 2.4.1) usually include image-scanning tools to find vulnerabilities in a container image itself [90].

Scalability is a factor that is concerned with how well a system adapts to an increase or decrease in load. This can be measured in many ways where one being the overhead each additional computational unit has. Containers being lightweight and sharing hardware resources more efficiently implies they can scale up and down both quicker but also with a smaller resource footprint. However, as previously described, they are more susceptible to the noisy neighbor problem, where co-located containers could interfere [77, 90]. Moreover, scalability is also related to cloud-native applications, which are applications that are designed to run in the cloud [7]. A cloud-native application is characterized by being horizontally scalable, stateless, and resilient, and is often built using microservices, see 2.4. Cloud-native applications thus require small and lightweight environments making containers a natural fit.

Legacy support is another relevant factor to consider when comparing VMs and containers. It can be measured by how well a system can keep running old software or software depending on older hardware. In both cases, VMs is a good fit since they are a complete system. A VM can simply replicate the state of a legacy system, even including the hardware that was used. This allows an organization to acquire new infrastructure while still maintaining its legacy applications. Containers are also able to support legacy applications, but not to the same extent, as they are inevitably using a modern kernel [90, 91].

2.5.3 Towards Stronger Container Isolation

So far, containers have been described as having weaker isolation than VMs due to it sharing the underlying OS kernel. However. several projects have been created to tackle this issue without completely resorting to a VM. These include Firecracker, Kata Containers and gVisor.

Firecracker is a KVM-based hypervisor that creates lightweight VMs, called microVMs. Compared to an ordinary VM, a microVM is much more lightweight and can start in a fraction of a second. Firecracker achieves such speeds through optimizations and a heavy stripped-down guest [8, 92].

Kata Container is another VM alternative that aims to provide a lightweight environment, while still having strong isolation properties. Unlike Firecracker, a Kata Container still uses a container, but wraps it using a lightweight version of a VM, which enables Kata Container to act as a runtime, such as containerd. This means Kata Container is simpler to implement than Firecracker since it can run ordinary containers [8].

Another approach to strong container isolation is gVisor, an extension to a container that enforces stricter isolation by restricting system calls. This approach differs from both Firecracker and Kata Container in that it does not use a VM at all, and instead relies on intercepting system calls and file access requests [93].

In summary, these three technologies provide different approaches to the same problem. Firecracker is more VM-like, while gVisor is most similar to a container. However, due to the varying added overhead, they are yet to be widely adopted [8]. Figure 2.10 illustrates the differences between how Kata Containers and gVisor wrap a container to provide a more secure execution environment.

2.6 Extending Kubernetes with KubeVirt

Kubernetes was recently extended to accommodate VMs as a native resource through the KubeVirt project. KubeVirt is designed to integrate well with the Kubernetes ecosystem and provide a similar interface to VMs as any other Kubernetes resource. VMs created with KubeVirt are scheduled and communicate as ordinary Pods, allowing them to blend into the environment [94]. This means that a Kubernetes resource that integrates with a Pod can also integrate with a KubeVirt VM. For instance, Services can expose a VM externally and PersistentVolumeClaims can maintain its persistent storage. While KubeVirt uses Pods to wrap a VM, a VM abstraction is implemented as a CRD.

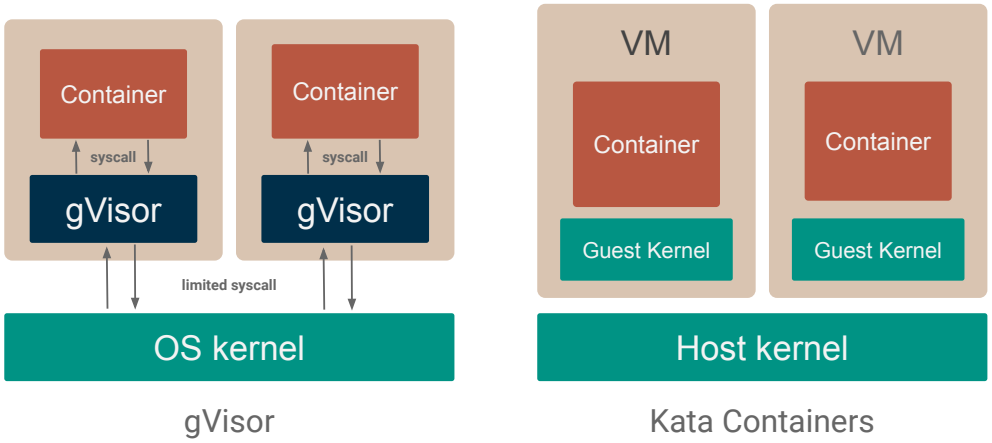

Figure 2.10: A overview of how gVisor and Kata Containers approaches container isolation differently by either using a proxy for system calls and file access or wrapping the container in a VM.

This allows KubeVirt to perform validation that is specific to a VM, such as referencing a non-existing disk [95, 96].

KubeVirt's design pattern follows Kubernetes conventions, using a declarative approach to managing the VMs and any other resources revolving around it, such as snapshots and live migration. A VM is split into a *VirtualMachine* and *VirtualMachineInstance*, where the VirtualMachine represents a desired state and the VirtualMachineInstance describes a running state. This relationship is similar to that of a StatefulSet and its Pods [97]. Moreover, KubeVirt implements a VM controller to ensure the desired state is met, which is responsible for requesting an environment in which the VM can run, but also update the running VM when its desired configuration changes. [98, 95].

KubeVirt's declarative approach stretches to aspects that have been imperative operations in traditional VM management systems. The most prominent examples of this are live migrations and snapshots that are both requested using configuration files. As such, they are implemented as CRD that triggers live migration or snapshot operations once created [99, 100]. This is a novel take on operations that traditionally have been imperative. However, the processes of triggering an operation upon resource creation have existed in Kubernetes for a long time. For instance, a storage device is connected to a Pod when a PersistentVolumeClaim is created [101].

A consequence of being built on top of Kubernetes is that KubeVirt-related resources can be managed using the same tools used for other resources, such as the CLI tool kubectl. However, certain operations are still unique to VMs

and are not yet supported by kubectl. This includes accessing a VM over Secure Shell (SSH) or Virtual Network Computing (VNC), which are inherently imperative operations. For this reason, KubeVirt provides a complementary CLI tool that implements these missing features [102].

2.6.1 KubeVirt Technical Overview

On a technical level, KubeVirt does not implement a new deployable unit for their VMs and was instead designed to encapsulate them using Pods (see 2.4.2). This design has allowed KubeVirt VMs to integrate well with other Kubernetes resources such as Sevices and PersistentVolumeClaims.

KubeVirt can be split into four core components: *virt-controller*, *virt-handler*, *virt-launcher* and the *libvirt daemon*. The virt-controller and virt-handler are both concerned with managing the life cycle of the VMs, where the virt-controller is responsible for creating the Pod for the VM and the virt-handler is responsible for creating the VM itself. The virt-launcher manages the runtime environment for a VM making it responsible for requesting start and stop operations, as well as monitoring. Finally, the libvirt daemon (libvirtd) provides the interface to manage the VMs hosted by the underlying KVM hypervisor. The relationship between the three components is illustrated in figure 2.11, where the virt-controller operates closest to Kubernetes API-server, virt-handler operates on every node to communicate with the underlying hypervisor, and virt-launcher operates as a sidecar container in every Pod to manage the runtime environment of the VM [95, 98].

2.6.2 KubeVirt and GitOps

GitOps is a set of practices that enables using Git as a storage for the desired state of a system. Using a storage managed by Git allows resources to be version-controlled and audited. This approach keeps developers and maintainers in focus since it allows their resources and codebase to exist in the same system. For some scenarios, the resources managed in Git might even be the same resources that host their application [103]. For this reason, GitOps becomes a natural fit for KubeVirt since VMs are managed in files [104].

2.6.3 Comparison with Traditional Approaches

So far, KubeVirt has been presented as a silver bullet for VM management. However, it is important to note that KubeVirt does not represent a new virtualization technology, but rather a unifying technology. Behind the hood,

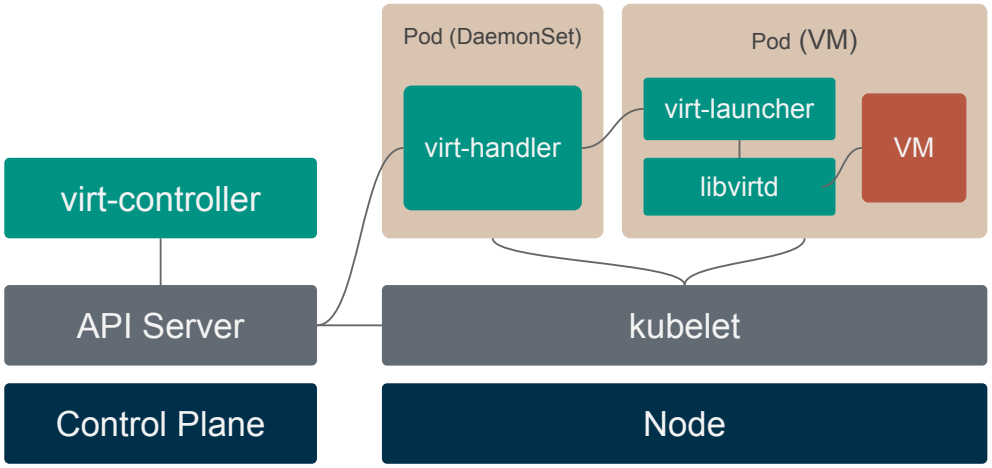

Figure 2.11: Overview of KubeVirt components, their logical place in the Kubernetes ecosystem and how they interact with each other. The green components represent the KubeVirt internal components.

KubeVirt uses Quick EMUlator (QEMU) and KVM to create the underlying VM, which is similar to any other traditional VM management system. This means that KubeVirt should in theory provide the same level of isolation and performance as any other management system. However, KubeVirt does fall short in that the VM is encapsulated inside a Pod, as this could lead to a performance overhead. Another aspect to highlight is that KubeVirt itself is not directly comparable to existing VM management systems, but rather forms a fully-fledged system when evaluated together with Kubernetes. For example, KubeVirt's implementation does not implement its scheduler or storage solution and instead relies on Kubernetes implementation and integrations, while OpenStack has its own scheduler as well wide support for different storage solutions including data warehousing [105, 95, 106, 107].

2.7 Related Work

2.7.1 Current challenges

The Cloud Native Computing Foundation (CNCF) presents in a report some of the current challenges of managing VM using traditional approaches, such as an IaaS platform [18]. They argue that a platform such as OpenStack is overly complex and difficult to manage for an organization that only wants to manage VMs. They conclude by suggesting KubeVirt with Kubernetes as a

simpler alternative to VM management.

Moreover, the CNCF report also mentions the potential benefits of a unified platform for both VMs and containers. They argue that a unified platform reduces the required skillset and management burden for personnel, as well as reducing the architectural complexity of the system. These benefits are supported in a report by Kubacki [19]. He argues that managing both a traditional VM management system and a Kubernetes could lead to unnecessary costs for an organization. Additionally, he argues that by allowing VMs and containers to exist in a single platform VM-only organizations could potentially be driven toward adopting containers for suitable workloads.

2.7.2 KubeVirt challenges

While KubeVirt is presented as a solution to many challenges related to managing VMs, it is important to consider the challenges that come with KubeVirt. A study by Trakulmaiphol and Piromsopa presents some shortcomings when moving into a KubeVirt-based system and found some issues related to how networking [108]. One of the issues was related to how IP address management differs in Kubernetes compared to traditional VM management systems. For instance, static IP addresses were difficult to assign and manage. In the study, they developed a framework to solve some of the issues on the application level, such as DNS queries, and provided a set of guidelines related to KubeVirt's unique features including the Multus Container Network Interface.

A common practice when managing Kubernetes clusters is to deploy Kubernetes nodes on VMs created by a VM management system. This means that when using KubeVirt, VMs are created inside other VMs leading to nested virtualization. Ye et al. argue in a study that nested virtualization raises security concerns since it increases the size of the [109]. A larger means that the size of the system's attack vector increases, which in turn leads to an increased risk for vulnerabilities. This study is particularly relevant as it highlights a potential limitation if KubeVirt is simply seen as another Kubernetes resource and not as a VM management system.

2.7.3 Usability of Kubernetes and KubeVirt

Using Kubernetes as a VM management system is a relatively new concept and there is limited research on the usability of KubeVirt. However, several studies have evaluated the usability of Kubernetes, which is equally relevant since KubeVirt is built on top of Kubernetes.

Kubernetes API is inherently declarative, meaning it uses a configuration to describe a desired state it should be in. A study by Sonninen investigates many of the possible benefits of such an approach [110]. The study found that a declarative approach enables a system to use GitOps, which is a way to manage infrastructure using files in storage backed by Git. This simplifies resource management through its built-in version control and audit, as well as rollbacks (given that it is supported by the resource). Two studies exemplify these benefits when presenting their case study on when CERN migrated an internal project from a cluster of VM to Kubernetes. The first study conducted by Imran et al. describes how migrating to Kubernetes improved the overall system management compared to using a VM cluster [111]. While many of their experienced benefits came from adopting containers instead of VMs, the study still suggests Kubernetes itself provided many benefits, such as reduced operational costs. The other study conducted by Badran presents the practical benefits they experienced by using Kubernetes and is more focused on the adoption of a declarative approach for system management [112]. It describes how using Kubernetes instead of a VM cluster allowed them to create reusable configurations that enabled their entire infrastructure to be managed as code.

The benefits of moving into Kubernetes are further exemplified in a study by Gelley [113]. It presents a case study where an organization moved its application from a PaaS provided by Cloud Foundry into Kubernetes. The study not only describes how Kubernetes provides more flexible configuration and deployment compared to the previous PaaS but also how the organization was able to make use of the tools Kubernetes provides to enable auto-scaling and monitoring. Moreover, the study concludes that by moving into Kubernetes, possible future migration to other platforms was made simpler due to how widely adopted Kubernetes is.

User experience in Kubernetes was further investigated in a study by Packard et al. that collected several projects that moved into Kubernetes [114]. The study covered many aspects, including administration such as managing persistent storage and quotas, but also developer experience such as deploying applications. The study found that system administrators in general face a relatively steep learning curve when using Kubernetes since they need to learn many of the concepts that make up an application. Interestingly, it also found that many users without prior knowledge of Kubernetes could perform basic tasks without any major issues, such as deploying an application. This finding is especially useful when evaluating KubeVirt since the ability to deploy a built-in resource in Kubernetes is similar, if not identical, to deploying a KubeVirt-based resource. Another study that investigated the user experience

of Kubernetes was conducted by Boronin [115]. The study presents a case study where an organization migrated from a monolithic system design to a more cloud-native design. The study is relevant as it is not only concerned with the technical aspects of Kubernetes but also considers the organizational impact of migrating to Kubernetes. For instance, transferring skill sets among personnel and the impact on the organization's culture.

2.7.4 Performance Evaluations

Regarding performance, several studies have evaluated popular systems that can manage VMs, including open-source alternatives such as OpenStack and CloudStack, or proprietary systems such as Amazon EC2.

A study by Kang et al. compared the performance of OpenStack and Amazon EC2 using a wide range of configurations and analyzed diverse aspects based on different hypervisor, storage and networking configurations [116]. Apart from the performance comparisons, the study also investigated performance impact when using more advanced features, such as Infiniband networking instead of Ethernet. The study showed mixed results, where OpenStack outperforms EC2 in some scenarios such as when using Infiniband networking, but EC2 tends to outperform OpenStack in parallel processing. As such, the study concluded that component-wise, each system has strengths. This study brings with it relevant perspectives as it highlights many of the aspects that are important to consider when evaluating the performance of VM management systems.

Another study by Mullerikkal and Sastri compared OpenStack and CloudStack regarding implementation complexity, stability and performance [117]. Compared to the previous study, these performance tests were designed to measure disk and network I/O, and not CPU or memory usage. The study found that OpenStack was significantly more complex than CloudStack, but that it was overall more stable. The study also found that OpenStack had a slightly but overall better performance than CloudStack. These claims are further supported by another study conducted by Paradowski et al. [118] a few years later that also compared CloudStack and OpenStack. The study benchmarked the two systems in terms of practical tasks such as time to deploy a VM and resource utilization during the deployment. The study found that OpenStack was slightly faster in most operations and had a lower resource utilization. While the tests were designed to represent user interactions, their scale was relatively small and did not explore any limitations of either system.

In contrast, a later study by Amaral benchmarked KubeVirt uses a setup

with a large number of VMs relative to the size of the nodes, aiming to explore the limits of KubeVirt's control plane (scheduler etc.) [119]. The study found that, while KubeVirt was able to create all VMs, the control plane suffered a heavy load, indicating a possible large overhead when KubeVirt provisions a large number of VMs. This finding is important to consider when evaluating KubeVirt, as it indicates that KubeVirt may not yet be suitable for managing a large number of VMs.

In terms of network performance, a study conducted by Kolahdouzan investigated the networking traffic pattern in KubeVirt and OpenStack by performing networking performance tests to evaluate potential VM overhead [120]. The study found that the network overhead was significantly larger for scenarios in which VMs reside on separate nodes. However, while the study found overhead for both KubeVirt and OpenStack, it found that OpenStack had a considerably larger overhead. While this project will not focus on networking performance, it is important to consider the findings of this study when evaluating KubeVirt.

So far, only related work that evaluated higher-level systems has been presented. However, several studies have evaluated the performance of hypervisors. A study by Hwang et al. evaluated the performance of a range of hypervisors including KVM, Hyper-V and Xen [121]. The study did not manage to draw any conclusions on which hypervisor was most performant and instead concluded that it depended on the specific use case. This conclusion is interesting as it suggests that the performance of a hypervisor could be an obsolete factor when deciding which one to use. Either way, reviewing hypervisors is important when evaluating KubeVirt, as KubeVirt uses KVM as its hypervisor.

Chapter 3

Performance Analysis

As described in the literature study, Paradowski et al. presented a set of benchmarks used to evaluate CloudStack and OpenStack [118]. This study was used as a base when designing the performance analysis used in this project. Instead of comparing CloudStack with OpenStack, this project evaluated OpenNebula and KubeVirt. Choosing OpenNebula as the second system was based on its popularity and the fact it is open-source. OpenNebula also happened to be the current installation at Net Insight, which provided a production-ready environment when needed.

3.1 Setup

The testing process was conducted on Microsoft Azure, both due to its availability when tests could be run at any time, and also to reduce the risk of noise in the measurements that could be more prominent in a local setup. The performance analysis was thus hosted on a group of VMs in the cloud platform.

To enable the results to be easily reproducible, the setup and testing procedures were automated using a programmatic approach. This meant that the script created complete and operable environments for both KubeVirt and OpenNebula, including deploying the nodes in Azure, management system installation and configuration. Additionally, the scripts collected the results of the tests and stored them locally on the requesting computer, and then deleted the Azure VMs. Before moving on, it is important to distinguish between the VMs created in Azure and the VMs created by the VMs management systems, since this setup is essentially VMs running inside other VMs. For this reason, Azure VMs will from now on be referred to as *hosting nodes*.

Payload-wise, this project used a similar approach to the study by Parad-

owski et al. with predefined types for VMs with CPU ranging from 1 to 4 CPU cores, and RAM ranging from 128 MB to 4 GB (see Table 3.4 for a detailed overview). Moreover, VMs used the lightweight Cirros Linux image since a lightweight OS image is less likely to introduce noise in measurements [122]. This also meant a VM disk could be smaller, which allowed for more VMs to be created per node and for tests to run quicker since a lightweight image is usually faster to boot than a larger image.

Each test was run on three hosting nodes, one management node and two worker nodes. This setup was the minimal setup required to be able to facilitate the tests, as some tests such as the live migration ones, consisted of moving a VM between two workers. The hosting nodes were configured large enough to be able to run a variety of tests, including tests with many small VMs or few large VMs. However, it was also important to consider the cost of the setup since the hosting nodes came with a cost directly tied to their performance and capabilities. Azure offered many different VM types, many of which were suitable in terms of CPU and memory specification. However, only a few offered virtualization support, which was required when running nested VMs. This left the Azure Dv3-series as the most suitable VM type and, specifically, the D4v3 VM type that has 4 CPU cores and 16 GB of memory [123]. The price of the D4v3 VM type was 0.2 USD per hour, which was deemed acceptable for the project [124]. Table 3.1 shows the overview of the Azure node specification used in the tests.

Role	CPU	RAM	Disk
Control Plane (Azure D4v3)	4	16GB	100GB
Worker 1 (Azure D4v3)	4	16GB	100GB
Worker 2 (Azure D4v3)	4	16GB	100GB
Scale-up Workers 3-10 (Azure D4v3)	4	16GB	100GB

Table 3.1: Node specifications used for the nodes hosting the VM management systems.

Several metrics were collected on each hosting node to capture the utilization before, during and after the test was completed. Capturing for a brief moment before and after meant it could detect if some utilization was unrelated to a test. See Table 3.2 for a complete list of the metrics used. Further technical details on how metrics were collected are described later in this chapter.

The tests were designed for three different purposes: basic, limit testing and scaling. Basic tests were designed to evaluate the core functionality of a VM management system, such as creating and deleting VMs. These were

Metric	Unit	Description
CPU utilization	%	The percentage of CPU used by the system
Memory utilization	%	The percentage of memory used by the system
Disk utilization	%	The percentage of disk used by the system
Task duration	s	The duration of a specific task

Table 3.2: Performance metrics that were used to quantify the results of the tests.

the initial tests since the following tests were partially based on them. A limit testing category was added since the study by Paradowski did not consider this factor. These tests are designed to evaluate how a management system handles a burst of operations, such as creating many small VMs. Finally, scaling tests were designed to manage operations unrelated to VM creation and instead focused on how well the management system itself can scale. See table 3.3 for an overview of the tests and their purpose.

3.2 Building a Framework

A VM management system is often comprised of multiple services often running on separate nodes, and this project is no different since it uses a multi-node setup for the tests. This means that to accurately evaluate the performance of a VM management system, all nodes should be considered. This was deemed to be a tedious task if done manually, and thus a framework was created that automates the entire process of installing a management system. This included installing and configuring the management system, running the tests and cleaning up. It was designed to be VM management system agnostic, meaning any management system that supports the functionality defined in an interface could be evaluated using the framework. Consequently, this means the framework in theory could evaluate any other locally installable VM management systems, such as CloudStack and OpenStack. However, this would require adding an implementation for these in code.

3.2.1 Design

Modularity and loose coupling were the two most important factors when designing the framework. This meant the framework was self-sufficient and did not depend on external libraries, which led to the framework being easily installable and the test could be reproduced with little to no effort. Moreover,

Test	**Purpose**
Create each type of VM and delete it	Evaluate the basic functionality of the VM management systems without addressing potential load issues.
Create 20 type-1 VMs and then delete them	Evaluate how the VM management systems manage a relatively large burst of VMs.
Live migrate a type-4 VM	Evaluate how well the VM management systems execute a migration operation without addressing potential load issues.
Live migrate 10 type-1 VMs	Evaluate how well the VM management systems manage a relatively large burst of migration operations.
Scale up and down 2 to 10 workers without VMs	Evaluate how well the VM management systems scale up and down without any load.
Scale up and down 2 to 10 workers with a type-1 VM on each worker	Evaluate how the VM management systems handle scaling operations that incur migration operations.

Table 3.3: Performance tests that were used when evaluating the VM management systems. During each test, metrics were collected for each node.

Type	CPU	RAM	Disk Size
Type 1	1	128MB	1GB
Type 2	1	1GB	5GB
Type 3	2	2GB	10GB
Type 4	4	4GB	20GB

Table 3.4: VM specifications for the different types of VMS that were used in the tests.

the framework defined an interface that specified a set of operations that each test could depend on. By restricting the operations to the ones defined in the interface, the tests became more consistent and fair among the VM management system evaluated. Each implementation was given a state of the hosting nodes, including metadata such as name and IP address, to execute the operation. This meant each implementation could communicate with the hosting nodes however desired. The implementations added for KubeVirt and OpenNebula both used SSH as a base, by sending commands to the control node over a SSH tunnel. The full definition of the interface can be found in the appendix A.1

As previously described, metrics were collected before, during and after the test is completed. This posed a challenge where a bad implementation could introduce noise in the measurements. To minimize the effect of the metrics collector itself, a daemon process was installed on every node. A daemon is a service that acts like a detached process, running as long as its host does. However, metrics were not needed all the time, and only when a test was active. This control was ensured by placing an empty file called *run* in a specific directory. The daemon simply checked for the existence of the file and started collecting metrics accordingly. The metrics were collected locally on each hosting node, where the daemon placed a with all its collected metrics in a predefined format. After each test was finished, the framework simply stopped the daemon from collecting metrics and then downloaded its output file. This approach reduced the number of required interactions with each hosting node to zero since the metrics are collected after the test is finished.

On a technical level, the metrics were collected using a set of tools installed on each hosting node. These tools were installed before any test was conducted. The choice of tools was based on how standardized they were, as well as general availability to ensure future attempts to reproduce the results are possible. Table 3.5 shows an overview of the tools used by the daemon. A design consideration that was required was to run them asynchronously since

each tool imposed a delay that, summarized among the tools, was substantial. Additionally, each tool produced an output with its unique format, which required the use of system commands such as *awk* and *grep* to extract the needed information. The daemon was written in Bash to ensure that it could run on any Linux distribution and did not require any external dependencies. The code for the daemon can be found in the appendix A.2 and the framework design is visualized in Figure 3.1.

Tool	**Utilization**	**Description**
mpstat	CPU	Reports activities for each available processor
free	Memory	Displays the total amount of free and used memory in the system
iostat	Disk	Reports CPU utilization and disk I/O statistics

Table 3.5: Tools used by the metrics collector daemon. Each tool was installed on each Azure node during the installation phase.

The framework's flow was split into four phases to allow for both caching and modular control. This was useful both when developing the framework, but also when running the tests, as the environments did not have to be deleted between each iteration. Skipping a phase was controlled by editing a configuration file. However, it is important to note that certain phases cannot always be skipped. For instance, if an environment has been torn down, the next iteration must both recreate and reinstall each hosting node. While skipping a clean-up phase could introduce noise in the results due to potential caching functionalities, it was deemed appropriate since each test was associated with a cost in Azure. The entire workflow is illustrated in Figure 3.2 and is described below.

1. **Setup**: The setup phase is responsible for creating the nodes and networking in Azure. It is also responsible for installing base packages that every node requires, such as *sysstat* to collect utilization metrics and *jq* to parse JSON output.

2. **Installation**: The installation phase is responsible for installing and configuring the VM management system on the Azure nodes. Each VM management system implements an installation method that is responsible for installing and configuring the system with one control node and two worker nodes such that it can run a VM. The installation method

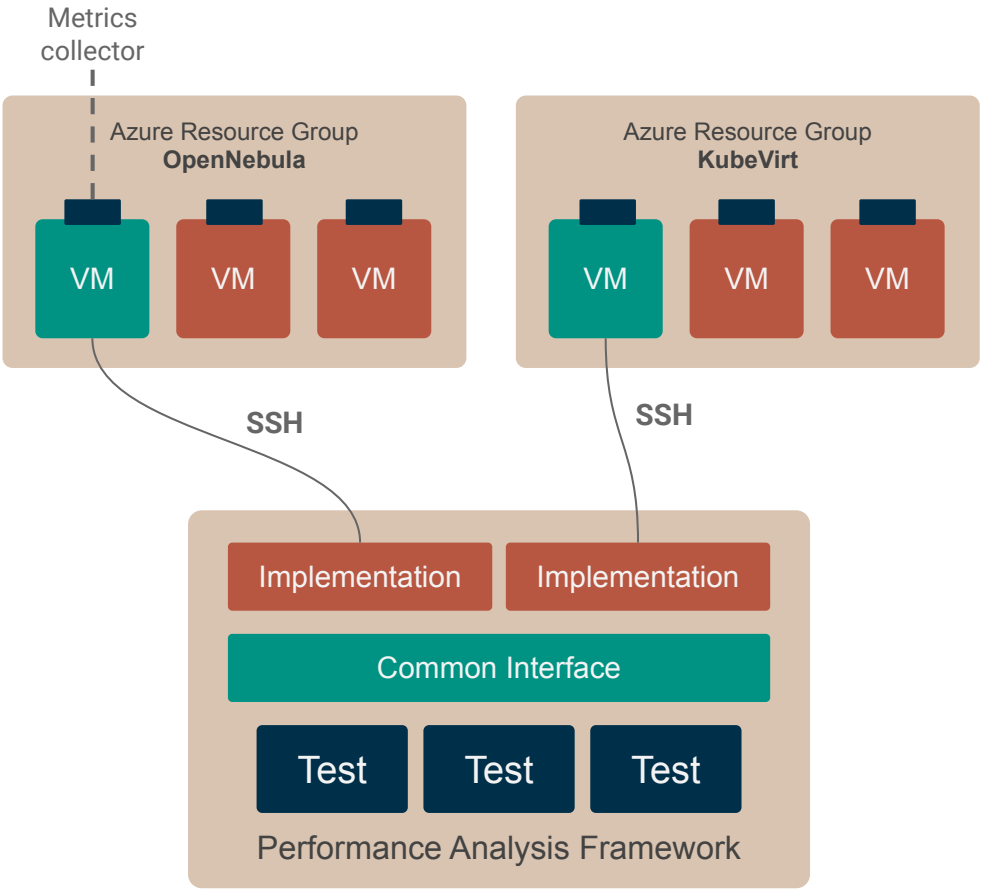

Figure 3.1: Overview of the framework designed with a framework communicating with the VM management systems through SSH. The metric collector was installed on each node to collect utilization metrics. The green VM figure represents the VM management system control node and the orange VM figures represent the worker nodes.

also installs several standby workers that will be used when scaling up and down the number of worker nodes. By having standby workers, the framework can independently measure the cost of scaling up and down without the cost of downloading dependencies. While larger dependencies can be seen as a metric on their own, it is important to note that production environments usually generate OS images with prepared dependencies, rendering the cost of downloading dependencies irrelevant.

3. **Benchmark**: The benchmark phase is responsible for running the performance tests. The tests are described in 3.3 and are designed to evaluate the general performance of the VM management system. Each test is a combination of the methods the framework interface offers, such as creating VMs in different sizes, migrating VMs between nodes and scaling up and down the number of worker nodes. This phase generates and saves the results of the tests in a JSON file.

4. **Teardown**: The teardown phase is responsible for deleting the Azure nodes and ensuring that no resources are left behind. Having this as a separate phase allows for tests to run sequentially without the time cost of creating and deleting nodes for each test.

3.2.2 Implementation

On a technical level, the framework was implemented using the programming language Go. This choice was made due to its performance but also its concurrency features since many tests involved asynchronously making requests to a VM management system.

Furthermore, the most important part of the framework was arguably to provide fair and reproducible comparisons. As described previously, each VM management system implemented an interface that defined methods for creating and deleting VMs. While this provided uniform operations from the framework's point of view, it also made the method's implementation crucial: KubeVirt's implementation of creating a VM should be as close to OpenNebula's implementation as possible to ensure that the comparisons are fair and that no unnecessary overhead is introduced. One example of this was that any network-related configuration had to be omitted since network-related metrics were not considered in the project. Another example was that the scheduling configuration had to be edited to ensure that VMs were created in as similar a manner as possible. However, all features that a VM management system offered and that were beneficial for the specific operation were considered. For

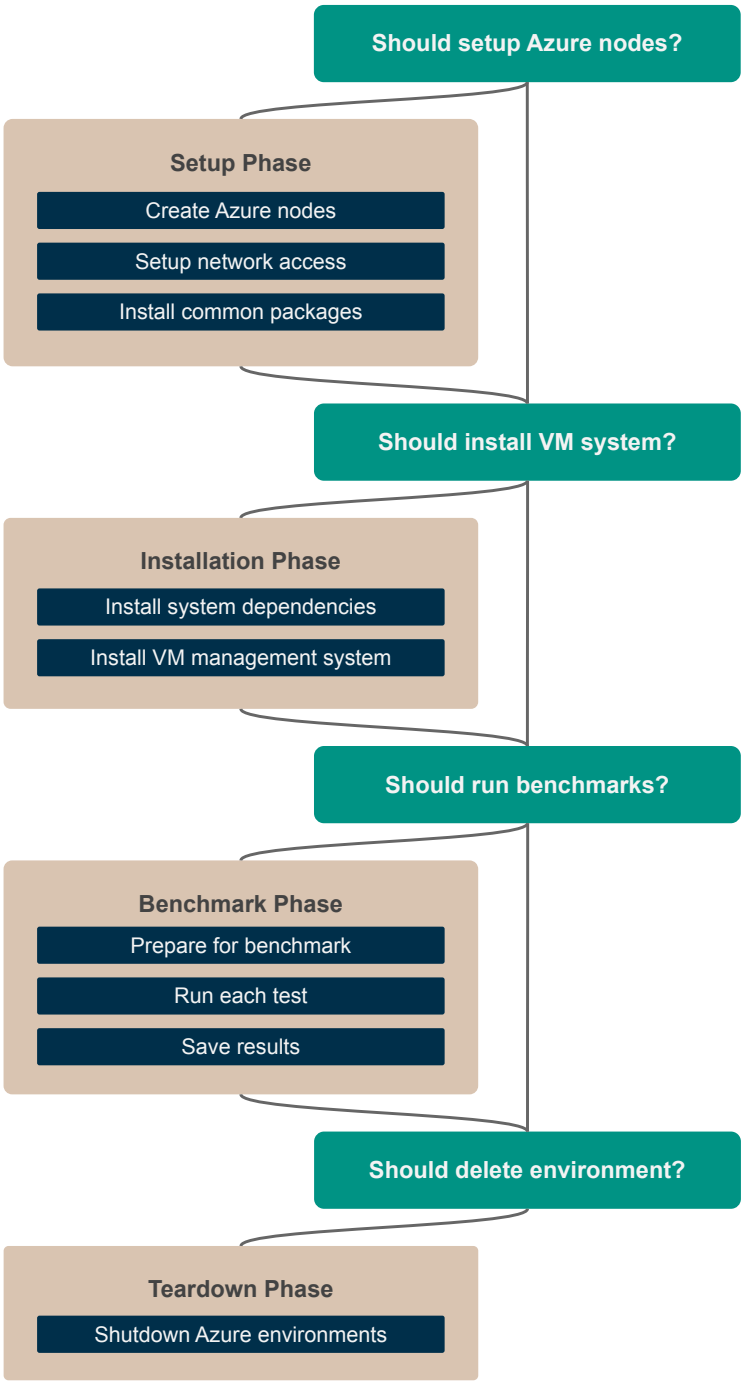

Figure 3.2: Overview of the workflow of the framework. The framework was divided into four phases that were responsible for different parts of the workflow. Each phase could be run independently of each other and disabled if not needed.

instance, OpenNebula did not allow a live migration action to be performed on a VM that is not running, while KubeVirt allowed the operation to be requested despite the state of a VM. This allowed KubeVirt to omit the waiting time for a VM to be in the correct state before requesting a migration action.

Below is an extraction of OpenNebula's and KubeVirt's implementations for creating a VM.

KubeVirt

```
kubectl apply -f <<EOF
    apiVersion: kubevirt.io/v1
    kind: VirtualMachine
    metadata:
        name: $NAME
    spec:
        running: true
        template:
            spec:
            domain:
                devices:
                    rng: {}
                    disks:
                    - disk:
                        bus: virtio
                        name: containerdisk
                    - disk:
                        bus: virtio
                        name: emptydisk
                    resources:
                    requests:
                        cpu: 100m
                        memory: $MEMORYMi
                    limits:
                        cpu: $CPU_CORES
                        memory: $MEMORYMi
            volumes:
            - name: emptydisk
                emptyDisk:
```

```
                capacity: $DISK_SIZEGi
            - name: containerdisk
                containerDisk:
                image: $OS_IMAGE
EOF
```

OpenNebula

```
onetemplate instantiate $TEMPLATE_ID --name $NAME <<EOF
    CPU="0.1"
    VCPU="$CPU_CORES"
    MEMORY="$MEMORY"
    DISK=[
        SIZE="$DISK_SIZE"
        IMAGE="$OS_IMAGE"
    ]
EOF
```

The entire framework is open-source and available at `INSERTURLHERE`. The GitHub repository contains all the code required to run the framework, including documentation on how to set up and run the framework.

Chapter 4

Usability Analysis

The qualitative aspects of this project are comprised of a usability analysis, which was comprised of a set of tasks designed in cooperation with a system administrator at Net Insight. The goal of the usability analysis was to mimic tasks that a system administrator could face on a daily basis. Moreover, the tasks were designed to be done using the Kubernetes CLI tool kubectl. This approach was deemed most appropriate since KubeVirt does not come with a Graphical User Interface (GUI) like a traditional VM management system, and using a third-party alternative could risk introducing bias that is not directly tied to KubeVirt.

4.1 Setup

The usability analysis was hosted using a managed Kubernetes cluster in Azure (AKS). Similarly to the performance analysis, this approach was chosen due to its availability. This means that the participants could perform the tasks anywhere at any time. Additionally, by not being required to install a local Kubernetes environment, the participants could focus on the tasks rather than the infrastructure. A usability analysis that evaluates the installation process of KubeVirt is thus a possible future research direction. Moreover, the predefined storage options provided by AKS allowed for simple management of persistent disks and snapshots. Similar to the performance analysis, the nodes setup was crucial to enable certain operations such as live migration that involves moving a VM between two worker nodes. For this reason, the usability opted for the same setup with one control node and two worker nodes.

4.2 Task Design

To ensure the focus of the analysis was kept on the usability of KubeVirt, it was designed as a step-by-step guide split into three tasks. This allowed the participants to familiarize themselves with KubeVirt before moving on to more complex tasks. the tasks are described below.

- **Basic usage**
  This task was designed to act as a starting point for the usability analysis. The participants were asked to create a single VM configured with a *containerDisk*, connect to it using SSH and VNC and then delete it. This task acted as a starting point for the participants and its purpose was to evaluate how well KubeVirt supports the most basic operations.

- **Maintenance**
  The second task dived into maintenance-related tasks, which are recurring tasks a system administrator might face when maintaining a set of VMs. This task can be divided into two parts: live migration and snapshots. The first part involved creating a VM on the first worker node, then using KubeVirt's mechanisms for live migration, moving it to the second worker node without any downtime. To keep the focus on live migration, a lightweight VM was used. The second part of this task was concerned with creating and restoring a snapshot of a VM. A snapshot of a VM represents the state of a VM at a given time point. Similarly to the first part, a lightweight VM was used to keep the focus on the snapshot-related operations. Both live migration and snapshotting were included in the usability analysis since they are applied differently in KubeVirt compared to traditional VM management systems due to KubeVirt's declarative approach, see 2.6.

- **Debugging**
  The third task was concerned with debugging and how well KubeVirt allowed a user to resolve potential configuration errors. As described in 2.6, KubeVirt is implemented using CRDs, which has built-in field validation to prevent misconfiguration. However, the validation does not stretch to include runtime issues, such as cloud-init configuration mismatches or typos. For this reason, this task was designed to present the user with a VM that has a cloud-init configuration error causing it to not be accessible over SSH. In this scenario, the participants had to find an alternative entry point to the VM to debug the issue or resolve

the issue immediately by inspecting the configuration file. Similarly to previous tasks, a lightweight VM was used to keep the focus on the debugging aspect of the task.

Feedback from the participants was collected from the participants either by answering a set of open-ended questions or as a summary of a group of tasks.

4.3 Conducting the Usability Analysis

A group of system administrators at Net Insight were asked to perform the tasks presented above. They were chosen primarily by their role and experience with OpenNebula as well as Kubernetes. This provided a mix of knowledge among the participants, which reduced the risk of biases. The participants were given separate namespaces, a kubeconfig file to access the Kubernetes cluster and then presented with the tasks in the form of a PDF document. By giving each participant its namespace, it reduced the risk of interference between them. Moreover, the participants were given two weeks to complete the tasks, and while this is a long time relative to the number of tasks they were given, it allowed them to familiarize themselves with KubeVirt and potentially explore areas or features around it. The extra time to complete the tasks was also useful considering that they were encouraged to use the official KubeVirt documentation. The entire usability analysis can be found in the appendix C.

Chapter 5

Results

This section contains the results of the performance analysis and usability analysis. Each test in the performance analysis was run 50 times, and the result visualizations were averaged using a rolling mean with a window size of 5% of the total number of data points. As described in the previous chapter, the usability analysis received feedback in different forms, either by answering questions or providing a free-form response.

5.1 Performance Analysis

This section contains the results of the performance analysis with KubeVirt and OpenNebula. Only the most significant results are presented in this section, and the complete list of results can be found in the appendix B.

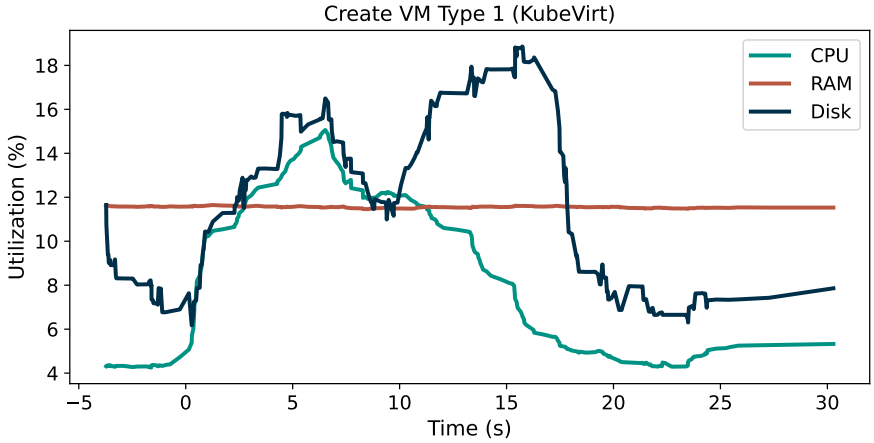

Figure 5.1: Resource utilization on the control node when creating a VM type 1 in KubeVirt. CPU and disk utilization follow a similar pattern nearly reaching 20% while memory utilization remains constant at 12% for the duration of the test.

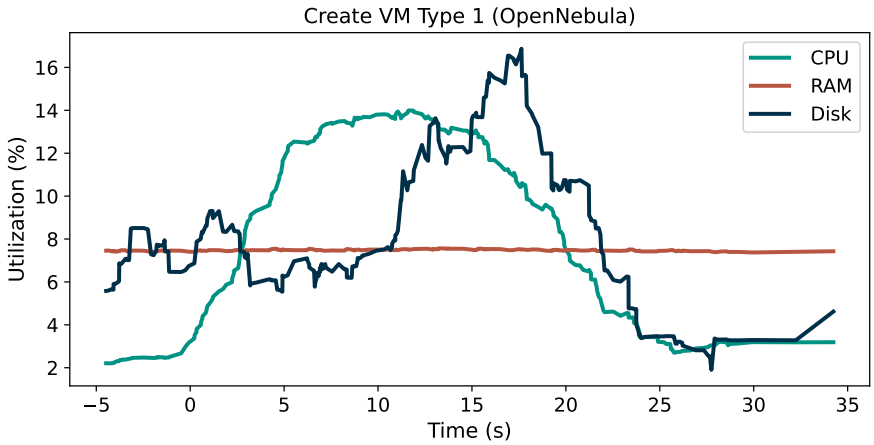

Figure 5.2: Resource utilization on the control node when creating a VM type 1 in OpenNebula. CPU and disk utilization both increase and peak after half the duration of the test. Memory utilization remains constant below 8% for the duration of the test.

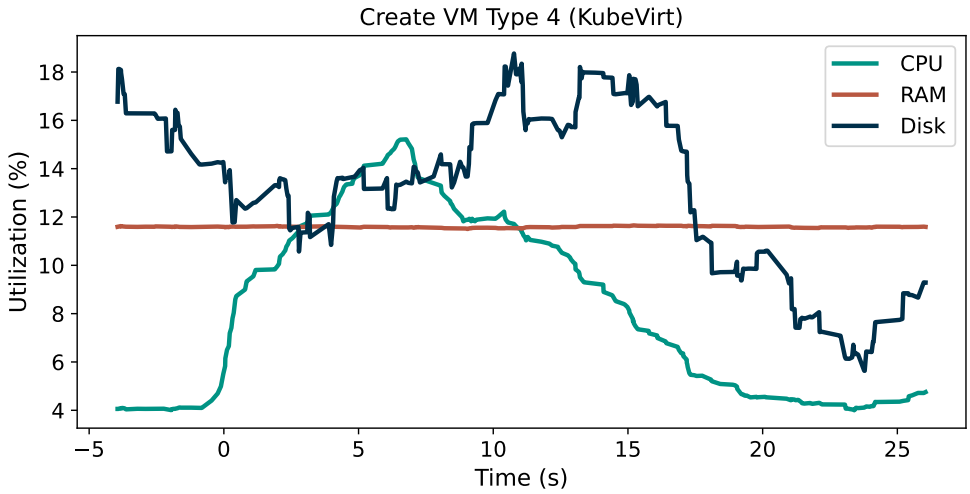

Figure 5.3: Resource utilization on the control node when creating a VM type 4 in KubeVirt. CPU utilization peaks quickly while disk utilization is sporadic. The sporadic behavior is not in line with the operations performed during the test. Memory utilization remains constant at 12% for the duration of the test.

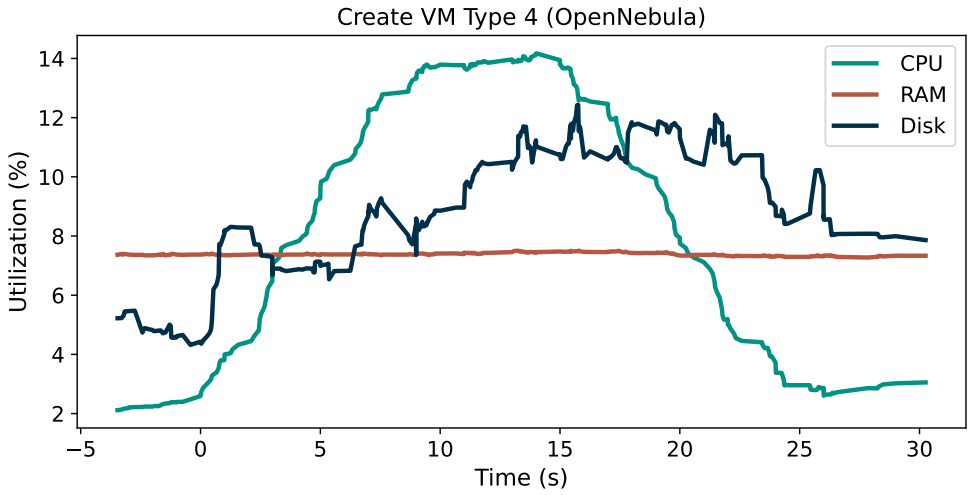

Figure 5.4: Resource utilization on the control node when creating a VM type 4 in OpenNebula. CPU utilization is the most prominent among the resources plateauing at nearly 15% utilization. Memory utilization remains constant at 8% for the duration of the test.

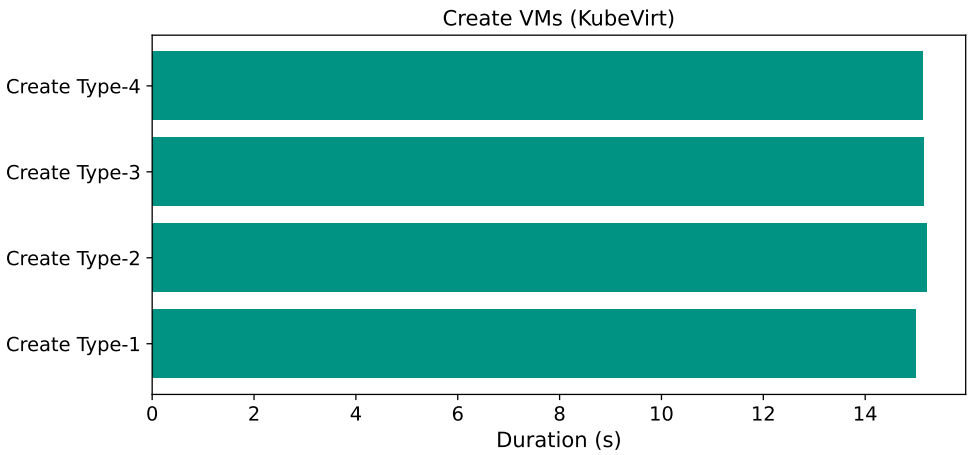

Figure 5.5: Duration of creating VMs in KubeVirt. The duration for all types remains constant for each VM type reaching slightly above 14 seconds

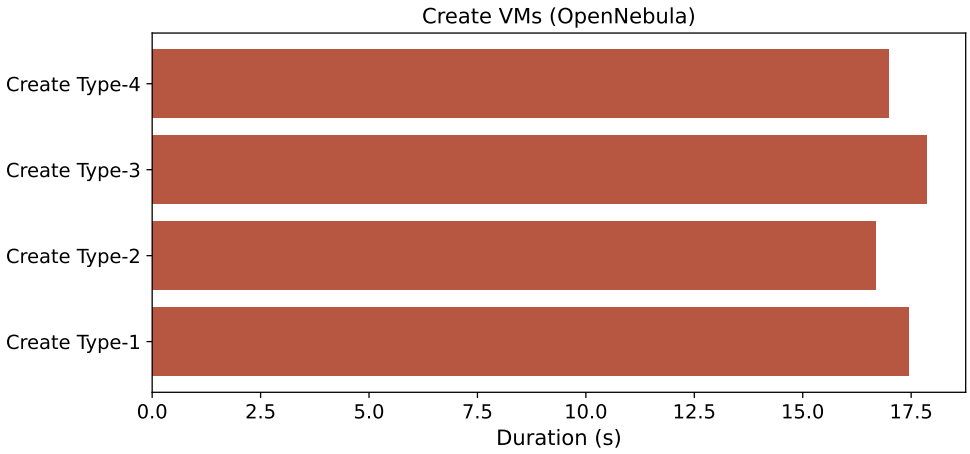

Figure 5.6: Duration of creating VMs in OpenNebula. The duration for all types remains constant for each VM type reaching nearly 18 seconds.

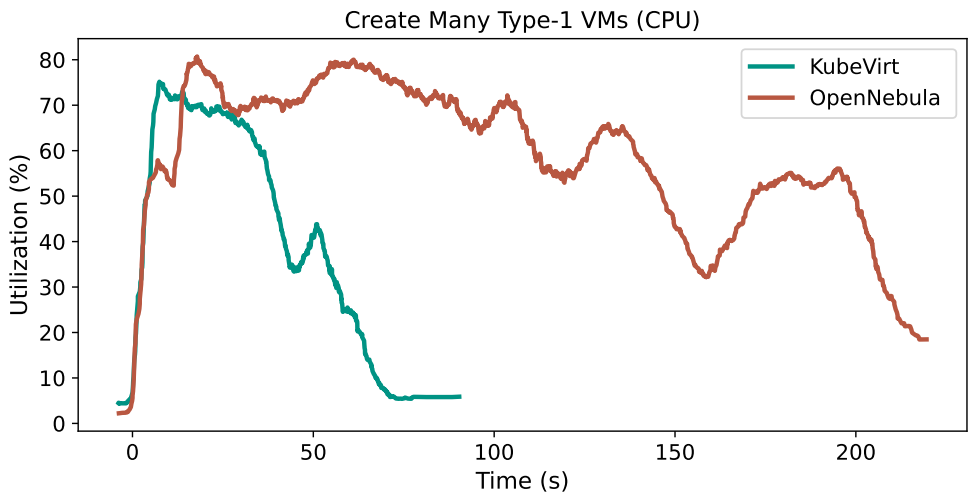

Figure 5.7: CPU utilization on the control node when creating 20 type-1 VMs. The CPU utilization peaks quickly for both KubeVirt and OpenNebula due to the burst of requests. KubeVirt reaches a slightly lower peak than OpenNebula and subsides significantly faster. OpenNebula does not manage to return to a utilization level of an idle state within the duration of the test, as evidenced by the utilization not reaching base utilization in the end.

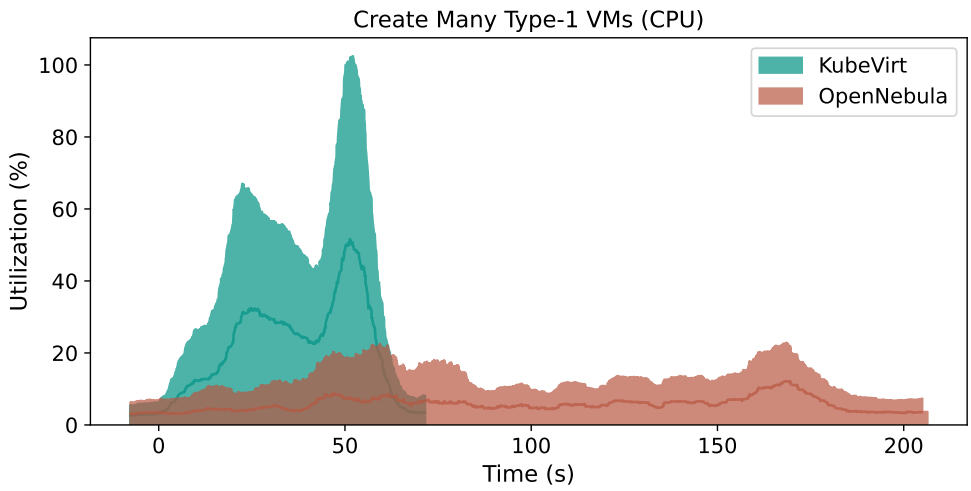

Figure 5.8: Cumulative CPU utilization among worker nodes when creating 20 type-1 VMs. The CPU utilization among the worker nodes is vastly different between KubeVirt and OpenNebula, where KubeVirt experiences a short-lived large burst of CPU utilization while OpenNebula barely reaches a cumulative 20% utilization.

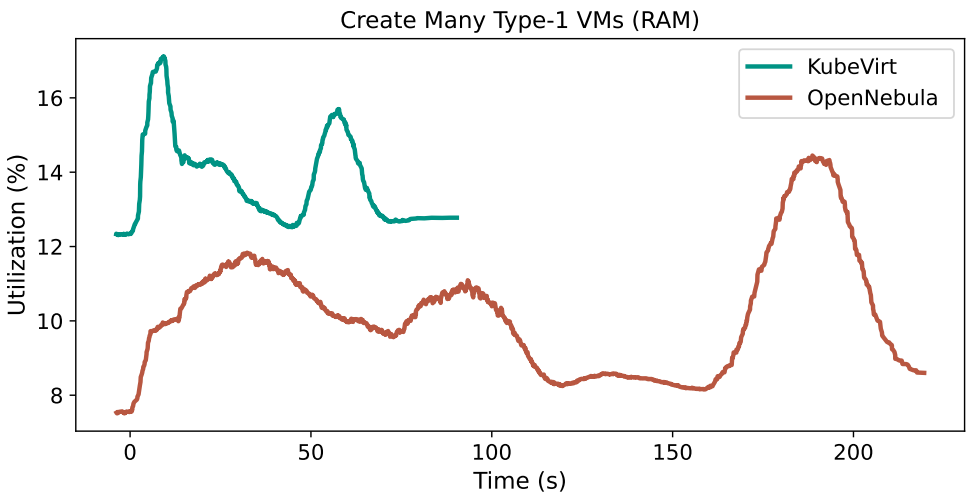

Figure 5.9: Memory utilization on the control node when creating 20 type-1 VMs. KubeVirt experiences a burst of memory utilization in the beginning, while OpenNeubla has a more gradual increase before spiking quickly toward the end of the test. While KubeVirt has a larger base memory utilization, OpenNebula experiences a larger induced load from the test, as evidenced by its longer slopes.

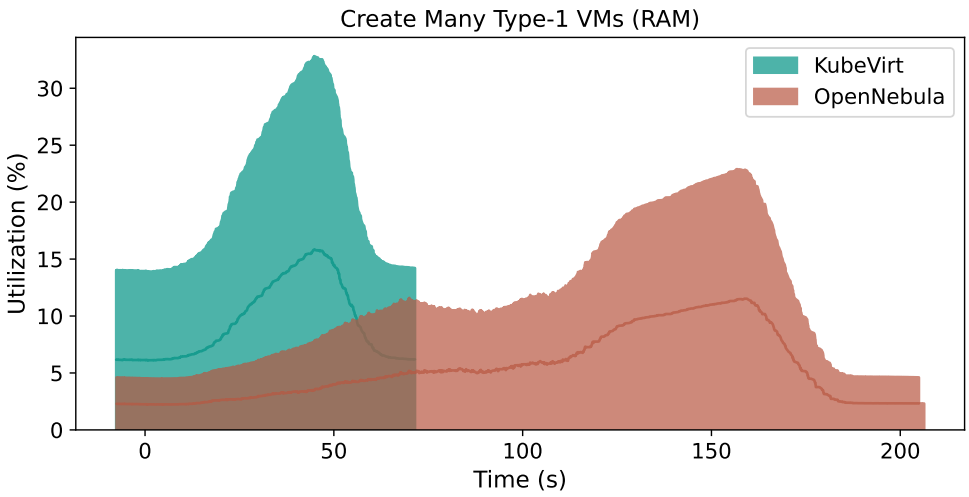

Figure 5.10: Cumulative memory utilization among worker nodes when creating 20 type-1 VMs. While KubeVirt experiences a larger memory utilization due to a larger base, both systems have a similar magnitude of the induced load, albeit KubeVirt's load is more short-lived.

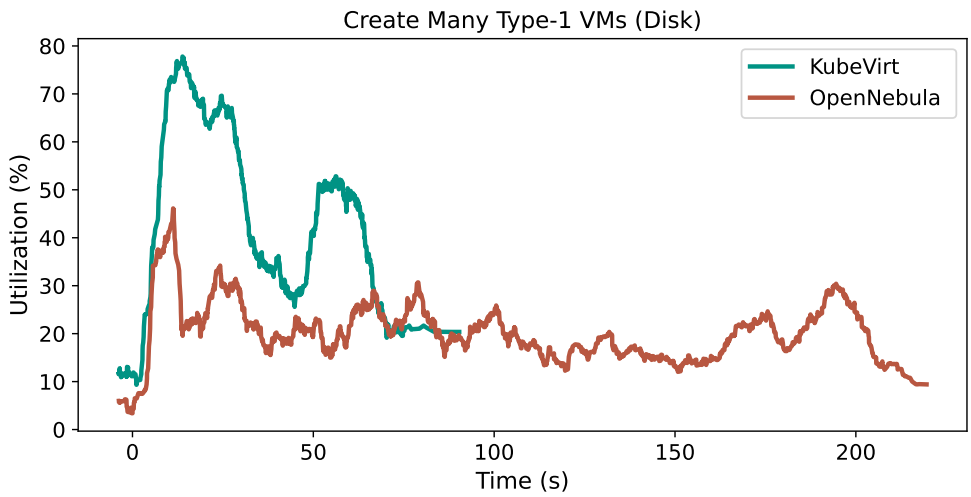

Figure 5.11: Disk utilization on the control node when creating 20 type-1 VMs. KubeVirt experiences a larger burst of disk utilization than OpenNebula, while OpenNebula maintains a steady disk utilization throughout the test.

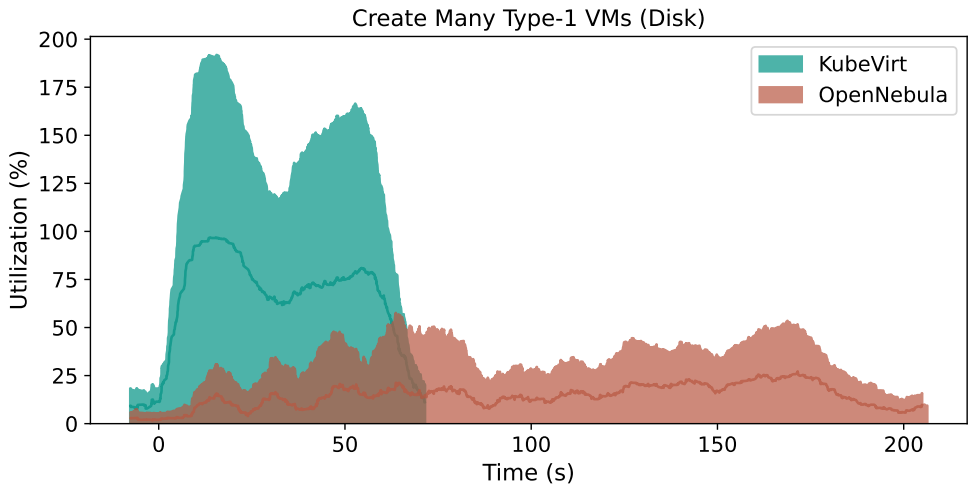

Figure 5.12: Cumulative disk utilization among worker nodes when creating 20 type-1 VMs. KubeVirt experiences a larger burst of disk utilization than OpenNebula, while OpenNebula maintains a low and steady disk utilization throughout the test.

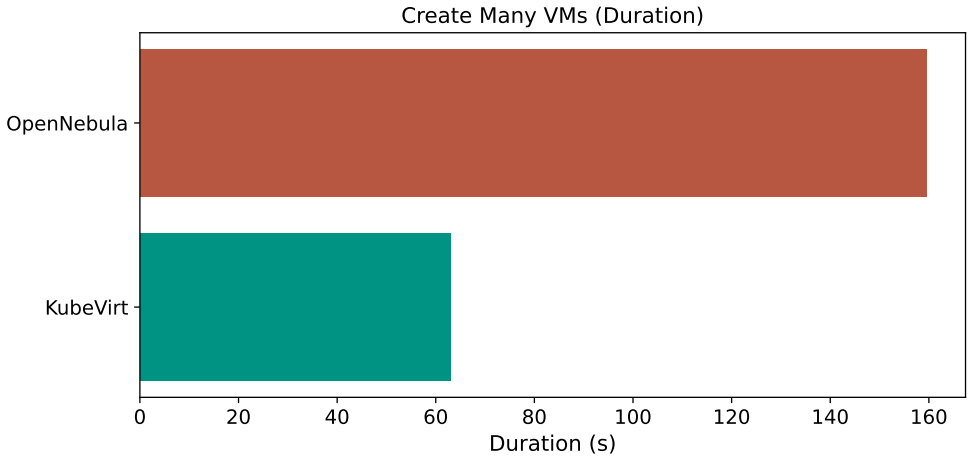

Figure 5.13: Duration of creating 20 type-1 VMs. KubeVirt is significantly faster to create 20 type-1 VMs than OpenNebula, taking 67 seconds compared to OpenNebula's 192 seconds.

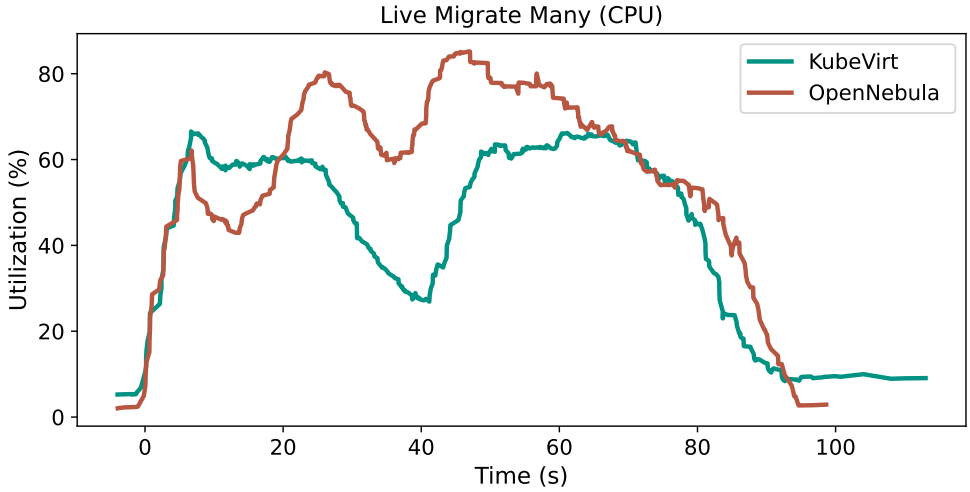

Figure 5.14: CPU utilization on the control node when live migrating 10 type-1 VMs. Both systems follow a similar pattern where KubeVirt has a slightly larger base utilization than OpenNebula, but the induced load is larger for OpenNebula.

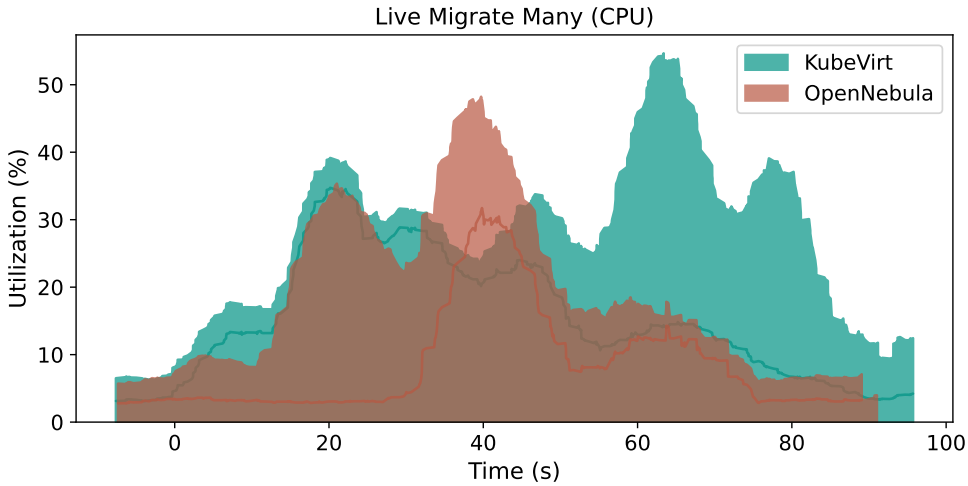

Figure 5.15: Cumulative CPU utilization among worker nodes when live migrating 10 type-1 VMs. Both systems experience a similar load for the first half of the test, but KubeVirt experiences a significantly larger load in the second half of the test.

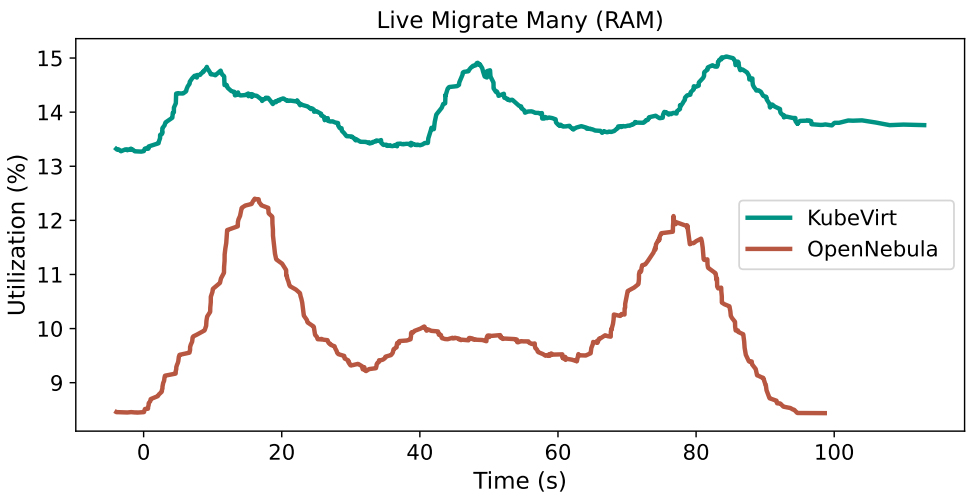

Figure 5.16: Memory utilization on the control node when live migrating 10 type-1 VMs. KubeVirt has a larger base memory utilization than OpenNebula, but the induced load is more than double for OpenNebula.

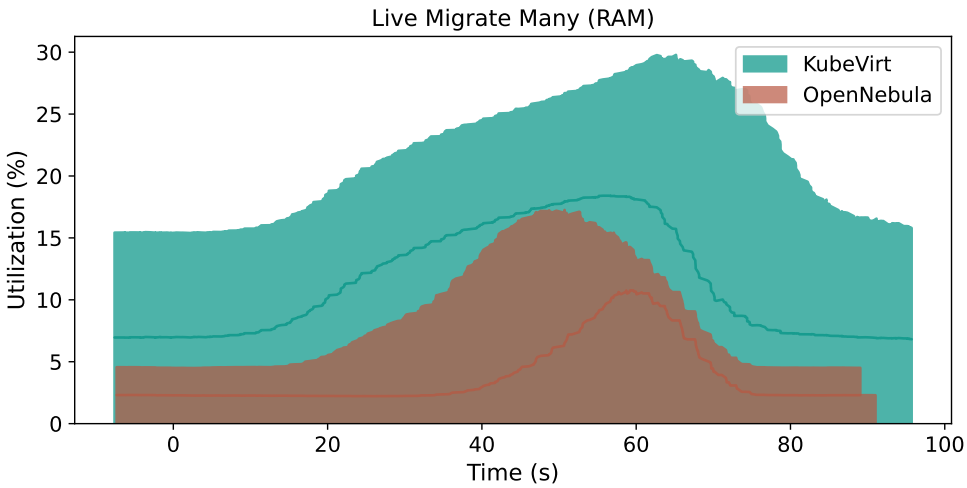

Figure 5.17: Cumulative memory utilization among worker nodes when live migrating 10 type-1 VMs. KubeVirt has a larger base memory utilization than OpenNebula, but both systems experience a similar induced load. OpenNebula returns to an idle state faster than KubeVirt.

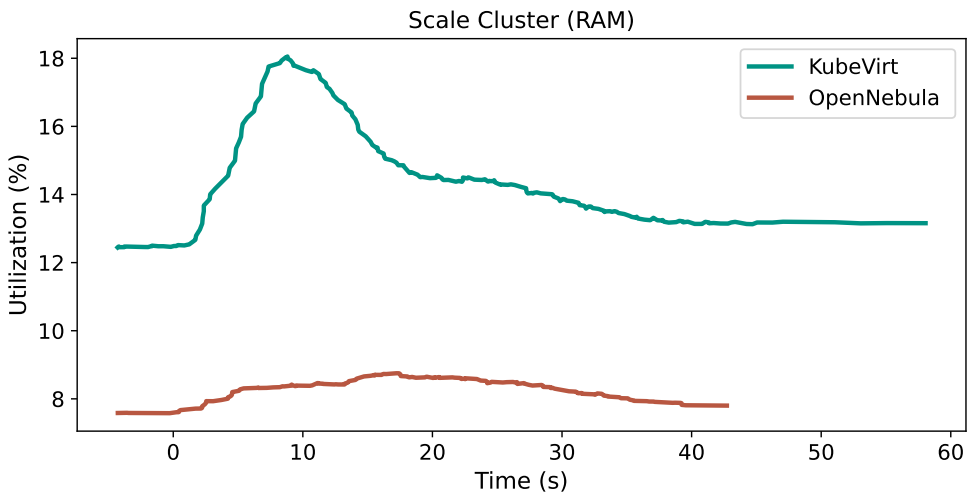

Figure 5.18: Memory utilization on the control node when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes. KubeVirt experiences a significantly larger memory utilization than OpenNebula. OpenNebula's memory utilization is barely affected by the scaling operation.

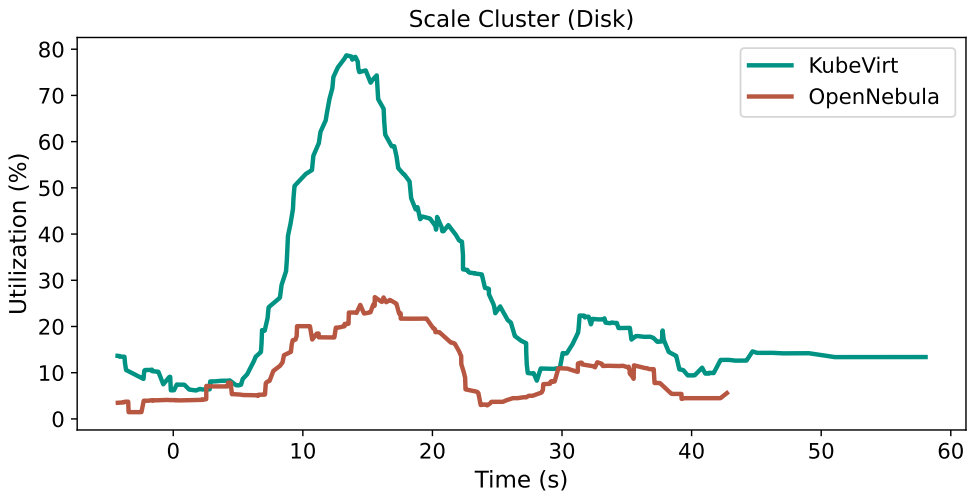

Figure 5.19: Disk utilization on the control node when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes. KubeVirt experiences a rapid increase in disk utilization, reaching a peak of over 70% utilization. OpenNebula has a less aggressive increase reaching barely 30% utilization.

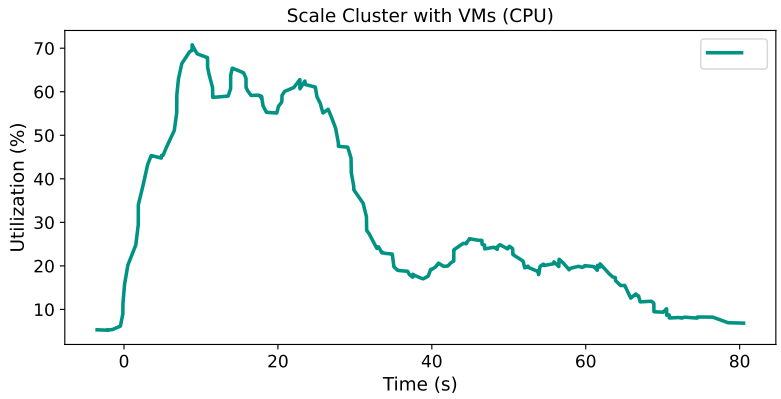

Figure 5.20: CPU utilization on the control node when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes with 1 type-1 VM running on each node. KubeVirt experiences an initial burst of CPU utilization for the scale-up operation, but a significantly lower utilization for the scale-down operation. OpenNebula required manual intervention to complete the test due to VMs ending up in bad states, which caused it to be disqualified.

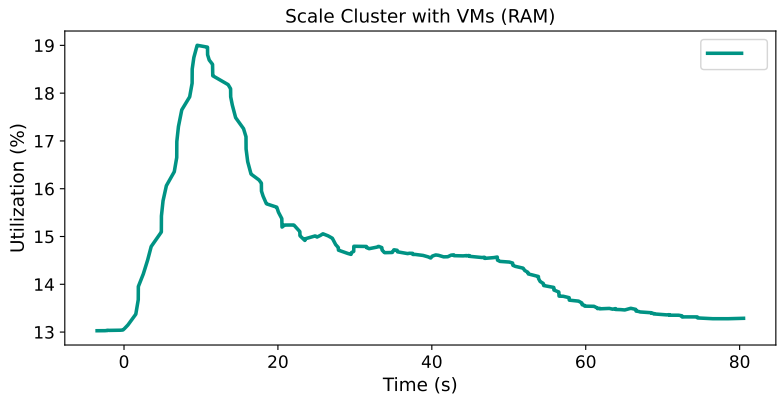

Figure 5.21: Memory utilization on the control node when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes with 1 type-1 VM running on each node. KubeVirt experiences a large burst of memory utilization for the scale-up operation, but a significantly lower utilization for the scale-down operation that gradually decreases towards the end. OpenNebula required manual intervention to complete the test due to VMs ending up in bad states, which caused it to be disqualified.

5.2 Usability Analysis

The result of the usability analysis consists of the answers collected from its participants. As described previously, a participant could submit feedback either by answering questions or by summarizing their experience with KubeVirt. Below are some extracted answers, but all the answers can be found in the appendix D.

Basic Usage

The participants found the process of creating a VM in KubeVirt to be mostly straightforward. Using the kubectl CLI tool appears to be one of the reasons the operation was smooth.

- Participant 1: *The act of doing kubectl apply -f and kubectl delete is quite neat.*
- Participant 2: *Very easy! I like that it is as simple as creating a YAML file and then applying it.*

However, KubeVirt's complementary CLI tool virtctl was not as user-friendly, as indicated by some participants experiencing issues with using it to connect to a VM over SSH. The issue in particular is related to which SSH public is used when connecting. An SSH public verifies the identity of the SSH client. When it worked, though, virtctl was straightforward to use.

- Participant 2: *Very simple! Simply doing virtual ssh cirros@my-vm and then typing in the password and then I was in!*

Another issue encountered was related to the structure of the manifests. Participant 1, in particular, found the VM manifest definition to be complex and that it negatively affects the process of creating a VM. *bus: virtio* is a configuration in KubeVirt that allows its disks to be attached as an ordinary hard disk [125].

- Participant 1: *... the VM definition is not very neat. It would be quite difficult for me to figure out I need to use b̈us: virtioẅithout having someone else tell me I need to.*

KubeVirt's complementary CLI tool *virtctl* was found to be useful when connecting to the VM, as it provided a single tool for both VNC and SSH.

Maintenance

Compared to the basic tasks, the maintenance tasks received a broader range of feedback. This is expected since it represents more complex tasks, but also operations that are implemented differently in KubeVirt compared to traditional VM management systems. Some examples include Participant 1 who argued that the live migration object in unnecessary and that the scheduler should have enough information from the node selector field and labels already.

- Participant 1: *I also found it unintuitive that I needed to create the migration resource after everything was correctly labeled.*

Similar to the basic usage tasks, the participants experienced issues with connecting to VMs using SSH, as it was not clear which SSH public key was used to connect to the VM.

- Participant 2: *I had to delete the entry from /.ssh/kubevirt_known_hosts each time I restarted the VM because virtual ssh complained about a key mismatch.*

Debugging

The debugging tasks received the most negative feedback out of all tasks. The process of debugging was complex and some participants were not able to resolve the issue. This added additional feedback regarding KubeVirt's documentation since the participants had to resort to debugging information elsewhere. However, the documentation was found to be lacking in terms of practical use cases and examples, and instead mostly focused on cluster management and administration-related operations. For this reason, the documentation did not provide any additional aid in solving the task.

- Participant 1: *I could not (easily) find documentation about how virtctl ssh worked so I could not figure out one of the issues. There seems to be no documentation for virtctl*

However, while debugging the issue presented in the task was found to be complex, participant 3 regarded the YAML files in general as a positive aspect of debugging, as they include the entire configuration of a VMs in text files.

- Participant 3: *Debugging the issue was easy because the manifest uses standard constructs that are easily googleable.*

Overall

While the feedback to KubeVirt was generally mixed, Participant 4, who already managed other Kubernetes resources, highlighted GitOps as one of KubeVirt's strengths.

- Participant 4: *It is primarily because I'm already storing all my Kubernetes manifest in Git and have the CI setup for that, so why not also create virtual machines in a similar way.*

The documentation was found to be lacking practical examples of how to get started outside the scope of the given tasks and instead focused more on the operational aspects of KubeVirt.

Chapter 6

Discussion

The goal of this project was to evaluate the applicability of managing vm in Kubernetes using KubeVirt, and by using a mixed-methods approach, this project has produced quantitative and qualitative results. This chapter will provide additional insight by interpreting the results and concluding whether the research questions can be answered.

6.1 Limited Features and Result Intepretation

The performance analysis was created to explore any quantifiable difference between managing VMs using KubeVirt and using a traditional approach. Since VM management systems are complex and span across multiple nodes, a framework was developed that included a common interface that a management system could implement to be benchmarked. This approach is modular and enables other VM management systems to be benchmarked by adding an implementation for it. However, this creates a flaw where, depending on the implementation, a VM management system can appear to be more or less efficient than it is. This means that, by definition, such a framework is only as good as the implementations are, which is why the implementations for KubeVirt and OpenNebula were designed as close to each other as possible. However, certain features and settings were not able to be tuned in the same way since they did not exist at all in both systems, such as the scheduling configuration where OpenNebula provides only a single option for scheduling: the number of VMs that can be dispatched concurrently. If both systems are not able to be tuned in the same way, it leaves the results for certain tests open for interpretation. To favor a system that is more tunable and configurable, this project has opted for an approach where if a system does not provide a specific

setting, that setting is assumed to be fully optimized. The primary example of this can be found in the *Create Many Type-1 VMs* tests, see figures 5.7, 5.9, 5.11 and 5.13. KubeVirt, using the Kubernetes scheduler, was able to create 20 type-1 VMs in 67 seconds, while it took OpenNebula 192 seconds on average. However, the results also suggest that KubeVirt uses more resources than OpenNebla, albeit during a significantly shorter time. This creates another dilemma when interpreting the results whether a longer but lower resource utilization is more favorable than a short burst. By interpreting the results, KubeVirt appears to use most of the available resources, while OpenNebula remains idle in comparison. For this reason, this project has chosen to favor a short burst of resource utilization since it reflects on the system's ability to use the resources it was allocated to their fullest potential.

However, it is not always the case that KubeVirt experienced a short-lived burst of utilization and OpenNebula experienced a long but low utilization. In the *Scale Cluster* tests, KubeVirt has a significantly larger disk utilization than OpenNeubla for a similar period, see figure 5.19. KubeVirt and OpenNebula have initially similar characteristics when scaling the cluster but KubeVirt disk utilization is significantly more intense. For this specific test, the result suggests that KubeVirt does not provide the apparent trade-off discussed previously. This could be due to the way KubeVirt implements adding additional nodes to the cluster since it requires a service to run on each hosting node (see 2.6.1). Another explanation could be that the specific Kubernetes distribution used in the tests is not optimized for scaling the cluster. However, exploring solutions for this is outside the scope of this project.

The final test in the performance analysis involved scaling a cluster whose hosting nodes had a VM running on it. It started by scaling a cluster from 2 to 10 worker nodes, then creating a type-1 VM on every worker node before scaling it down to 2 worker nodes again. This required the VM management system to move the VMs to other worker nodes, preferably those that would remain after the cluster was scaled down. As such, this imposed a challenge on the management system since a delayed scale operation could result in VMs being wrongfully scheduled to a node that was just about to disconnect. KubeVirt managed to complete this task without any significant overhead compared to scaling without VMs. However, OpenNebula was not able to complete it due to some VMs getting stuck on hosts that should have disconnected. Since the test was designed to disconnect the worker and then wait for each host to be empty, the OpenNebula implementation hung indefinitely. A possible explanation for this could be that OpenNebula attempted to move a VM to a host that was just about to disconnect but hadn't yet, causing the receiving host to

end up in a bad state. Another explanation could be that OpenNebula required more than one control plane node to handle such bursts for migration operation since its environment only included a single control plane node. However, this is purely speculative and exploring the cause of the issue is a possible future research direction. The issue of VMs stuck on hosts could be resolved by manual intervention, but since this was deemed unfair to the other system, OpenNebula was disqualified from the *Scale with VMs* tests. For this reason, OpenNebula's result was omitted from the test and only KubeVirt was evaluated.

6.2 Kubernetes Overhead and Induced Load

Throughout the results of the performance analysis, there is a pattern where OpenNebula experiences a consistently lower base overhead than KubeVirt, despite of whether an operation is active or not. Having a larger overhead means that the management system has more background processes, which is generally not desired. A clear example of this can be found in the *Create Tiny* test, see figures 5.1, 5.2, 5.3 and 5.4, where KubeVirt's CPU utilization hovers just above 4% while OpenNebula stays around 2% utilization, and memory utilization being 8% and 12% respectively. This is further supported by the *Live Migrate* test where their memory utilization is also 8% and 12% respectively. This is highly indicative that KubeVirt is more resource-intensive while not performing any operation. However, before any conclusions are drawn, let's consider the surrounding factors.

The first possible cause of this could be tied to Kubernetes not originally being designed to manage VMs. Certain operations and orchestration mechanisms might not yet translate well to VMs, which could manifest as resource usage overhead. Secondly, KubeVirt might not be the culprit in the situation, and the overhead could be tied to Kubernetes itself. This could be taken a step further, where the overhead could be larger for certain Kubernetes distributions. However, this is unlikely since the Kubernetes distribution used in the tests is designed to be as lightweight as possible. Lastly, as described in 2.6.1, KubeVirt enables VMs to run in Kubernetes by wrapping them inside a Pod. While this has enabled seamless integration with other Kubernetes resources, it also implies there could be a layer of indirection, meaning that any traffic to and from the VM has extra routing steps. However, whether a Pod imposes a significant overhead for KubeVirt VMs is a possible future research direction.

The flip side of this is that many of the results suggest the opposite regarding the induced load, meaning the added load on the system when performing

a specific operation. It can be calculated as the difference between the total utilization and the base utilization. This essentially isolates base overhead from the equation and adds another dimension to the evaluation. The induced load can be difficult to interpret for reasons previously described where there is a trade-off between a short burst of utilization and a longer small utilization. The results suggest that KubeVirt has a significantly lower induced load for most tests, where in certain cases OpenNebula experiences an induced load twice as large as KubeVirt. This is exemplified in the *Create Many VMs* test where OpenNebula both takes twice as long to complete but also experiences an induced memory load twice as large as KubeVirt. This finding is essentially the opposite of the finding with base overhead, which creates another point of discussion of which factor is most prominent. From the results, it appears that induced load is a more costly factor when comparing the two systems. One example is the *Migrate Many* test where OpenNebula reaches an induced load of 76% while KubeVirt reaches just above 50%. Previously, the difference between the system's base overhead was under 10 percentage points, however, this example shows an induced load with a difference of 26 percentage points. However, while this is indicative of KubeVirt being more scalable, it remains to be examined whether this holds for a system with many more nodes.

In summary, the performance analysis has shown that KubeVirt has a larger base overhead, but a significantly lower induced load for most of the tests. This suggests that KubeVirt is more scalable than OpenNebula, but could be resource-heavy in scenarios where the available hardware resources are small or where operations are few.

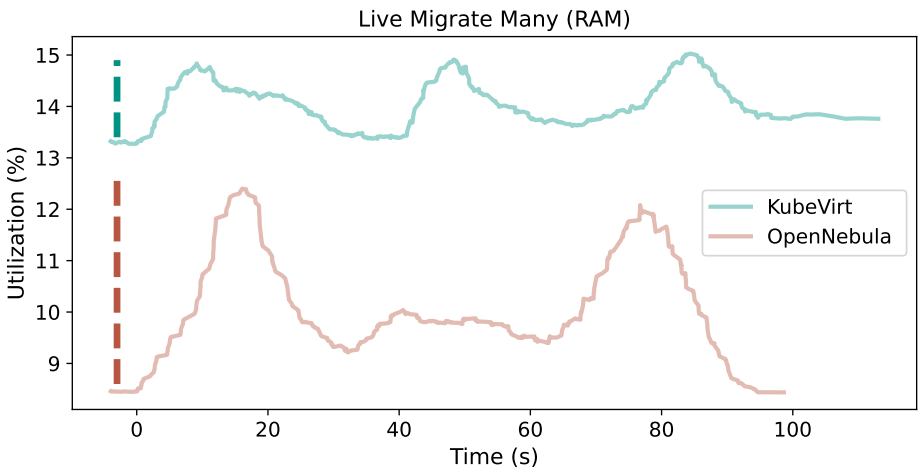

Figure 6.1: Induced memory utilization on the control node when live migrating 10 type-1 VMs. The induced load is a more significant part of the total utilization for OpenNebula than for KubeVirt. The figure is based on the results from figure 5.16.

6.3 Jagged Disk Utilization

Throughout the performance analysis results, there is a recurring and notable pattern specifically tied to disk utilization. The disk utilization graphs are jagged and, in many cases, do not follow the expected pattern of the specific operation performed in the test. This is particularly evident in the *Create VM Type 4* tests, see figures 5.3 and 5.4. The disk utilization for both Kubevirt and OpenNebula is initially larger than expected and does not return to the same state after the test is finished. Since the tests were run multiple times in no particular order, this likely means that noise was introduced in the measurement that caused each test to generate data points in different ranges. When the induced disk utilization was low for a particular test, background disk utilization from sources such as OS-level operations, was more prominent in the results. By then applying a rolling mean smoothing function to the dataset, the noise was reduced, but due to the variability, some jagged features remained. This is further supported by the fact that the disk utilization is less jagged for tests with a larger expected disk utilization, such as the *Create Many Type-1 VMs* tests since these tests generate a larger induced load. While the jagged disk utilization is not a problem in itself, it is important to consider when interpreting the results, especially those where disk utilization is small, as it could introduce bias unrelated to the operation performed.

6.4 Mixed Opinions Using KubeVirt

The usability analysis was designed such that it represents the tasks commonly performed by a system administrator. The results of the analysis include mixed opinions and feedback. While certain basic operations were perceived as straightforward, more complex tasks received more negative feedback. To understand this discrepancy, it is important to consider if the feedback is contradictory or if it targeted the same aspect of the technology. Starting with the positive feedback, many participants found that the CLI tool kubectl was beneficial. This was confirmed by many participants and could be linked to previous Kubernetes experience, and could thus indicate that previous Kubernetes knowledge affects the perceived usability of KubeVirt positively. Defining VMs in manifests was mostly positive due to the way it contains all information regarding a VM, but also how it enables simpler resource management and allows external tools to integrate better. However, containing all the technical information about a VM in a manifest also proved to be confusing. This is not an unexpected outcome since a traditional VM management system often hides many configuration options from the user to improve usability. As such, it is rather a trade-off between transparency and usability. Mixed opinions were also received regarding management tasks. This is particularly true for live migration, which could be caused by how KubeVirt redefines a live migration operation as a declarative operation. Structuring a live migration operation as a resource, where the action is performed once the resource is created is a common pattern in Kubernetes, but the results of the usability analysis suggest this approach could be confusing.

Other negative feedback was more concerned with exploring KubeVirt outside the scope of the usability analysis. Specifically, the official KubeVirt documentation was found to be lacking practical examples of how to get started or how to debug an issue encountered with a VM. While evaluating documentation was not directly part of the usability analysis, it is still a significant problem for technology that involves as many user interactions as KubeVirt.

In summary, the mixed responses from the usability analysis can be caused by biases among the participants regarding previous experience with either system. It could also be caused by KubeVirt being a controversial approach to managing VMs. A common theme for Kubernetes throughout this report is that it is known for scalability and integration, but also for having a steep learning curve. Since KubeVirt follows the design patterns of Kubernetes, it inevitably inherits both its positive and negative aspects. This poses an interesting future research direction that could explore whether the previous VM

management system affects the perceived usability of KubeVirt.

6.5 Concluding Research Questions

The results from the performance analysis indicate both strengths and weaknesses for KubeVirt. As discussed in previous sections, KubeVirt has a larger base overhead compared to OpenNebula. However, the induced load is significantly lower for most tasks in KubeVirt compared to OpenNebula. This suggests that KubeVirt could be more efficient when performing operations and thus could be more scalable than OpenNebula. Moreover, the usability analysis indicates that KubeVirt in many cases benefits from the inherited aspects of Kubernetes, but just like Kubernetes itself, it can be perceived as complex. This section aims to summarize the results and discussion points and answer this project's research questions.

What performance, scalability, and operational efficiencies can be gained by integrating VMs in Kubernetes?

KubeVirt was shown to inhibit a relatively large overhead while not performing any operations, evidenced by the difference in resource utilization compared to OpenNebula. This suggests that KubeVirt is not suitable for environments in which nodes are relatively small, since it might not provide performance efficiencies compared to existing VM management systems. However, KubeVirt's relatively small induced load compared to OpenNebula indicates that there exist environments in which KubeVirt does provide significant performance and, especially, scalability efficiencies. This means that given that the base overhead is not a significant part of a node's capacity, KubeVirt is a more performant and scalable alternative than OpenNebula. Operational efficiency comes with a similar argument. Since KubeVirt inherits Kubernetes's positive and negative aspects, there are scenarios in which KubeVirt is too complex of a system to justify potential benefits. These scenarios include organizations whose system administrators are not familiar with Kubernetes. However, given that the operators have experience with Kubernetes, the benefits of KubeVirt have shown to be substantial. This is mostly due to VMs following a more cloud-native workflow, supporting many integrations that might only otherwise be available for containers. One such example is GitOps, which enables KubeVirt VMs to be managed using Git.

What are the current challenges and limitations in managing VMs and containerized applications in separate environments?

The literature study included several studies to explore the current challenges and limitations of managing VMs and containers in separate environments. It concluded that vm and container share many aspects in how they are used, such as how they both provide a virtual computing environment. This means that containers and VMs are often used for similar or even the same kind of workloads. Moreover, having VMs and containers in separate management platforms directly implies that the architecture is more complex and that an organization must maintain a larger skillset to manage both systems. Additionally, features and mechanisms that could have been applied for both VMs and containers, such as scheduling, exist in duplicates. Furthermore, another perspective on these issues is not tied to the current form of resource management, but rather to what is not fully utilized by separating the management for VMs and containers. A platform such as Kubernetes provides a large variety of tools and third-party integrations that can be utilized by both VMs and containers. In summary, this project not only presents the benefits of integrating VMs and containers into a unified environment but also the benefits of integrating VMs into Kubernetes.

How can system administration be simplified by running VMs in Kubernetes using KubeVirt?

The literature study and usability analysis covered this research question, and they both reached similar conclusions. They suggest that the declarative approach that KubeVirt employs can improve debugging and ease of management due to how contained the information is. One such example is from the usability analysis where one participant found that the configuration files made debugging quicker. Furthermore, using KubeVirt could come with additional benefits due to its declarative design. For instance, managing VMs through manifests opens up the possibility of using GitOps, which removes the management-related burden from a user by providing version control and auditing. However, as mentioned throughout this report, Kubernetes still comes with a steep learning curve, and the benefits suggested here are only fully utilized if the person using is familiar with Kubernetes and its concepts.

Chapter 7

Conclusions

This project has successfully evaluated KubeVirt with Kubernetes as a VM management system. It used a mixed-method approach that consisted of a literature study, a performance analysis and a usability analysis to create a general image of the applicability of KubeVirt in a real-world scenario. The literature study has delved into both KubeVirt and Kubernetes as technologies, but also what led to KubeVirt's creation. It found that since KubeVirt is based on Kubernetes, it inherits many of its features and workflows. This enables VM created with KubeVirt to integrate well with the Kubernetes ecosystem and other Kubernetes resources, but also integrations that otherwise were only available for containers. The performance analysis found that KubeVirt comes with a larger base overhead than the compared traditional VM management system, which indicates that KubeVirt might not be suitable for all kinds of scenarios. However, it also found that KubeVirt appears to be more scalable due to its low induced load. The usability analysis found that users are generally keen on the idea of using a Kubernetes-based approach, but that it comes with a steep learning curve. Additionally, external resources such as documentation were found to be lacking, which indicates that KubeVirt is not yet fully developed. In summary, this project has shown that KubeVirt has its use cases, but that it should be implemented with care. Where if Kubernetes is not well received in an organization, KubeVirt probably won't be either.

Chapter 8

Future Work

The discussion chapter brought up many possible future research directions, related to both performance and usability. These include exploring the source of the overhead discrepancy found in the performance analysis, and why KubeVirt experiences are larger overhead than OpenNebula. Additionally, future work could investigate the usability of KubeVirt is different depending on the previous VM management system. This could be done by conducting a usability analysis with people experienced in various systems. Lastly, since VMs are often used to host applications with strict security requirements, it could be an interesting future research topic to evaluate if KubeVirt with Kubernetes provides as secure environments as used today.

Bibliography

[1] Awodele Oludele et al. "On the evolution of virtualization and cloud computing: A review". In: *Journal of Computer Sciences and Applications* 2.3 (2014), pp. 40–43.

[2] Susanta Nanda Tzi-cker Chiueh and Stony Brook. "A survey on virtualization technologies". In: *Rpe Report* 142 (2005).

[3] M. Rosenblum and T. Garfinkel. "Virtual machine monitors: current technology and future trends". In: *Computer* 38.5 (2005), pp. 39–47. DOI: 10.1109/MC.2005.176.

[4] IBM. *Virtualization*. Feb. 2024. URL: https://www.ibm.com/topics/virtualization.

[5] Ankita Desai et al. "Hypervisor: A survey on concepts and taxonomy". In: *International Journal of Innovative Technology and Exploring Engineering* 2.3 (2013), pp. 222–225.

[6] Vitor Goncalves da Silva, Marite Kirikova, and Gundars Alksnis. "Containers for Virtualization: An Overview." In: *Appl. Comput. Syst.* 23.1 (2018), pp. 21–27.

[7] Dennis Gannon, Roger Barga, and Neel Sundaresan. "Cloud-native applications". In: *IEEE Cloud Computing* 4.5 (2017), pp. 16–21.

[8] Ramzi Debab and Walid Khaled Hidouci. "Containers runtimes war: a comparative study". In: *Proceedings of the Future Technologies Conference (FTC) 2020, Volume 2*. Springer. 2021, pp. 135–161.

[9] Xingyu Wang, Junzhao Du, and Hui Liu. "Performance and isolation analysis of RunC, gVisor and Kata Containers runtimes". In: *Cluster Computing* 25.2 (2022), pp. 1497–1513.

[10] OpenStack Foundation. *OpenStack*. Accessed 2024-03-06. Mar. 2024. URL: https://www.openstack.org/.

[11] Apache CloudStack. *Apache CloudStack*. Accessed 2024-03-06. Mar. 2024. URL: https://cloudstack.apache.org/.

[12] OpenNebula. *OpenNebula*. Accessed 2024-03-06. Mar. 2024. URL: https://opennebula.io/.

[13] Kubernetes. *Kubernetes*. Accessed 2024-03-06. Mar. 2024. URL: https://kubernetes.io/.

[14] HashiCorp. *Nomad*. Accessed 2024-03-06. Mar. 2024. URL: https://www.nomadproject.io/.

[15] Docker. *Docker Swarm*. Accessed 2024-03-06. Mar. 2024. URL: https://docs.docker.com/engine/swarm/.

[16] Subrota Kumar Mondal et al. "Kubernetes in IT administration and serverless computing: An empirical study and research challenges". In: *The Journal of Supercomputing* (2022), pp. 1–51.

[17] Guoqing Li et al. "The Convergence of Container and Traditional Virtualization: Strengths and Limitations". In: *SN Computer Science* 4.4 (May 2023). https://doi.org/10.1007/s42979-023-01827-9, p. 387. ISSN: 2661-8907. DOI: 10.1007/s42979-023-01827-9.

[18] Zou Nengren September 22, Ambassador post by Zou Nengren, and September 22. *Best practices for transitioning VM clusters to KubeVirt 1.0*. Sept. 2023. URL: https://www.cncf.io/blog/2023/09/22/best-practices-for-transitioning-vm-clusters-to-kubevirt-1-0/.

[19] Marcin Kubacki. *what is Kubevirt*. Feb. 2024. URL: https://storware.eu/blog/what-is-kubevirt/.

[20] Fred Douglis and Orran Krieger. "Virtualization". In: *IEEE Internet Computing* 17.2 (2013), pp. 6–9. DOI: 10.1109/MIC.2013.42.

[21] Hasan Fayyad-Kazan, Luc Perneel, and Martin Timmerman. "Full and para-virtualization with Xen: a performance comparison". In: *Journal of Emerging Trends in Computing and Information Sciences* 4.9 (2013), pp. 719–727.

[22] Tim Abels, Puneet Dhawan, and Balasubramanian Chandrasekaran. "An overview of xen virtualization". In: *Dell Power Solutions* 8 (2005), pp. 109–111.

[23] Carl Boettiger. "An introduction to Docker for reproducible research". In: *SIGOPS Oper. Syst. Rev.* 49.1 (Jan. 2015). https://doi.org/10.1145/2723872.2723882, pp. 71–79. ISSN: 0163-5980. DOI: 10.1145/2723872.2723882.

[24] Stephen J. Bigelow. *What's the difference between type 1 vs. type 2 hypervisor?: TechTarget*. Jan. 2021. URL: https://www.techtarget.com/searchitoperations/tip/Whats-the-difference-between-Type-1-vs-Type-2-hypervisor.

[25] Hasan Fayyad-Kazan, Luc Perneel, and Martin Timmerman. "Benchmarking the performance of microsoft hyper-v server, vmware esxi and xen hypervisors". In: *Journal of Emerging Trends in Computing and Information Sciences* 4.12 (2013), pp. 922–933.

[26] Jordan Shropshire. "Analysis of monolithic and microkernel architectures: Towards secure hypervisor design". In: *2014 47th Hawaii International Conference on System Sciences*. IEEE. 2014, pp. 5008–5017.

[27] Hasan Fayyad-Kazan, Luc Perneel, and Martin Timmerman. "Benchmarking the performance of microsoft hyper-v server, vmware esxi and xen hypervisors". In: *Journal of Emerging Trends in Computing and Information Sciences* 4.12 (2013), pp. 922–933.

[28] Pankaj Kadam and Nilesh Alone. "Review on KVM Hypervisor". In: *International Journal of Recent Technology and Engineering (IJRTE) ISSN* (2014), pp. 2277–3878.

[29] David Freet et al. "Open source cloud management platforms and hypervisor technologies: A review and comparison". In: *SoutheastCon 2016* (2016), pp. 1–8.

[30] Xiaolong Wen et al. "Comparison of open-source cloud management platforms: OpenStack and OpenNebula". In: *2012 9th International Conference on Fuzzy Systems and Knowledge Discovery*. IEEE. 2012, pp. 2457–2461.

[31] Google Cloud. *High Performance Computing*. Accessed 2024-03-19. Mar. 2024. URL: https://cloud.google.com/discover/what-is-high-performance-computing.

[32] Oracle. *High Performance Computing*. Accessed 2024-03-19. Mar. 2024. URL: https://www.oracle.com/se/cloud/hpc/what-is-hpc/.

[33] Microsoft. *Availability Zones*. Accessed 2024-03-19. Mar. 2024. URL: https://learn.microsoft.com/en-us/azure/reliability/availability-zones-overview?tabs=azure-cli.

[34] Maged Michael et al. "Scale-up x scale-out: A case study using nutch/lucene". In: *2007 IEEE International Parallel and Distributed Processing Symposium*. IEEE. 2007, pp. 1–8.

[35] IBM. *Cloud scalability: Scale-up vs. scale-out*. Accessed 2024-03-19. Mar. 2024. URL: https://www.ibm.com/blog/cloud-scalability-scale-up-vs-scale-out/.

[36] Sunilkumar S Manvi and Gopal Krishna Shyam. "Resource management for Infrastructure as a Service (IaaS) in cloud computing: A survey". In: *Journal of network and computer applications* 41 (2014), pp. 424–440.

[37] Chnar Mustafa Mohammed and Subhi RM Zeebaree. "Sufficient comparison among cloud computing services: IaaS, PaaS, and SaaS: A review". In: *International Journal of Science and Business* 5.2 (2021), pp. 17–30.

[38] Tiago Rosado and Jorge Bernardino. "An overview of openstack architecture". In: *Proceedings of the 18th International Database Engineering & Applications Symposium*. 2014, pp. 366–367.

[39] Rakesh Kumar et al. "Apache cloudstack: Open source infrastructure as a service cloud computing platform". In: *Proceedings of the International Journal of advancement in Engineering technology, Management and Applied Science* 111 (2014), p. 116.

[40] Anurag Choudhary, Pradeep Kumar Verma, and Piyush Rai. "A walkthrough of Amazon Elastic Compute Cloud (Amazon EC2): A Review". In: *International Journal for Research in Applied Science and Engineering Technology* 9.11 (2021), pp. 93–97.

[41] Marshall Copeland et al. "Getting started with Azure virtual machines". In: *Microsoft Azure: Planning, Deploying, and Managing Your Data center in the Cloud* (2015), pp. 147–176.

[42] SPT Krishnan et al. "Google compute engine". In: *Building Your Next Big Thing with Google Cloud Platform: A Guide for Developers and Enterprise Architects* (2015), pp. 53–81.

[43] Apache CloudStack. *CloudStack Live Migration*. Feb. 2024. URL: https://svn.apache.org/repos/asf/cloudstack/docsite/html/docs/en-US/Apache_CloudStack/4.2.0/html/Admin_Guide/manual-live-migration.html.

[44] Daniel Moritz et al. "Enhancing a virtual security lab with a private cloud framework". In: *Proceedings of 2013 IEEE International Conference on Teaching, Assessment and Learning for Engineering (TALE)*. IEEE. 2013, pp. 314–320.

[45] Younes Khair, Abdeslam Dennai, and Youssef Elmir. "An experimental performance evaluation of OpenNebula and eucalyptus cloud platform solutions". In: *Artificial Intelligence and Heuristics for Smart Energy Efficiency in Smart Cities: Case Study: Tipasa, Algeria*. Springer. 2022, pp. 450–457.

[46] Amazon Web Services. *Amazon EC2*. Feb. 2024. URL: `https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html`.

[47] Rawan Aljamal, Fahed Jubair, and Ali El-Mousa. "Analyzing the Cost-Effectiveness of Running HPC Applications on Microsoft Azure Virtual Machines". In: *2021 IEEE Jordan International Joint Conference on Electrical Engineering and Information Technology (JEEIT)*. IEEE. 2021, pp. 78–82.

[48] Microsoft. *Azure Virtual Machines*. Feb. 2024. URL: `https://learn.microsoft.com/en-us/azure/virtual-machines/overview`.

[49] TechTarget. *Google Compute Engine*. Feb. 2024. URL: `https://www.techtarget.com/searchaws/definition/Google-Compute-Engine#:~:text=Google%20Compute%20Engine%20(GCE)%20is,compute%20clusters%20for%20that%20purpose.`.

[50] Solanke Vikas et al. "Private vs public cloud". In: *International Journal of Computer Science & Communication Networks* 3.2 (2013), p. 79.

[51] Guruh Aryotejo, Daniel Y Kristiyanto, et al. "Hybrid cloud: bridging of private and public cloud computing". In: *Journal of Physics: Conference Series*. Vol. 1025. 1. IOP Publishing. 2018, p. 012091.

[52] Nane Kratzke. "About microservices, containers and their underestimated impact on network performance". In: *arXiv preprint arXiv:1710.04049* (2017).

[53] Arnaldo Pereira Ferreira and Richard Sinnott. "A performance evaluation of containers running on managed kubernetes services". In: *2019 IEEE international conference on cloud computing technology and science (CloudCom)*. IEEE. 2019, pp. 199–208.

[54] Docker. *Layers*. Feb. 2024. URL: https://docs.docker.com/build/guide/layers/.

[55] Docker. *Dockerfile reference*. Feb. 2024. URL: https://docs.docker.com/reference/dockerfile/.

[56] Carl Boettiger. "An introduction to Docker for reproducible research". In: *ACM SIGOPS Operating Systems Review* 49.1 (2015), pp. 71–79.

[57] Steve Buchanan et al. "Container registries". In: *Introducing Azure Kubernetes Service: A Practical Guide to Container Orchestration* (2020), pp. 17–34.

[58] Red Hat. *What is a container registry?* Feb. 2024. URL: https://www.redhat.com/en/topics/cloud-native-apps/what-is-a-container-registry.

[59] Open Container Initiative. *Open Container Initiative*. Feb. 2024. URL: https://opencontainers.org/.

[60] Arun Gurta. "Kubernetes". In: *Kubernetes. Retrieved May* 24 (2019), p. 2019.

[61] Kubernetes. *Pods*. Accessed 2024-03-19. Mar. 2024. URL: https://kubernetes.io/docs/concepts/workloads/pods/.

[62] Kubernetes. *Sidecar containers*. Accessed 2024-04-20. Mar. 2024. URL: https://kubernetes.io/docs/concepts/workloads/pods/sidecar-containers/.

[63] Kubernetes. *Services*. Accessed 2024-03-19. Mar. 2024. URL: https://kubernetes.io/docs/concepts/services-networking/service/.

[64] Kubernetes. *Deployments*. Accessed 2024-03-19. Mar. 2024. URL: https://kubernetes.io/docs/concepts/workloads/controllers/deployment/.

[65] Kubernetes. *Persistent Volumes*. Accessed 2024-03-19. Mar. 2024. URL: https://kubernetes.io/docs/concepts/storage/persistent-volumes/.

[66] Kubernetes. *StatefulSets*. Accessed 2024-03-19. Mar. 2024. URL: https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/.

[67] Kakoli Banerjee et al. "A Survey on Kubernetes Policy Report Custom Resource Definition Kube-Bench Adapter". In: *Advances in Data and Information Sciences: Proceedings of ICDIS 2022*. Springer, 2022, pp. 315–322.

[68] Kubernetes. *Custom Resource Definitions*. Feb. 2024. URL: https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/.

[69] Kubernetes. *Controllers*. Accessed 2024-03-20. Mar. 2024. URL: https://kubernetes.io/docs/concepts/architecture/controller/.

[70] Kubernetes. *Kubernetes Components*. Accessed 2024-04-20. Mar. 2024. URL: https://kubernetes.io/docs/concepts/overview/components/.

[71] Kubernetes. *kube-proxy*. Accessed 2024-03-20. Mar. 2024. URL: https://kubernetes.io/docs/reference/command-line-tools-reference/kube-proxy/.

[72] Kubernetes. *kubelet*. Accessed 2024-03-20. Mar. 2024. URL: https://kubernetes.io/docs/reference/command-line-tools-reference/kubelet/.

[73] Prateek Sharma et al. "Containers and virtual machines at scale: A comparative study". In: *Proceedings of the 17th international middleware conference*. 2016, pp. 1–13.

[74] Alessandro Randazzo and Ilenia Tinnirello. "Kata containers: An emerging architecture for enabling mec services in fast and secure way". In: *2019 Sixth International Conference on Internet of Things: Systems, Management and Security (IOTSMS)*. IEEE. 2019, pp. 209–214.

[75] Shashank Mohan Jain and Shashank Mohan Jain. "Cgroups". In: *Linux Containers and Virtualization: A Kernel Perspective* (2020), pp. 45–80.

[76] Amazon Web Services. *Noisy Neighbor*. Feb. 2024. URL: https://docs.aws.amazon.com/wellarchitected/latest/saas-lens/noisy-neighbor.html.

[77] Prateek Sharma et al. "Containers and virtual machines at scale: A comparative study". In: *Proceedings of the 17th international middleware conference*. 2016, pp. 1–13.

[78] Jinchun Kim et al. “Dynamic memory pressure aware ballooning”. In: *Proceedings of the 2015 International Symposium on Memory Systems*. 2015, pp. 103–112.

[79] Ishan Banerjee et al. “Memory overcommitment in the ESX server”. In: *VMware Technical Journal* 2.1 (2013), pp. 2–12.

[80] Christopher Clark et al. “Live migration of virtual machines”. In: *Proceedings of the 2nd conference on Symposium on Networked Systems Design & Implementation-Volume 2*. 2005, pp. 273–286.

[81] Andrey Mirkin, Alexey Kuznetsov, and Kir Kolyshkin. “Containers checkpointing and live migration”. In: *Proceedings of the Linux Symposium*. Vol. 2. 2008, pp. 85–90.

[82] Radostin Stoyanov and Martin J Kollingbaum. “Efficient live migration of linux containers”. In: *High Performance Computing: ISC High Performance 2018 International Workshops, Frankfurt/Main, Germany, June 28, 2018, Revised Selected Papers 33*. Springer. 2018, pp. 184–193.

[83] CRIU. *CRIU*. Feb. 2024. URL: https://criu.org/.

[84] Red Hat. *Containers vs VMs*. Feb. 2024. URL: https://www.redhat.com/en/topics/containers/containers-vs-vms.

[85] GitHub. *Self-hosted runner security*. Accessed 2024-04-20. Mar. 2024. URL: https://docs.github.com/en/actions/hosting-your-own-runners/managing-self-hosted-runners/about-self-hosted-runners#self-hosted-runner-security.

[86] Kong. *What Are Virtual Machines (VMs)?* Feb. 2024. URL: https://konghq.com/learning-center/service-mesh/virtual-machines.

[87] Chris Greamo and Anup Ghosh. “Sandboxing and virtualization: Modern tools for combating malware”. In: *IEEE Security & Privacy* 9.2 (2011), pp. 79–82.

[88] Tripwire. *What is Configuration Drift?* Feb. 2024. URL: https://www.tripwire.com/state-of-security/what-is-configuration-drift.

[89] Shared Rotlevi. *Configuration Drift Explained*. Feb. 2024. URL: https://www.wiz.io/academy/configuration-drift.

[90] Sysdig. *Containers vs. Virtual Machines – Making an Informed Decision*. Feb. 2024. URL: https://sysdig.com/learn-cloud-native/kubernetes-security/containers-vs-virtual-machines-making-an-informed-decision/.

[91] Microsoft. *Virtual machines: virtual computers within computers*. Feb. 2024. URL: https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-a-virtual-machine.

[92] Anjali, Tyler Caraza-Harter, and Michael M Swift. "Blending containers and virtual machines: a study of firecracker and gVisor". In: *Proceedings of the 16th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*. 2020, pp. 101–113.

[93] Anjali, Tyler Caraza-Harter, and Michael M Swift. "Blending containers and virtual machines: a study of firecracker and gVisor". In: *Proceedings of the 16th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*. 2020, pp. 101–113.

[94] Kubernetes. *kube-scheduler*. Accessed 2024-02-29. Feb. 2024. URL: https://kubernetes.io/docs/reference/command-line-tools-reference/kube-scheduler/.

[95] KubeVirt. *KubeVirt Architecture*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/architecture/.

[96] KubeVirt. *API Validation*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/operations/api_validation/.

[97] KubeVirt. *Virtual Machine Instances*. Accessed 2024-04-21. Mar. 2024. URL: https://kubevirt.io/user-guide/virtual_machines/virtual_machine_instances/.

[98] Kubernetes. *Getting to know KubeVirt*. Accessed 2024-03-20. Mar. 2024. URL: https://kubernetes.io/blog/2018/05/22/getting-to-know-kubevirt/.

[99] KubeVirt. *Snapshot Restore API*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/operations/snapshot_restore_api/.

[100] KubeVirt. *Live Migration*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/operations/live_migration/.

[101] Kubernetes. *Persistent Volumes*. Accessed 2024-02-29. Feb. 2024. URL: https://kubernetes.io/docs/concepts/storage/persistent-volumes/.

[102] KubeVirt. *virtctl*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/operations/virtctl_client_tool/.

[103] Florian Beetz and Simon Harrer. "Gitops: The evolution of devops?" In: *IEEE Software* 39.4 (2021), pp. 70–75.

[104] KubeVirt. *GitOps*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/operations/gitops/.

[105] KubeVirt. *Comparing KubeVirt to other technologies*. Accessed 2024-02-29. Feb. 2017. URL: https://kubevirt.io/2017/technology-comparison.html.

[106] KubeVirt. *Network Binding Plugins*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/virtual_machines/network_binding_plugins/.

[107] KubeVirt. *Disks and Volumes*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/virtual_machines/disks_and_volumes/.

[108] Supakorn Trakulmaiphol and Krerk Piromsopa. "An Implementation of DNS Operator and Network Guideline for Migrating Virtual Machine to KubeVirt". In: *2022 26th International Computer Science and Engineering Conference (ICSEC)*. 2022, pp. 84–89. DOI: 10.1109/ICSEC56337.2022.10049369.

[109] Mengmei Ye et al. "Free the Turtles: Removing Nested Virtualization for Performance and Confidentiality in the Cloud". In: *2023 IEEE 16th International Conference on Cloud Computing (CLOUD)*. 2023, pp. 275–281. DOI: 10.1109/CLOUD60044.2023.00039.

[110] Oskar Sonninen. "Perceived benefits of declarative software deployment: an exploratory case study". In: (2020).

[111] Muhammad Imran et al. "Migration of CMSWEB cluster at CERN to Kubernetes: a comprehensive study". In: *Cluster Computing* 24.4 (2021), pp. 3085–3099.

[112] Rukaiah NJH Badran. *Automation and improvement in CMSWEB Kubernetes clusters*. Tech. rep.

[113] Sravanthi Gelley. “Migrate Cloud Foundry Application to Kubernetes”. In: (2022).

[114] Michael Packard et al. “Real-World, Self-Hosted Kubernetes Experience”. In: *Practice and Experience in Advanced Research Computing*. 2021, pp. 1–5.

[115] Mikhail Boronin. *Hybrid Cloud Migration Challenges. A case study at King*. 2020.

[116] Mikyung Kang et al. “A comparison of system performance on a private openstack cloud and amazon ec2”. In: *2017 IEEE 10th International Conference on Cloud Computing (CLOUD)*. IEEE. 2017, pp. 310–317.

[117] Jaison Paul Mullerikkal and Yedhu Sastri. “A comparative study of OpenStack and CloudStack”. In: *2015 Fifth International Conference on Advances in Computing and Communications (ICACC)*. IEEE. 2015, pp. 81–84.

[118] Aaron Paradowski, Lu Liu, and Bo Yuan. “Benchmarking the Performance of OpenStack and CloudStack”. In: *2014 IEEE 17th International Symposium on Object/Component/Service-Oriented Real-Time Distributed Computing*. 2014, pp. 405–412. DOI: 10.1109/ISORC.2014.12.

[119] Marcelo Amaral. *Kubevirt scale test by creating 400 vmis on a single node for FOSDEM 2022: IBM Research*. Feb. 2022. URL: https://research.ibm.com/publications/kubevirt-scale-test-by-creating-400-vmis-on-a-single-node.

[120] RISSO FULVIO GIOVANNI OTTAVIO and Alireza Kolahdouzan. “Network Automation, Orchestration, Cloud: Tools and Technologies”. In: (2019).

[121] Jinho Hwang et al. “A component-based performance comparison of four hypervisors”. In: *2013 IFIP/IEEE International Symposium on Integrated Network Management (IM 2013)*. IEEE. 2013, pp. 269–276.

[122] Robert-Steve Schmoll et al. “Comparing and evaluating application-specific boot times of virtualized instances”. In: *2019 IEEE 2nd 5G World Forum (5GWF)*. IEEE. 2019, pp. 602–606.

[123] Microsoft. *Dv3 and Dsv3 series*. Accessed 2024-03-20. Mar. 2024. URL: https://learn.microsoft.com/en-us/azure/virtual-machines/dv3-dsv3-series.

[124] Microsoft. *Azure Pricing Calculator*. Feb. 2024. URL: https://azure.microsoft.com/en-us/pricing/calculator/.

[125] The KubeVirt.io website CNCF. *KubeVirt - Disks*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/virtual_machines/disks_and_volumes/.

Appendix A

Performance Analysis Components

A.1 VM Management System Interface

```
type VmManagementSystem interface {
    Setup() error
    Get\Gls{vm}(name string) *models.\Gls{vm}
    List\Glspl{vm}() []models.\Gls{vm}
    Create\Gls{vm}(vm *models.\Gls{vm}, hostIdx ...int)
       error
    Delete\Gls{vm}(name string) error
    Migrate\Gls{vm}(name string, hostIdx int) error
    ConnectWorker(workerIdx int) error
    DisconnectWorker(workerIdx int) error
    WaitForRunning\Gls{vm}(name string) error
    DeleteAll\Glspl{vm}() error
}
```

A.2 Metrics Collector

```
#!/bin/bash

interval=1
OUTPUT_FILE="./output.json"
```

```
cpu_tmp_file=$(mktemp)
ram_tmp_file=$(mktemp)
disk_tmp_file=$(mktemp)

while true; do
    if [ ! -f "./run" ]; then
        sleep $interval
        continue
    fi

    (mpstat -P ALL "$interval" 1 | awk '/^[0-9]/␣{if␣(NR
       >1)␣print␣$3/100}' | jq -R -s -c 'split("\n")
       [1:-1]' | jq '[.[]␣|␣gsub(",";␣".")␣|␣tonumber]'
       > "$cpu_tmp_file") &

    (free | awk '/Mem:/␣{print␣$3/$2}' | sed 's/,/./' >
       "$ram_tmp_file") &

    (iostat -dx "$disk" 1 2 | awk '/Device/{flag=1;␣next
       }␣flag␣&&␣/^[^␣]/␣{util=$NF}␣END{print␣util/100}'
        | sed 's/,/./' > "$disk_tmp_file") &
    timestamp=$(date -u +"%Y-%m-%dT%H:%M:%SZ")

    wait

    name=$(hostname)
    cpu_core_utilization=$(cat "$cpu_tmp_file")
    cpu_utilization_avg=$(echo "$cpu_core_utilization" |
        jq 'add/length')
    ram_utilization=$(cat "$ram_tmp_file")
    disk_utilization=$(cat "$disk_tmp_file")

    # Create the JSON object for the current datapoint
    datapoint=$(jq -n \
                  --argjson cpuCores "
                     $cpu_core_utilization" \
                  --argjson cpuAverage "
                     $cpu_utilization_avg" \
                  --arg name "$name" \
                  --arg ramUtilization "$ram_utilization
                     " \
                  --arg diskUtilization "
```

```
                    $disk_utilization" \
                 --arg timestamp "$timestamp" \
                 '{name:␣$name,␣cpu:␣{cores:␣$cpuCores,
                    ␣average:␣$cpuAverage},␣ram:␣(
                    $ramUtilization␣|␣tonumber),␣disk:␣
                    ($diskUtilization␣|␣tonumber),␣
                    timestamp:␣$timestamp}')

    if [ ! -f "$OUTPUT_FILE" ]; then
        echo "[]" > "$OUTPUT_FILE"
    fi

    jq --argjson newData "$datapoint" '.␣+=␣[$newData]'
       "$OUTPUT_FILE" > tmp.$$.json && mv tmp.$$.json "
       $OUTPUT_FILE"

    sleep $interval
done
```

A.3 Metrics Collector Daemon

```
[Unit]
Description=Metrics scraping
[Service]
WorkingDirectory=/home/user
ExecStart=/bin/bash /home/user/metrics.sh
Restart=always
[Install]
WantedBy=multi-user.target
```

Appendix B

Performance Analysis Results

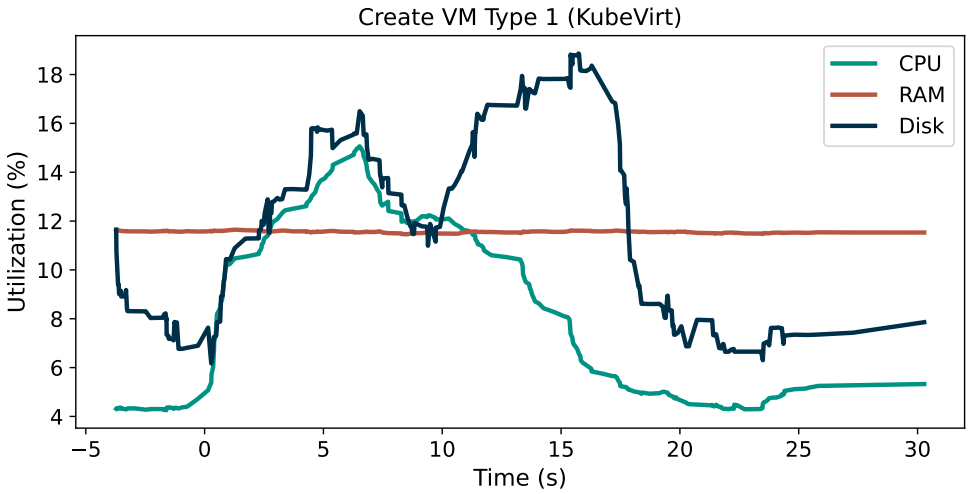

Figure B.1: Resource utilization on the control node when creating a VM type 1 in KubeVirt. CPU and disk utilization follow a similar pattern nearly reaching 20% while memory utilization remains constant at 12% for the duration of the test.

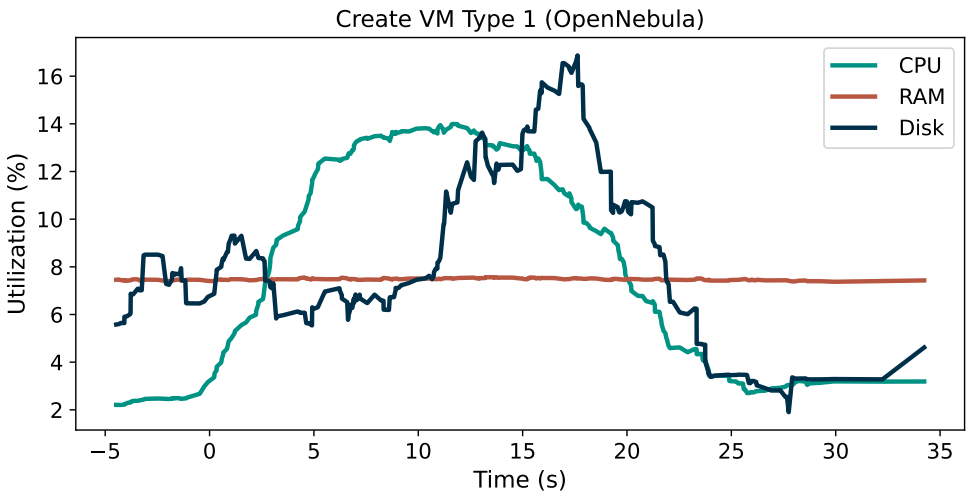

Figure B.2: Resource utilization on the control node when creating a VM type 1 in OpenNebula. CPU and disk utilization both increase and peak after half the duration of the test. Memory utilization remains constant below 8% for the duration of the test.

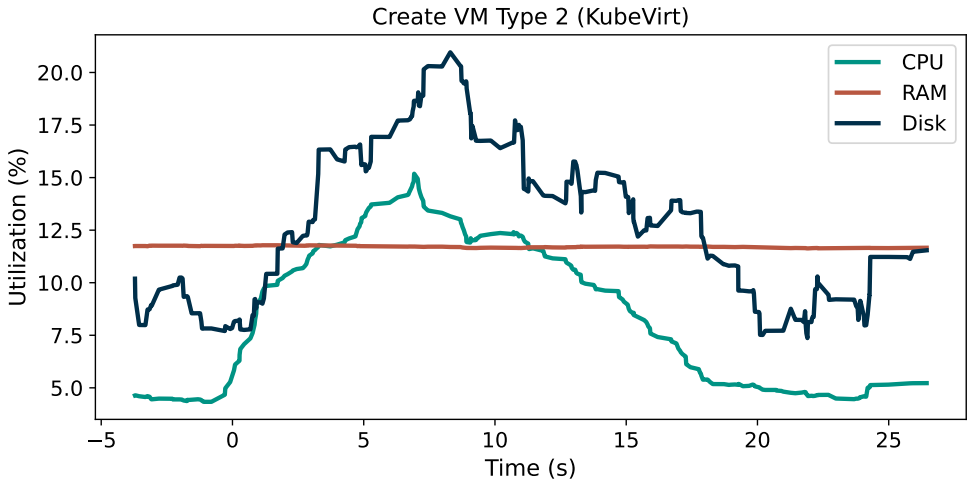

Figure B.3: Resource utilization on the control node when creating a VM type 2 in KubeVirt. The disk utilization is the most prominent among the resources, reaching over 20% utilization. Memory utilization remains constant at 12% for the duration of the test.

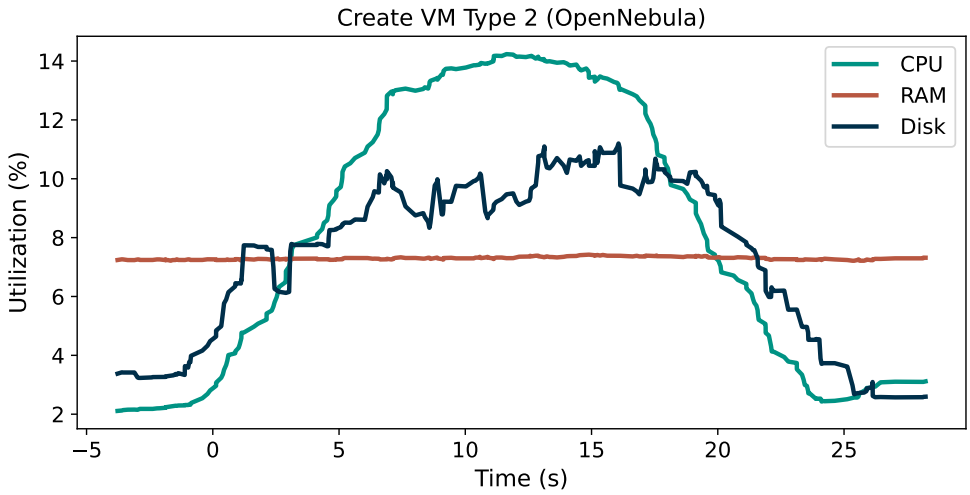

Figure B.4: Resource utilization on the control node when creating a VM type 2 in OpenNebula. Disk utilization plateaus quickly reaching just above 10% utilization while CPU utilization peaks just below 15%. Memory utilization remains constant at 8% for the duration of the test.

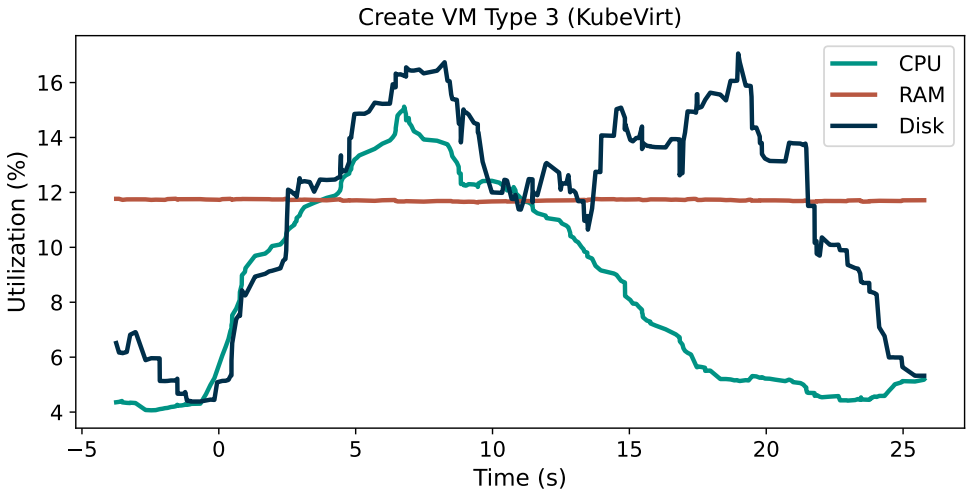

Figure B.5: Resource utilization on the control node when creating a VM type 3 in KubeVirt. CPU and disk utilization initially follow a similar pattern, but disk utilization remains high for the duration of the test. Memory utilization remains constant at 12% for the duration of the test.

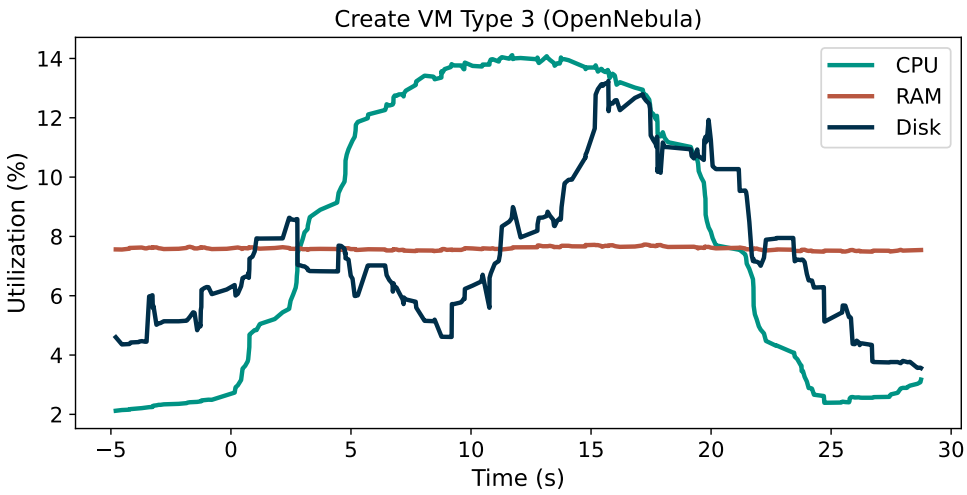

Figure B.6: Resource utilization on the control node when creating a VM type 3 in OpenNebula. CPU utilization is the most prominent among the resources reaching nearly 15% utilization. Memory utilization remains constant at 8% for the duration of the test.

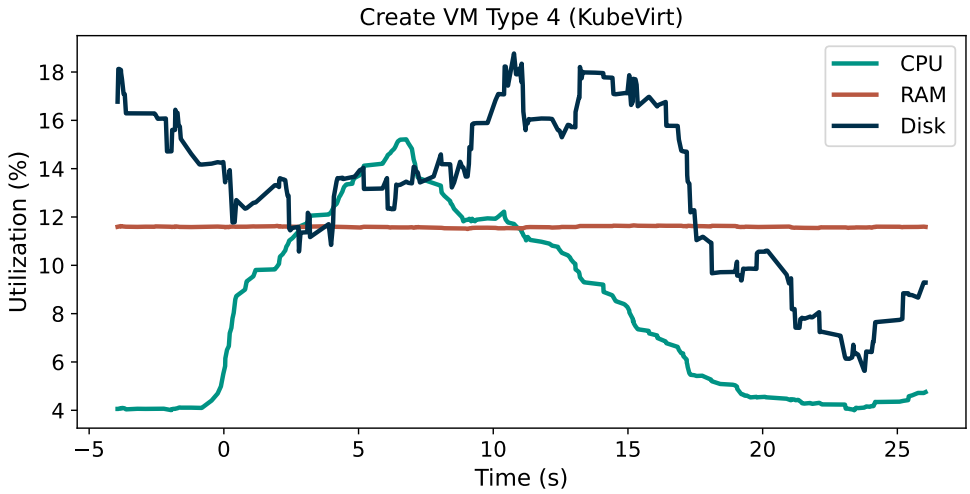

Figure B.7: Resource utilization on the control node when creating a VM type 4 in KubeVirt. CPU utilization peaks quickly while disk utilization is sporadic. The sporadic behavior is not in line with the operations performed during the test. Memory utilization remains constant at 12% for the duration of the test.

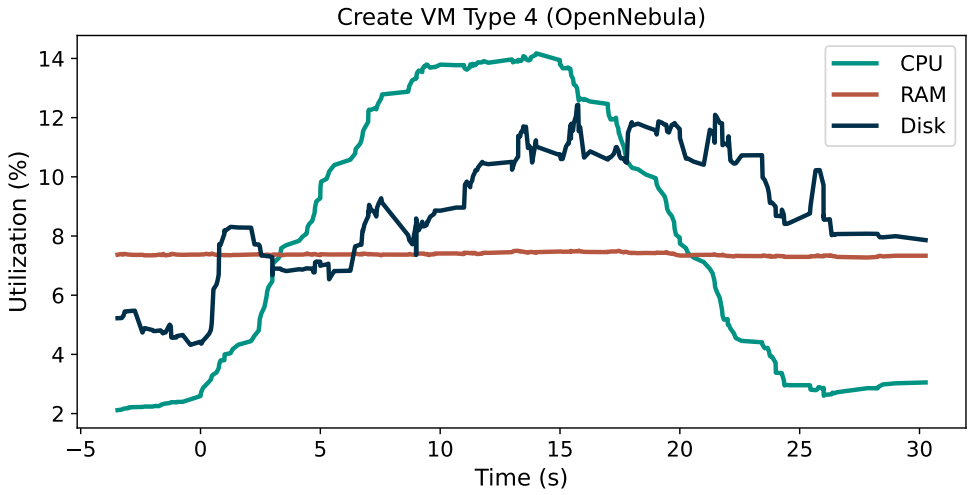

Figure B.8: Resource utilization on the control node when creating a VM type 4 in OpenNebula. CPU utilization is the most prominent among the resources plateauing at nearly 15% utilization. Memory utilization remains constant at 8% for the duration of the test.

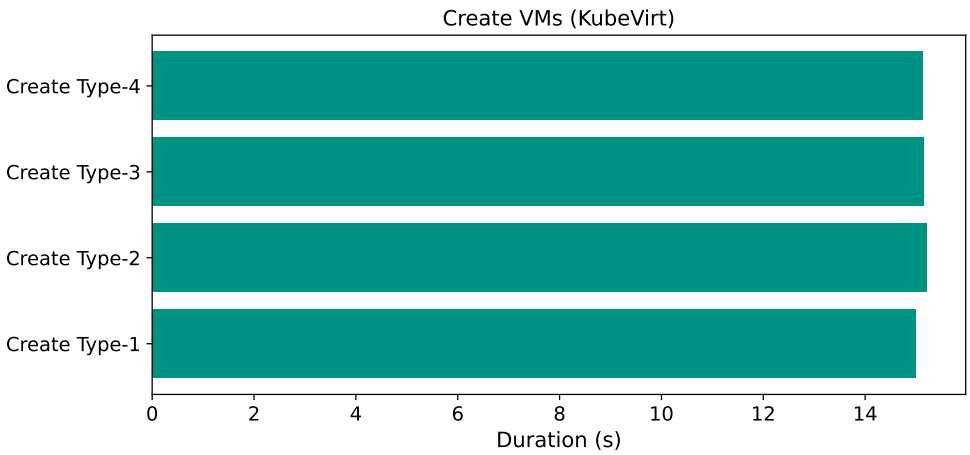

Figure B.9: Duration of creating VMs in KubeVirt. The duration for all types remains constant for each VM type reaching slightly above 14 seconds

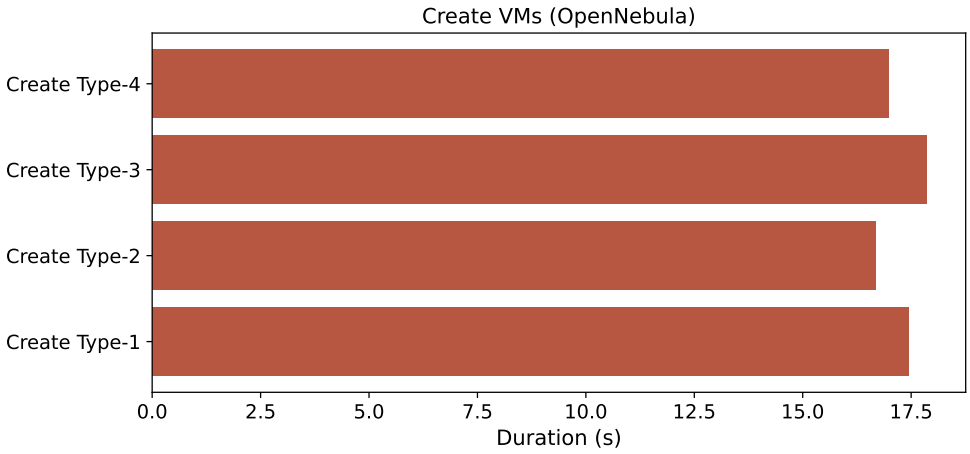

Figure B.10: Duration of creating VMs in OpenNebula. The duration for all types remains constant for each VM type reaching nearly 18 seconds.

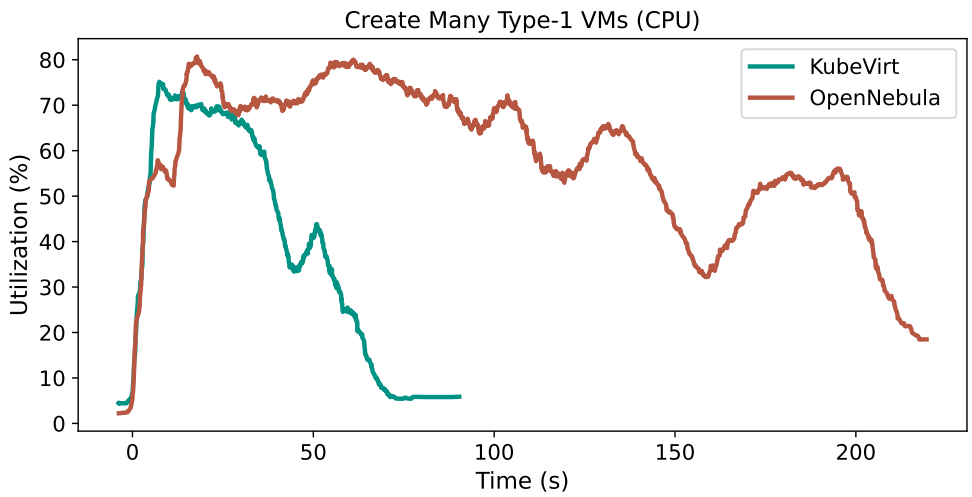

Figure B.11: CPU utilization on the control node when creating 20 type-1 VMs. The CPU utilization peaks quickly for both KubeVirt and OpenNebula due to the burst of requests. KubeVirt reaches a slightly lower peak than OpenNebula and subsides significantly faster. OpenNebula does not manage to return to a utilization level of an idle state within the duration of the test, as evidenced by the utilization not reaching base utilization in the end.

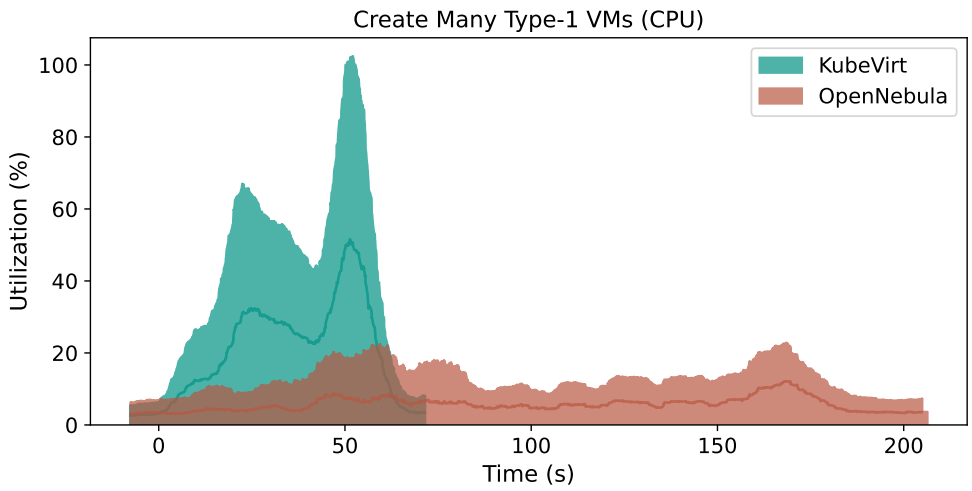

Figure B.12: Cumulative CPU utilization among worker nodes when creating 20 type-1 VMs. The CPU utilization among the worker nodes is vastly different between KubeVirt and OpenNebula, where KubeVirt experiences a short-lived large burst of CPU utilization while OpenNebula barely reaches a cumulative 20% utilization.

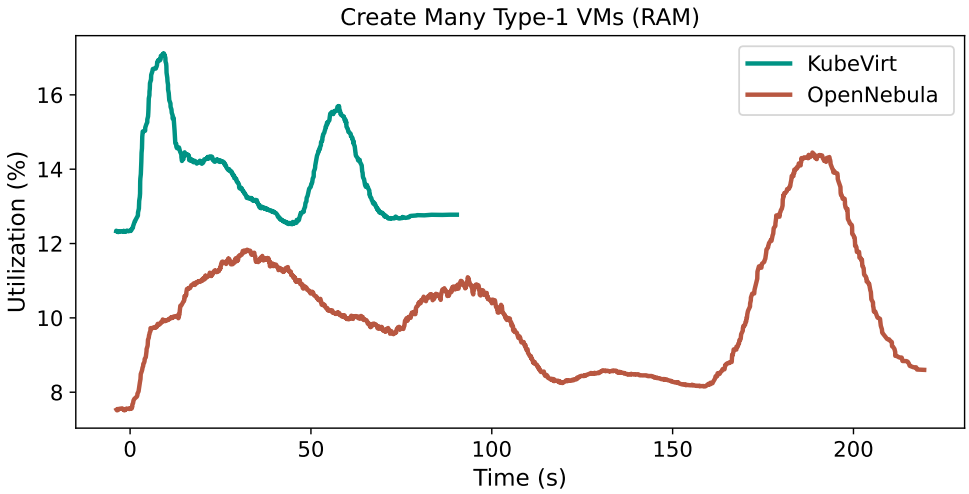

Figure B.13: Memory utilization on the control node when creating 20 type-1 VMs. KubeVirt experiences a burst of memory utilization in the beginning, while OpenNeubla has a more gradual increase before spiking quickly toward the end of the test. While KubeVirt has a larger base memory utilization, OpenNebula experiences a larger induced load from the test, as evidenced by its longer slopes.

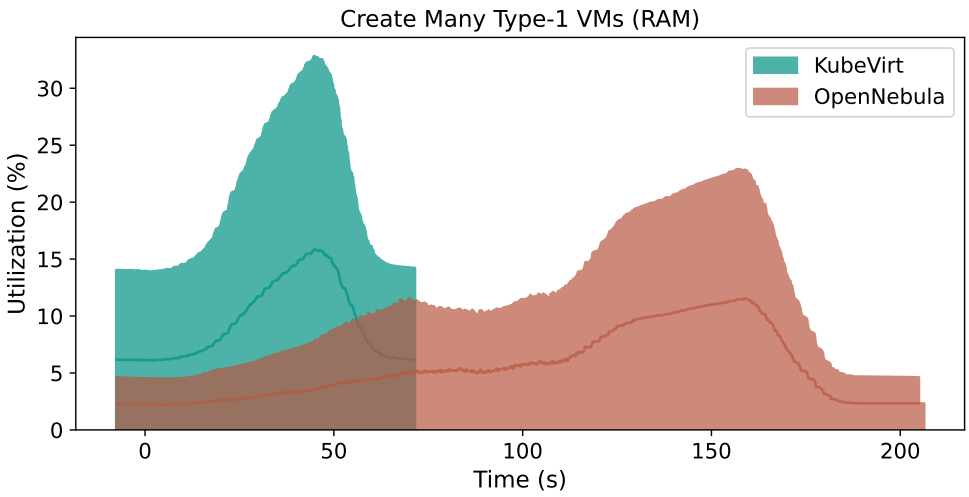

Figure B.14: Cumulative memory utilization among worker nodes when creating 20 type-1 VMs. While KubeVirt experiences a larger memory utilization due to a larger base, both systems have a similar magnitude of the induced load, albeit KubeVirt's load is more short-lived.

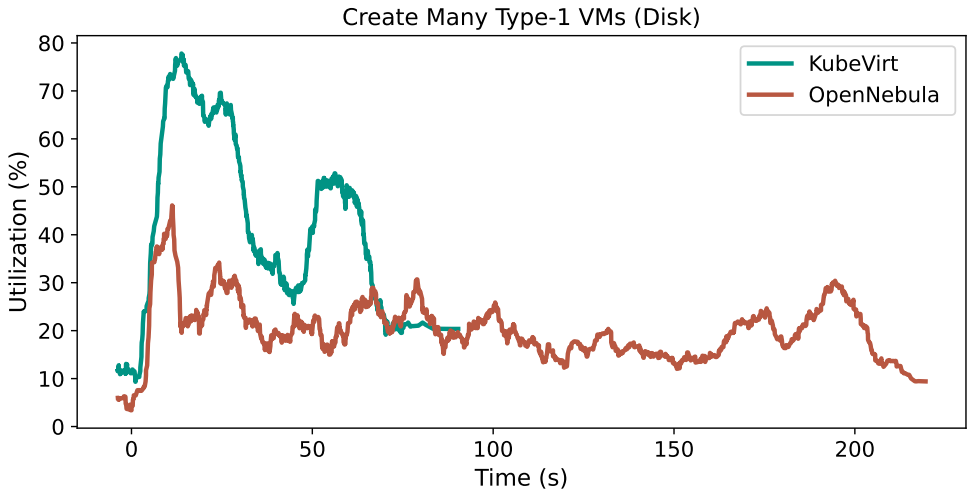

Figure B.15: Disk utilization on the control node when creating 20 type-1 VMs. KubeVirt experiences a larger burst of disk utilization than OpenNebula, while OpenNebula maintains a steady disk utilization throughout the test.

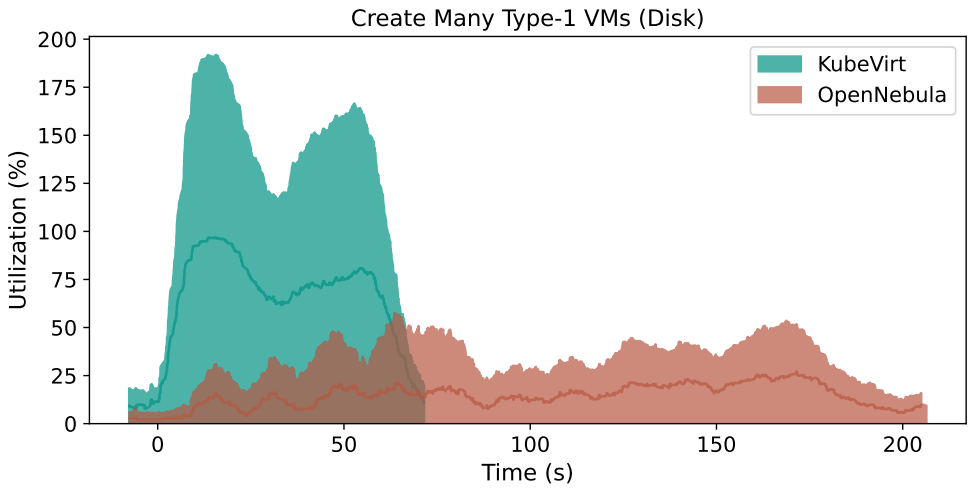

Figure B.16: Cumulative disk utilization among worker nodes when creating 20 type-1 VMs. KubeVirt experiences a larger burst of disk utilization than OpenNebula, while OpenNebula maintains a low and steady disk utilization throughout the test.

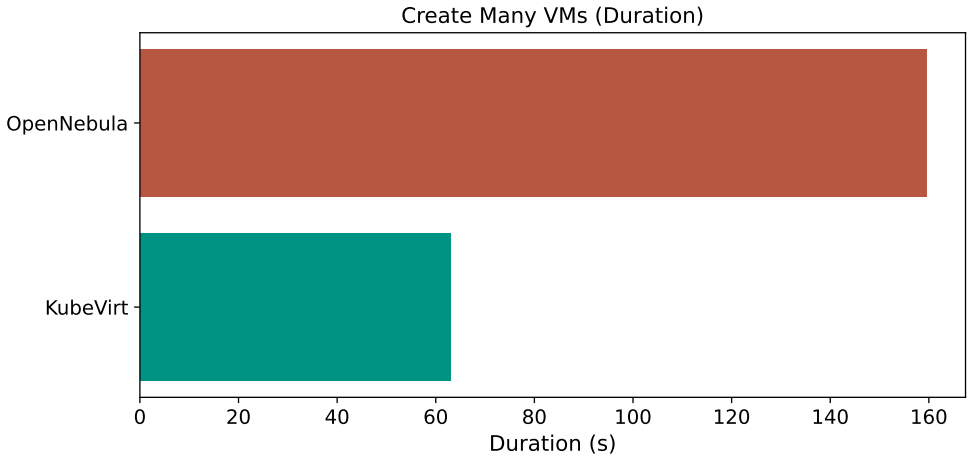

Figure B.17: Duration of creating 20 type-1 VMs. KubeVirt is significantly faster to create 20 type-1 VMs than OpenNebula, taking 67 seconds compared to OpenNebula's 192 seconds.

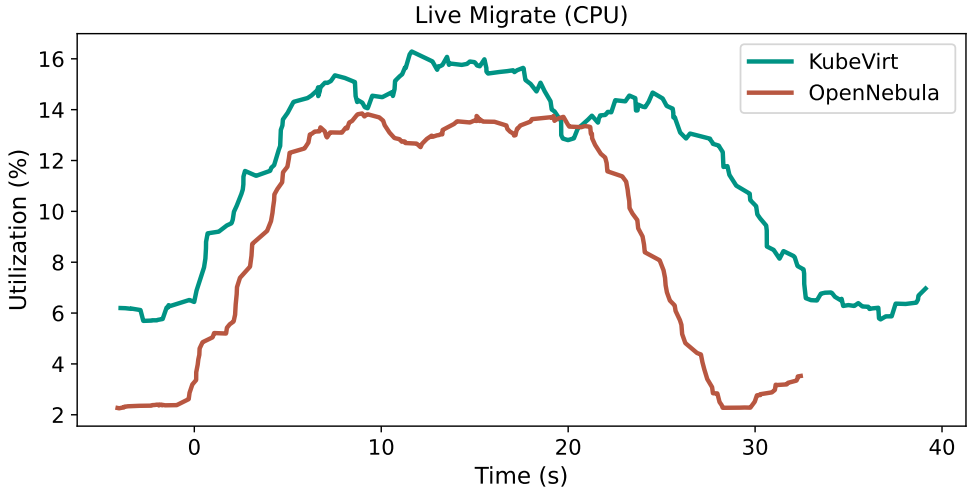

Figure B.18: CPU utilization on the control node when live migrating a type-4 VM. KubeVirt has a slightly larger base utilization than OpenNebula, but the induced load is slightly larger for OpenNebula than KubeVirt. They follow a similar pattern peaking at 16% and 14% respectively.

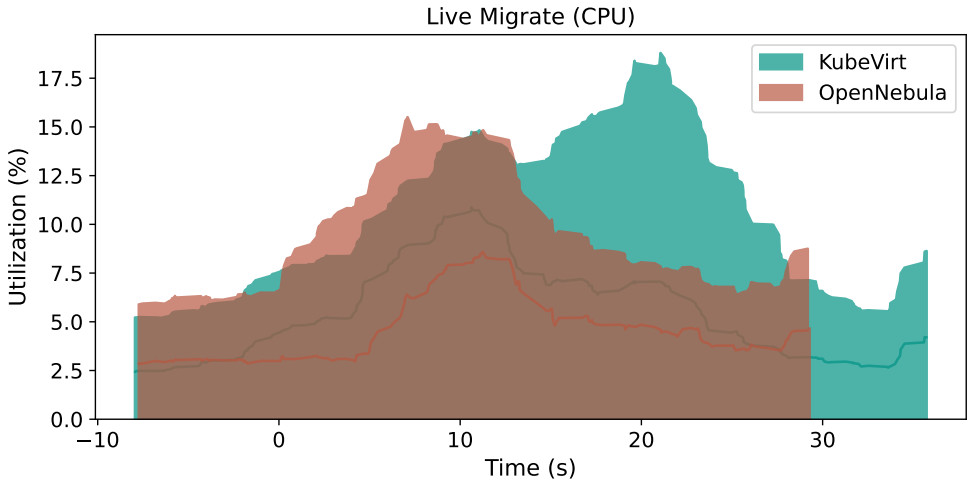

Figure B.19: Cumulative CPU utilization among worker nodes when live migrating a type-4 VM. Both systems follow a similar pattern where OpenNebula experiences a slightly lower utilization than KubeVirt. The induced load is shifted towards the end of the test for KubeVirt since it executes the operation more slowly.

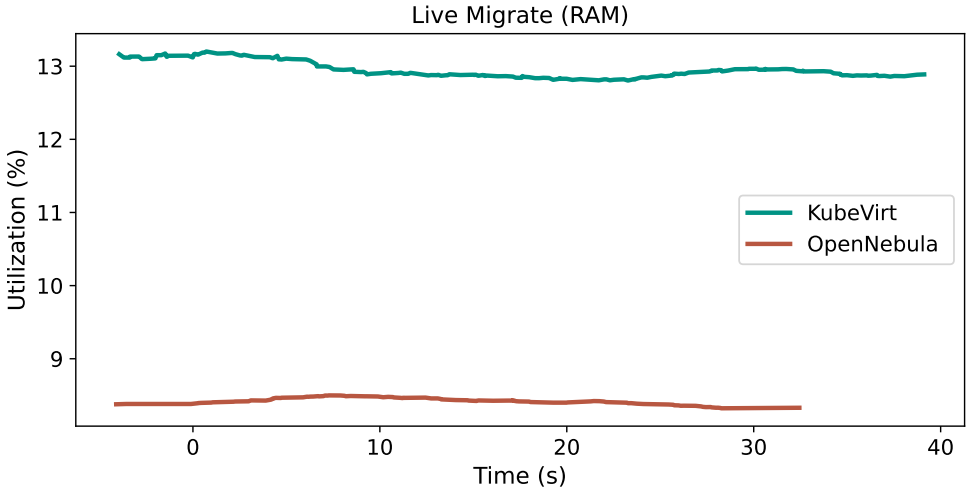

Figure B.20: Cumulative memory utilization on the control node when live migrating a type-4 VM. While KubeVirt has a larger base memory utilization than OpenNebula, the induced load is non-existent for both systems.

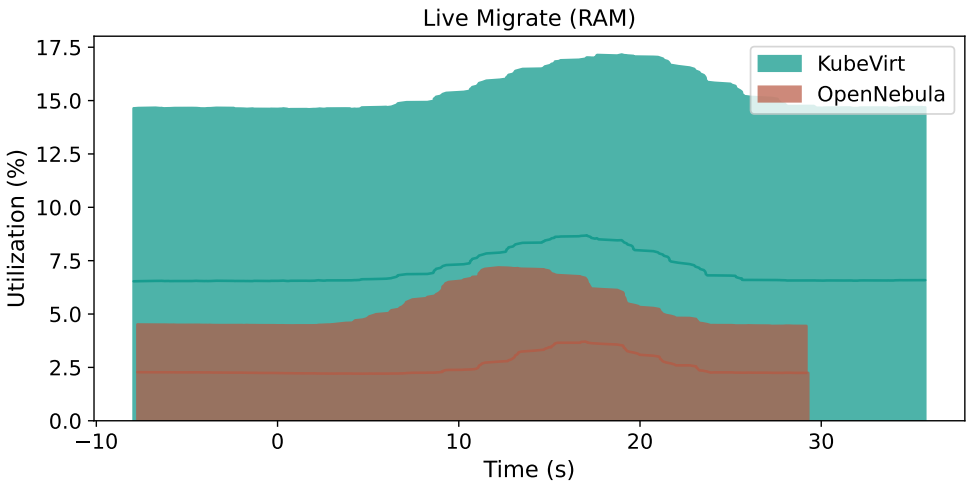

Figure B.21: Cumulative memory utilization among worker nodes when live migrating a type-4 VM. The load is similar for the two systems, where the load is shifted between the two workers over time. The overlap of the peaks between the workers shows that the migration process affects both workers simultaneously during a short period.

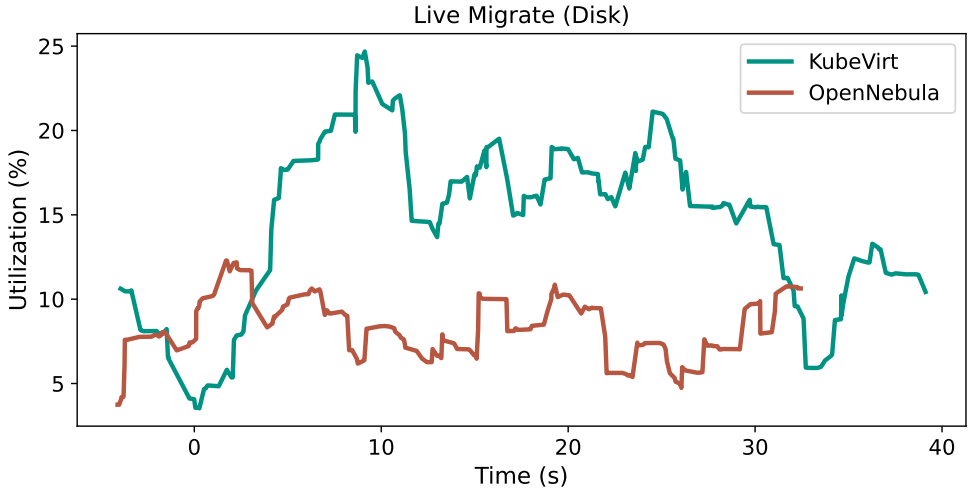

Figure B.22: Disk utilization on the control node when live migrating a type-4 VM. KubeVirt experiences a larger disk utilization than OpenNebula for the entirety of the test. However, the disk usage is sporadic and many peaks are not in line with the operations performed during the test.

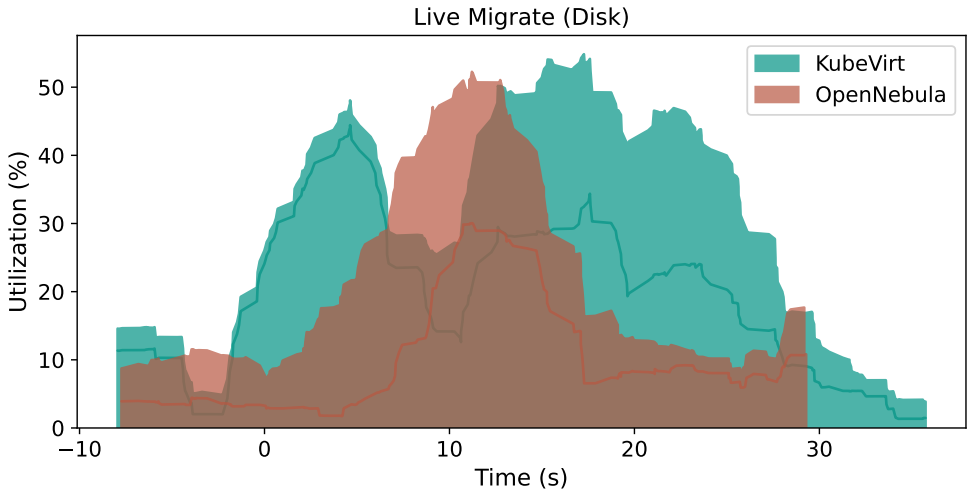

Figure B.23: Cumulative disk utilization among worker nodes when live migrating a type-4 VM. Both KubeVirt and OpenNebula reach a similar peak cumulative disk utilization. However, OpenNebula experiences a shorter plateau of disk utilization than KubeVirt.

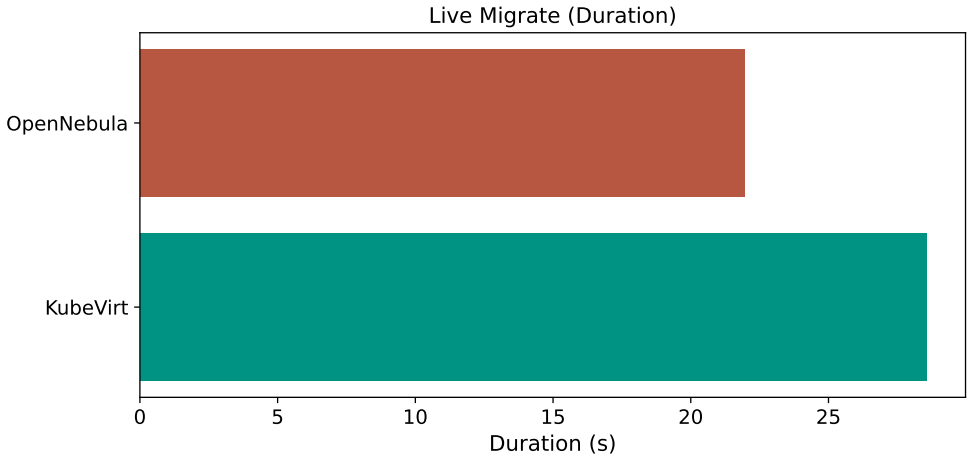

Figure B.24: Duration of live migrating a type-4 VM. KubeVirt is slower than OpenNebula, taking on average 7 seconds longer to complete. The duration is consistent for both systems.

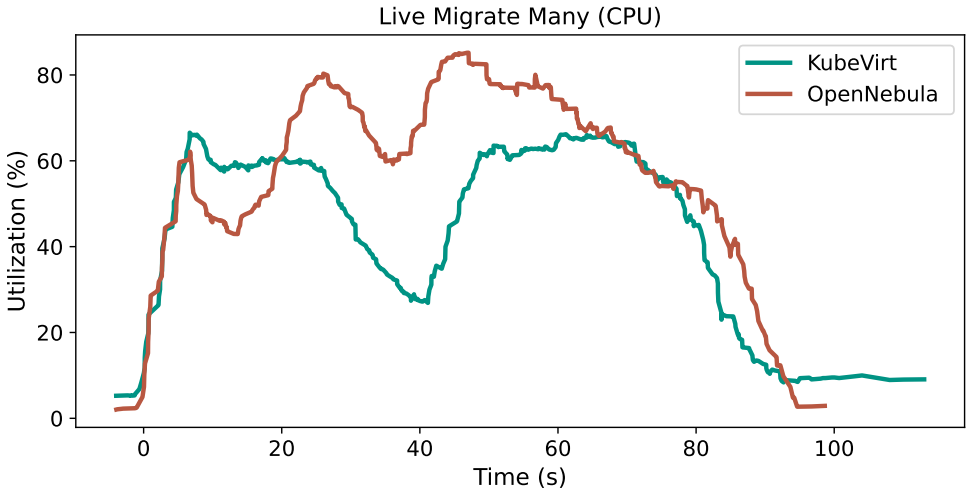

Figure B.25: CPU utilization on the control node when live migrating 10 type-1 VMs. Both systems follow a similar pattern where KubeVirt has a slightly larger base utilization than OpenNebula, but the induced load is larger for OpenNebula.

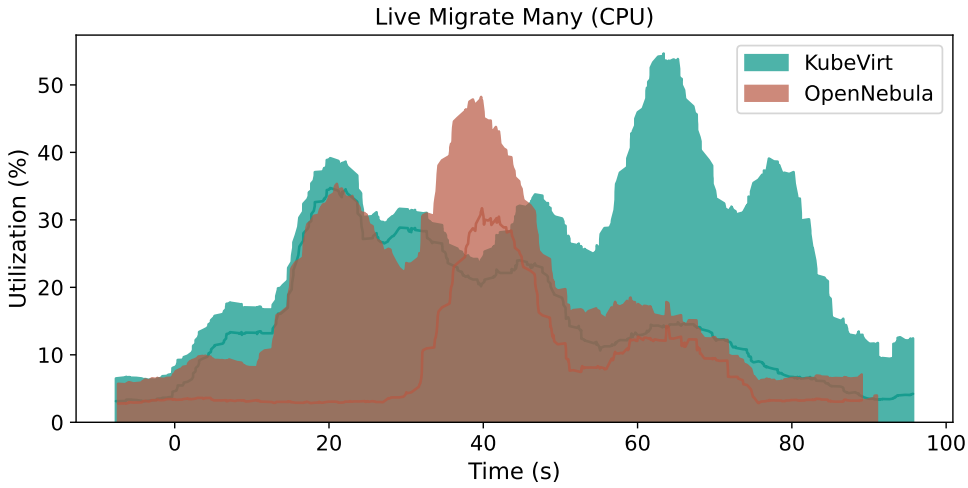

Figure B.26: Cumulative CPU utilization among worker nodes when live migrating 10 type-1 VMs. Both systems experience a similar load for the first half of the test, but KubeVirt experiences a significantly larger load in the second half of the test.

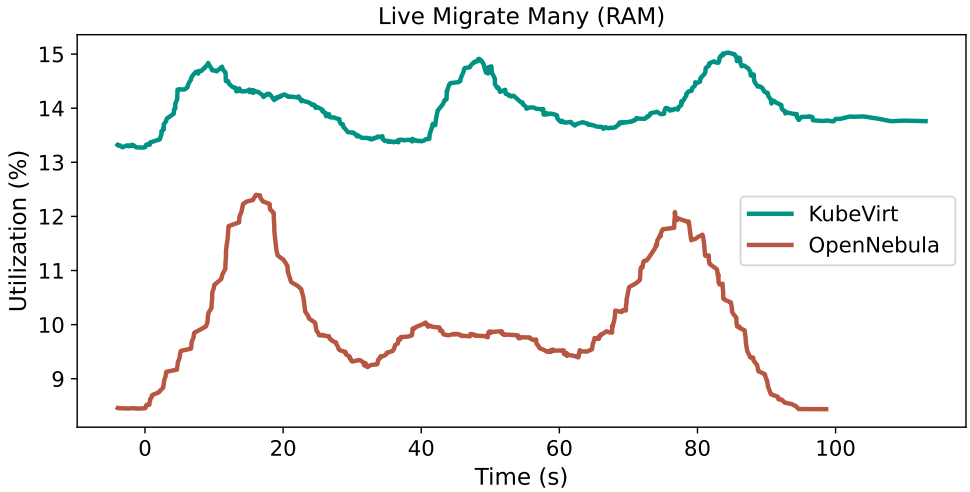

Figure B.27: Memory utilization on the control node when live migrating 10 type-1 VMs. KubeVirt has a larger base memory utilization than OpenNebula, but the induced load is more than double for OpenNebula.

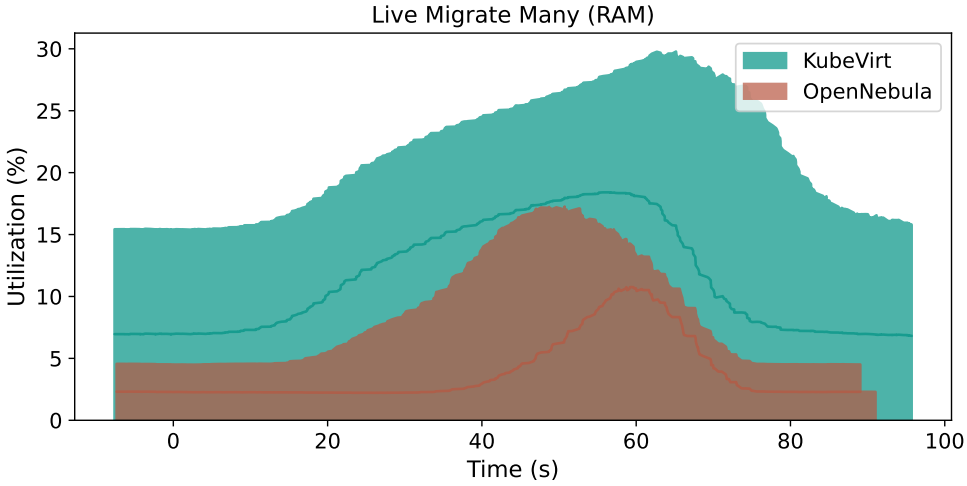

Figure B.28: Cumulative memory utilization among worker nodes when live migrating 10 type-1 VMs. KubeVirt has a larger base memory utilization than OpenNebula, but both systems experience a similar induced load. OpenNebula returns to an idle state faster than KubeVirt.

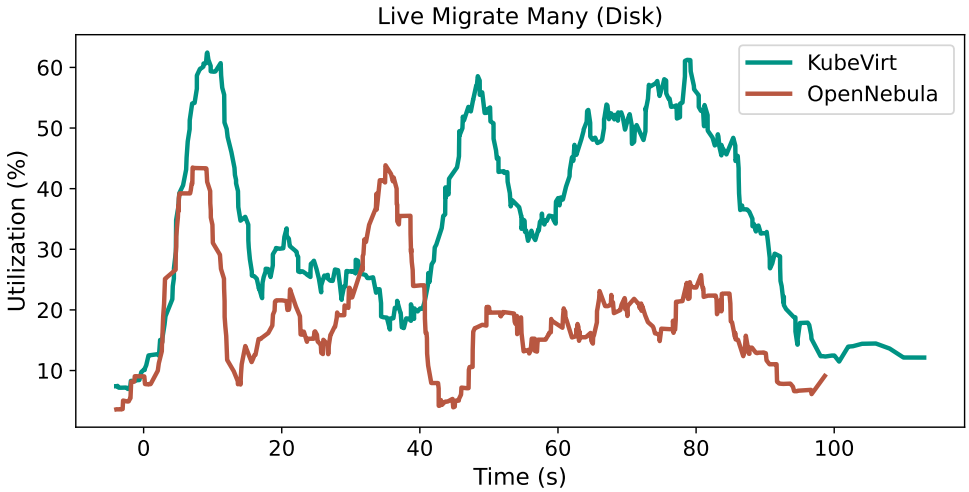

Figure B.29: Disk utilization on the control node when live migrating 10 type-1 VMs. KubeVirt has an overall larger disk utilization than OpenNebula. However, the disk utilization has sporadic peaks that are not in line with the operations performed during the test.

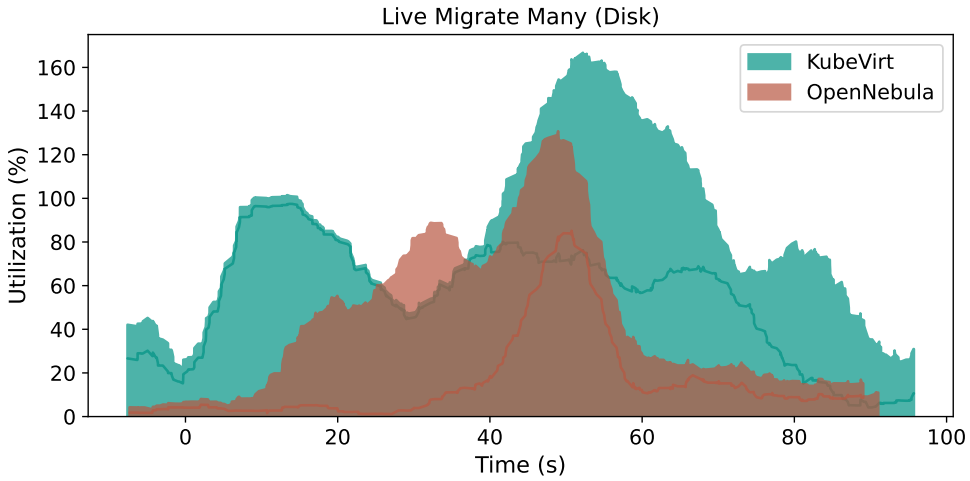

Figure B.30: Cumulative disk utilization among worker nodes when live migrating 10 type-1 VMs. KubeVirt has both a larger base utilization and cumulative disk utilization than OpenNebula. Both systems show a clear pattern of the disk utilization being shifted to a second worker node after half the duration of the test. However, KubeVirt takes a longer time to return to an idle state of the first worker node than OpenNebula.

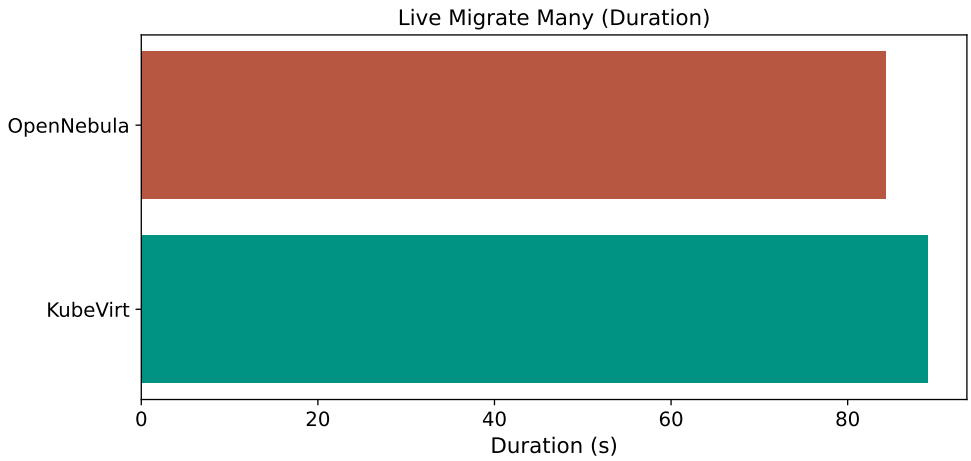

Figure B.31: Duration of live migrating 10 type-1 VMs. KubeVirt is slightly slower than OpenNebula, taking on average 3 seconds longer to complete, 87 seconds compared to OpenNebula's 84 seconds.

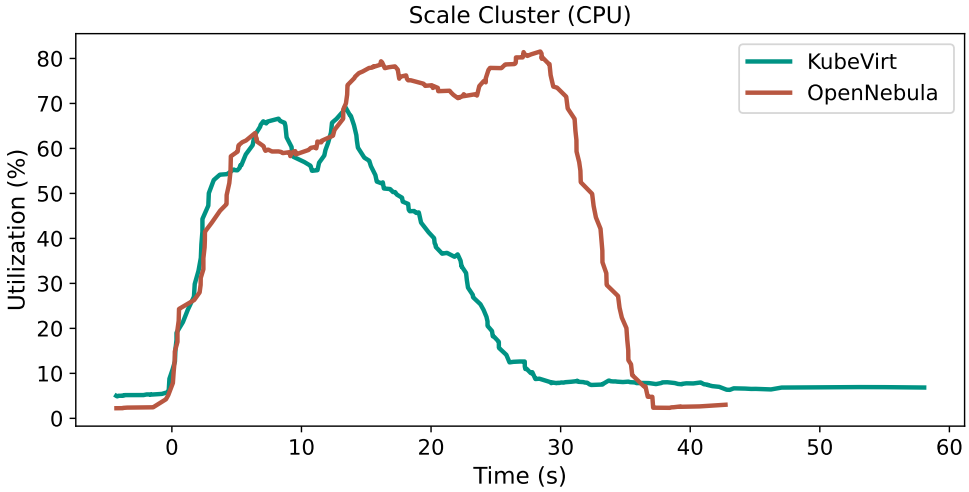

Figure B.32: CPU utilization on the control node when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes. Both systems are initially identical in CPU utilization, but KubeVirt subsides faster than OpenNebula.

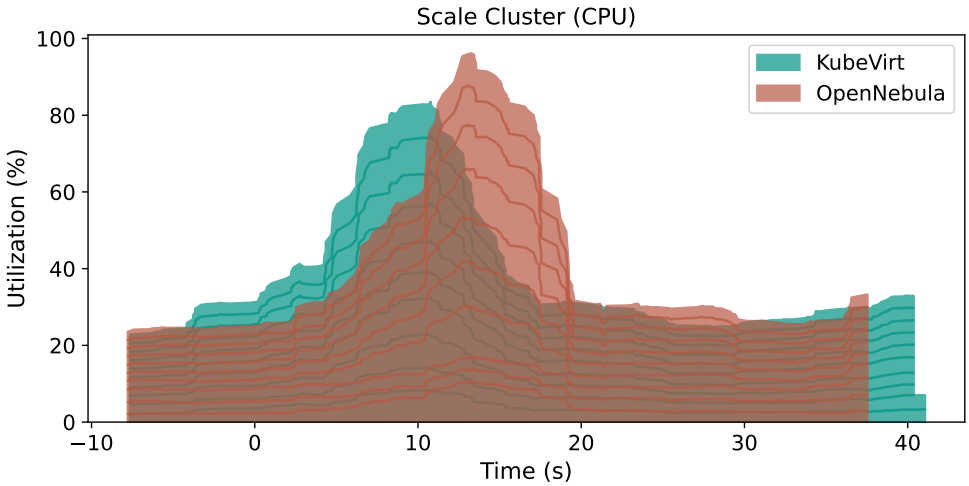

Figure B.33: Cumulative CPU utilization among worker nodes when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes. Both the base utilization and induced load are similar between the two systems. However, OpenNebula is slightly faster to complete the scaling operation than KubeVirt.

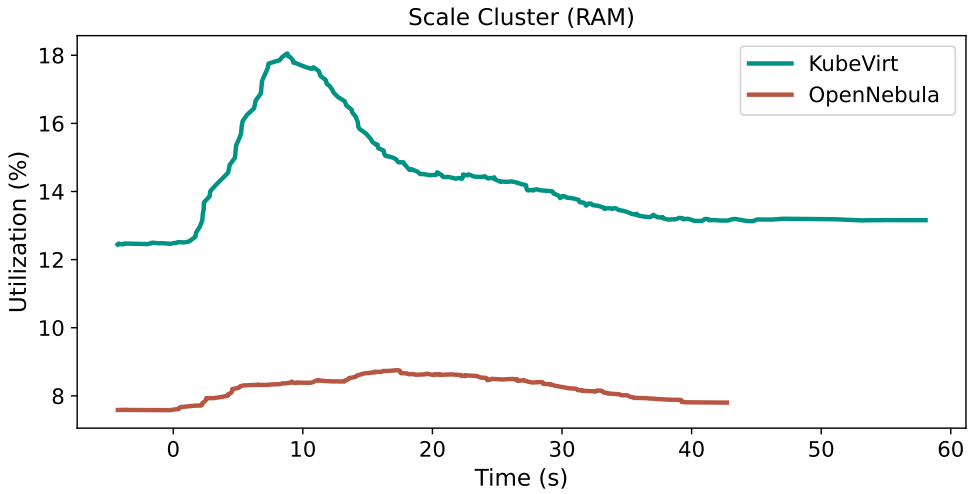

Figure B.34: Memory utilization on the control node when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes. KubeVirt experiences a significantly larger memory utilization than OpenNebula. OpenNebula's memory utilization is barely affected by the scaling operation.

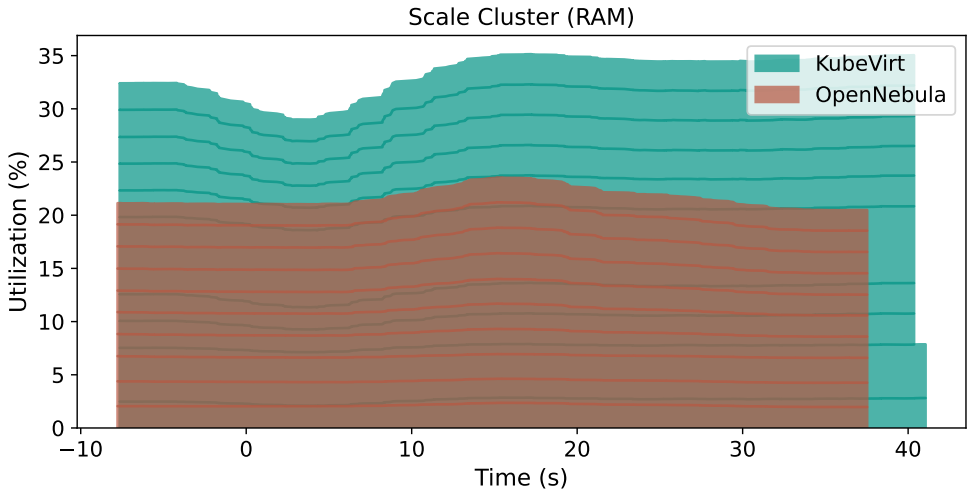

Figure B.35: Cumulative memory utilization among worker nodes when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes. KubeVirt experiences a slightly larger base memory utilization than OpenNebula, as well as a larger induced load from the scaling operation.

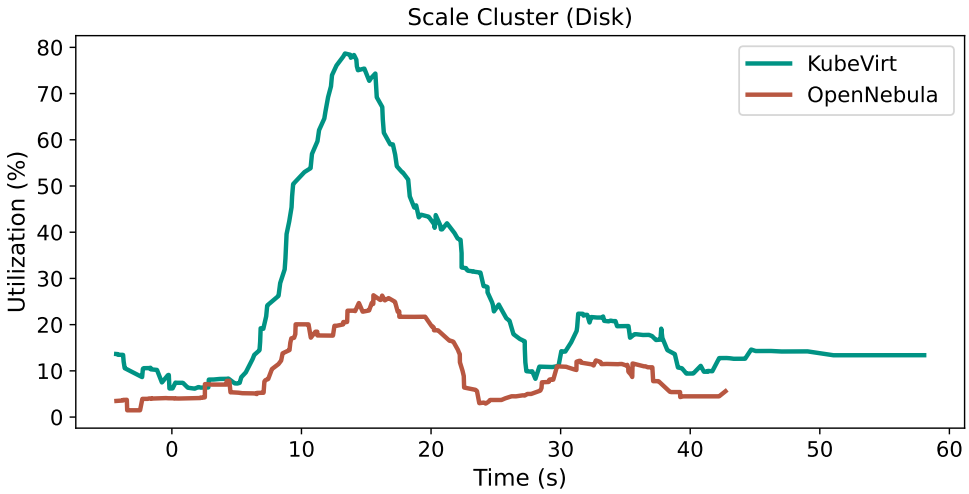

Figure B.36: Disk utilization on the control node when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes. KubeVirt experiences a rapid increase in disk utilization, reaching a peak of over 70% utilization. OpenNebula has a less aggressive increase reaching barely 30% utilization.

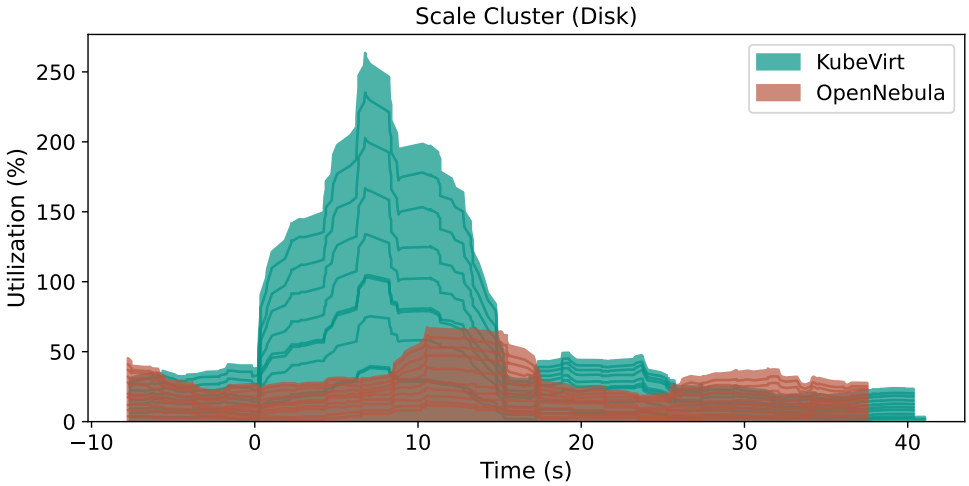

Figure B.37: Cumulative disk utilization among worker nodes when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes. KubeVirt experiences a significantly larger disk utilization than OpenNebula with a burst of disk utilization whereas OpenNebula has a more gradual increase.

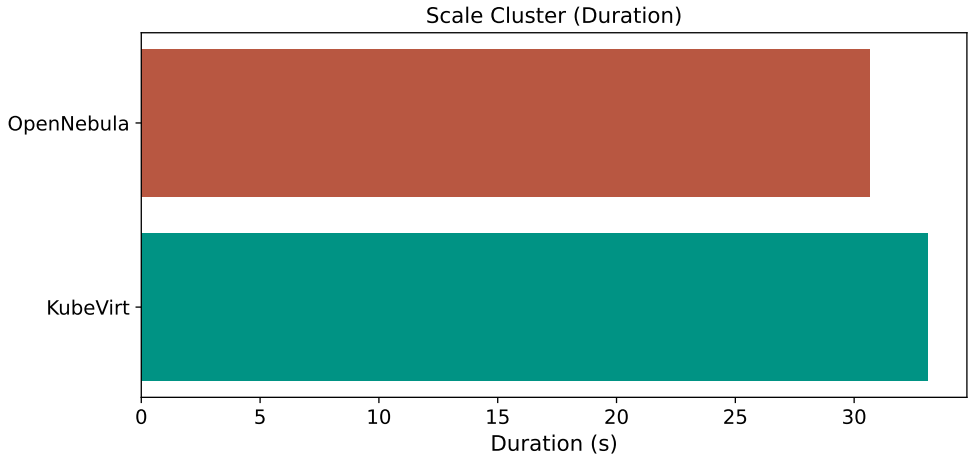

Figure B.38: Duration of scaling the cluster. KubeVirt is slightly slower than OpenNebula, taking on average 3 seconds longer to complete, 33 seconds compared to OpenNebula's 30 seconds.

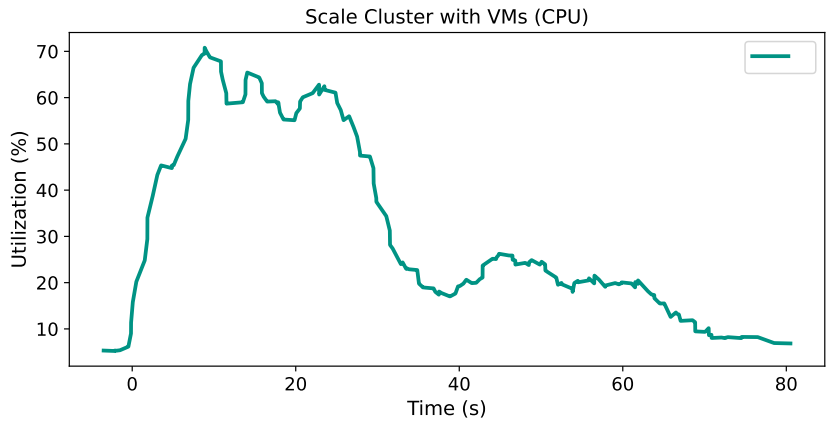

Figure B.39: CPU utilization on the control node when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes with 1 type-1 VM running on each node. KubeVirt experiences an initial burst of CPU utilization for the scale-up operation, but a significantly lower utilization for the scale-down operation. OpenNebula required manual intervention to complete the test due to VMs ending up in bad states, which caused it to be disqualified.

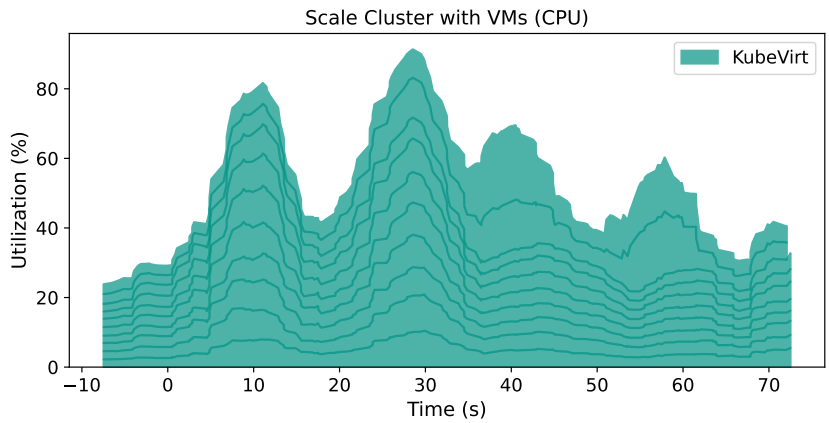

Figure B.40: Cumulative CPU utilization among worker nodes when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes with 1 type-1 VM running on each node. KubeVirt maintains utilization with periodic peaks throughout the test. OpenNebula required manual intervention to complete the test due to VMs ending up in bad states, which caused it to be disqualified.

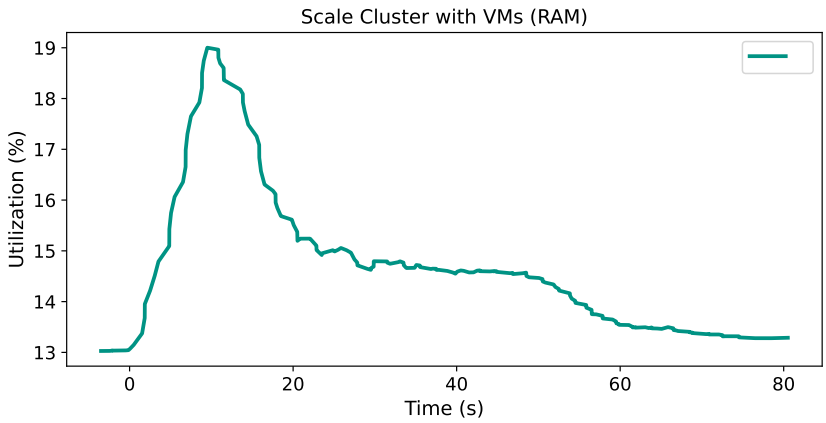

Figure B.41: Memory utilization on the control node when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes with 1 type-1 VM running on each node. KubeVirt experiences a large burst of memory utilization for the scale-up operation, but a significantly lower utilization for the scale-down operation that gradually decreases towards the end. OpenNebula required manual intervention to complete the test due to VMs ending up in bad states, which caused it to be disqualified.

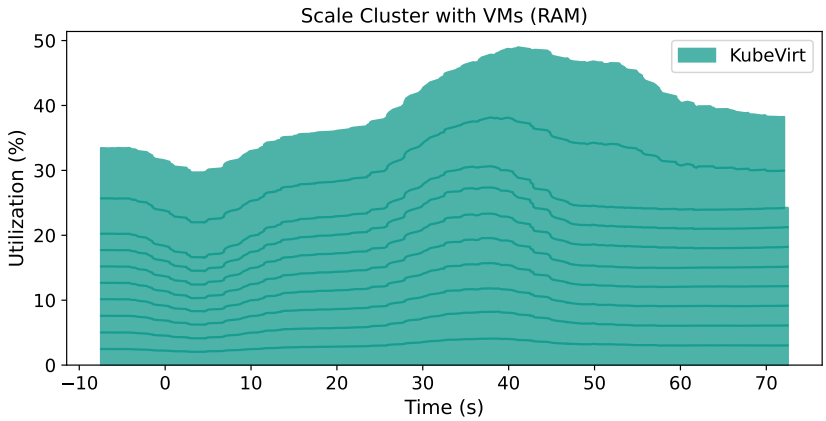

Figure B.42: Cumulative memory utilization among worker nodes when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes with 1 type-1 VM running on each node. KubeVirt maintains a steady memory utilization throughout the test. OpenNebula required manual intervention to complete the test due to VMs ending up in bad states, which caused it to be disqualified.

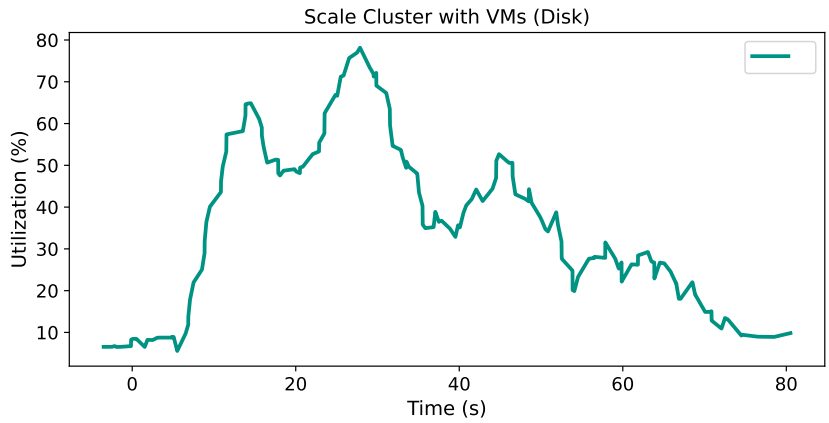

Figure B.43: Disk utilization on the control node when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes with 1 type-1 VM running on each node. KubeVirt experiences a large initial burst of disk utilization that gradually subsides toward the end of the test. OpenNebula required manual intervention to complete the test due to VMs ending up in bad states, which caused it to be disqualified.

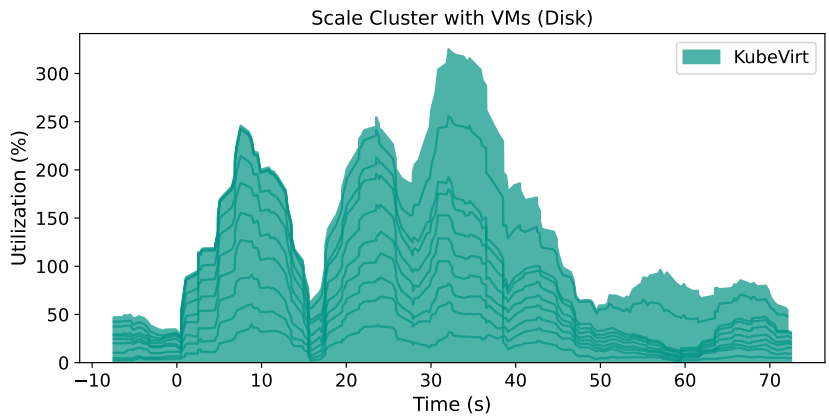

Figure B.44: Cumulative disk utilization among worker nodes when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes with 1 type-1 VM running on each node. KubeVirt experiences large periodic peaks of disk utilization throughout the test. OpenNebula required manual intervention to complete the test due to VMs ending up in bad states, which caused it to be disqualified.

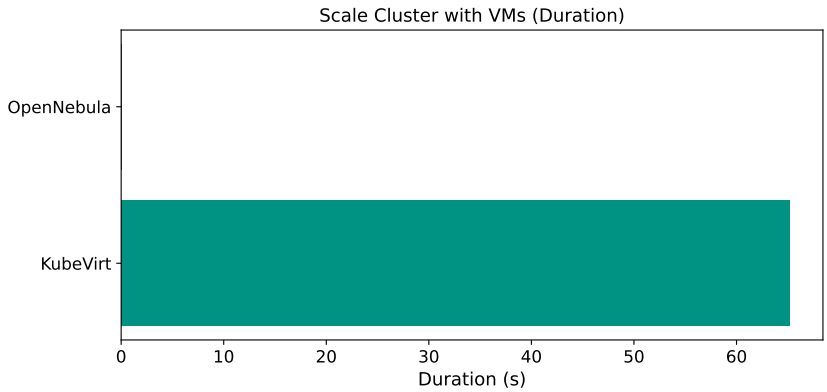

Figure B.45: Duration of scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes with 1 type-1 VM running on each node. KubeVirt completes the test in just below 70 seconds. OpenNebula required manual intervention to complete the test due to VMs ending up in bad states, which caused it to be disqualified.

Appendix C

Usability Tasks

The following is a replicated version of what was given to the participants for the usability analysis formatted for the appendix. The original version was provided as a separate document to the participants.

Introduction

This is a usability analysis that aims to evaluate KubeVirt by performing a set of tasks and then reflecting on the experience. Your answers will be used as a base to evaluate the usability of KubeVirt.

The following is a brief introduction to KubeVirt. If you are already familiar with KubeVirt or feel brave enough, you can skip this part.

KubeVirt can be seen as an extension to Kubernetes that allows VMs to run alongside containers. This means VMs can be treated as any other Kubernetes resource with its own manifests and configuration.

Certain VM operations are not possible using kubectl, such as accessing the VM using SSH or VNC, so KubeVirt provides a complementary CLI tool called *virtctl*.

Since KubeVirt is built on top of Kubernetes, it allows VMs to integrate with other resources, such as Services. For instance, a VM can be exposed to the network using a LoadBalancer Service. However, disk management is a bit different for VMs than for containers, where two examples are listed below.

- **containerDisk**: This disk is created at every boot and makes the VM mimic a container.

- **DataVolume**: This disk rerpesent a persistent disk, meaning it survives between reboots. It can be created using templates in the VM manifest.

Basic usage (Task 1/3)

The first task is about creating a VM using KubeVirt. You will need to create a VM and then access it using SSH or VNC. You can use a containerDisk for simplicity.

Manifests

Below is a minimal manifest for creating the VM using a containerDisk. The credentials for the Cirros image are user **cirros** and password **gocubsgo**.

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: my-vm
spec:
  running: true
  template:
    spec:
      domain:
        cpu:
          cores: 1
        devices:
          disks:
          - name: containerdisk
            disk:
              bus: virtio
          interfaces:
          - name: default
            masquerade: {}
        resources:
          requests:
            memory: 128M
      networks:
      - name: default
        pod: {}
```

```
volumes:
- name: containerdisk
  containerDisk:
    image: quay.io/kubevirt/cirros-
       container-disk-demo
```

Steps

1. Create a VM

```
kubectl apply -f my-vm.yaml
```

2. Access the VM through SSH or VNC (credentials vary depending on the image since no CloudInit is used)

```
virtctl ssh cirros@my-vm
virtctl vnc my-vm
```

3. Stop, start and restart the VM

```
virtctl stop my-vm
virtctl start my-vm
virtctl restart my-vm
```

4. Delete the VM

```
kubectl delete vm my-vm
```

Maintenance (Task 2/3)

The second task is related to maintenance, where you will create a snapshot of a VM and then restore it. You will also perform a live migration on a VM to move it between two nodes.

A live migration is a resource, like any other resource in Kubernetes, however, it does not move a VM by itself, and instead only tells the scheduler it is ready to move. See the References section for more information. For this use case, we want to force a move between two nodes, however, the Kubernetes scheduler is sophisticated and supports a wide range of scheduling strategies. This means we need to "cheat" a bit. We can force a move between two nodes by using the *spec.template.spec.nodeSelector* in the VM and editing the labels of the worker nodes in the cluster.

The snapshot operation, is simmilar to the live migration operation, done using Kubernetes resource. However, the catch here is that containerDisks cannot be snapshotted since they don't represent a persistent storage. Instead you will use a DataVolume.

Live Migration Manifests

The following is a minimal manifest for creating a VM that runs on a node with label **name: worker-1**. The credentials for the Cirros image are user **cirros** and password **gocubsgo**.

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: my-vm
spec:
  running: true
  template:
    spec:
      nodeSelector:
        # Edit this to use a label of your
           choice
        my-label: some-value
      domain:
        cpu:
          cores: 1
        devices:
          disks:
          - name: containerdisk
            disk:
              bus: virtio
          interfaces:
          - name: default
            masquerade: {}
        resources:
          requests:
            memory: 128M
      networks:
      - name: default
        pod: {}
```

```
      volumes:
      - name: containerdisk
        containerDisk:
          image: quay.io/kubevirt/cirros-
             container-disk-demo

apiVersion: kubevirt.io/v1
kind: VirtualMachineInstanceMigration
metadata:
  name: my-vm-migration
spec:
  vmiName: my-vm
```

Live Migration Steps

1. Ensure worker node 1 has your label (Make sure to replace `my-label` and `some-value` with your own values):

   ```
   kubectl label node usability-worker-1 my-label=some
      -value
   ```

2. Create a VM and wait for it to start. The VM should start on **usability-worker-1**:

   ```
   kubectl apply -f my-vm.yml
   ```

3. (Optional) Check if the VM is running on worker node 1:

   ```
   kubectl get vmis
   ```

4. Create a LiveMigration resource or use virtctl:

   ```
   kubectl apply -f my-vm-migration.yml
   ```

   ```
   virtctl migrate my-vm
   ```

5. Add the same label to worker node 2 and remove it from worker node 1 (Make sure to replace `my-label` and `some-value` with your own values):

   ```
   kubectl label node usability-worker-2 my-label=some
      -value
   kubectl label node usability-worker-1 my-label-
   ```

6. (Optional) Check if the VM is now running on worker node 2:

   ```
   kubectl get vmis
   ```

7. Unlabel worker node 2

   ```
   kubectl label node usability-worker-2 my-label-
   ```

8. Delete the VM

   ```
   kubectl delete vm my-vm
   ```

Snapshot Manifests

The following is a minimal manifest for creating a VM with a DataVolume. The credentials for the Cirros image are user **cirros** and password **gocubsgo**.

```yaml
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: my-vm
spec:
  running: true
  template:
    spec:
      domain:
        devices:
          rng: {}
          disks:
          - disk:
              bus: virtio
            name: datavolume-disk
          interfaces:
          - name: default
            masquerade: {}
        resources:
          requests:
            memory: 1Gi
      networks:
      - pod: {}
        name: default
      volumes:
```

```
      - dataVolume:
          name: datavolume
        name: datavolume-disk
  dataVolumeTemplates:
  - metadata:
      name: datavolume
    spec:
      pvc:
        storageClassName: azurefile-csi-kubevirt
        accessModes:
        - ReadWriteOnce
        resources:
          requests:
            storage: 5Gi
      source:
        registry:
          url: docker://quay.io/kubevirt/cirros-
             container-disk-demo
```

The following is a minimal manifest for creating a VirtualMachineSnapshot resource.

```
apiVersion: snapshot.kubevirt.io/v1alpha1
kind: VirtualMachineSnapshot
metadata:
  name: my-vm-snapshot
spec:
  source:
    apiGroup: kubevirt.io
    kind: VirtualMachine
    name: my-vm
```

The following is a minimal manifest for restoring a VM from a snapshot.

```
apiVersion: snapshot.kubevirt.io/v1alpha1
kind: VirtualMachineRestore
metadata:
  name: my-vm-restore
spec:
  target:
    apiGroup: kubevirt.io
    kind: VirtualMachine
```

```
    name: my-vm
  virtualMachineSnapshotName: my-vm-snapshot
```

Snapshot Steps

1. Create a VM

   ```
   kubectl apply -f my-vm.yml
   ```

2. Create a snapshot of the VM

   ```
   kubectl apply -f my-vm-snapshot.yml
   ```

3. Wait for the snapshot to be ready:

   ```
   kubectl get vmsnapshots
   ```

4. Stop the VM

   ```
   virtctl stop my-vm
   ```

5. Restore the VM from the snapshot

   ```
   kubectl apply -f my-vm-restore.yml
   ```

6. Wait for the VM to be restored

   ```
   kubectl get vmrestore
   ```

7. Delete the resources

   ```
   kubectl delete vmsnapshot my-vm-snapshot
   kubectl delete vm my-vm
   ```

Debugging (Task 3/3)

The third an final tasks is related to finding errors with your VM and resolving them. You will be given a VM manifest that successfully creates a KubeVirt VM but that is not accessible over SSH. You can find alternative way to enter the machine, such as through VNC, or go your own way.

Manifests

The following is a manifest for creating a Ubuntu VM that is not accessible through SSH. Credentials for the Ubuntu image should be user **cloud** and password **cloud**.

```yaml
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: my-vm
spec:
  running: true
  template:
    spec:
      domain:
        devices:
          disks:
          - name: containerdisk
            disk:
              bus: virtio
          - name: cloudinit
            disk:
              bus: virtio
          rng: {}
        resources:
          requests:
            cpu: 2
            memory: 2Gi
      volumes:
      - name: containerdisk
        containerDisk:
          image: quay.io/containerdisks/ubuntu
             :22.04
      - name: cloudinit
        cloudInitNoCloud:
          userData: |-
            #cloud-config
            users:
            - name: cloud
```

```
    # Generated with `mkpasswd -m sha
       -512 --rounds 4096`
    passwd: $6$rounds=4096$abc
    shell: /bin/bash
    lock-passwd: false
    chpasswd: { expire: False }
    sudo: ALL=(ALL) NOPASSWD:ALL
    ssh_authorized_keys:
    - ssh-ed25519 abc
  ssh_pwauth: True
```

Steps

1. Create a VM

   ```
   kubectl apply -f my-vm.yml
   ```

2. Access the VM through VNC

   ```
   virtctl vnc my-vm
   ```

3. Find out what the issue is.

4. Edit the VM manifest to fix the issue.

5. (Optional) Restart the VM (some changes require a restart)

   ```
   virtctl restart my-vm
   ```

6. Ensure the VM is accessible through SSH. You can use the *–local-ssh* flag if there is a problem with the SSH keys.

   ```
   virtctl ssh cloud@my-vm
   ```

7. Delete the VM

   ```
   kubectl delete vm my-vm
   ```

Thank you for participating in the usability analysis of KubeVirt!

Appendix D

Usability Analysis Answers

D.1 Participants

- Participant 1: Linus Wallgren, Net Insight AB
- Participant 2: Osvald Ivarsson, Net Insight AB
- Participant 3: Karl Böhlmark, Net Insight AB
- Participant 4: Sai Man Wong, Net Insight AB

D.2 Task 1/3

How would you describe the processes of creating and deleting a VM with KubeVirt?

- **Linus**

  The act of doing *kubectl apply -f* and *kubectl delete* is quite neat, but the VM definition is not very neat. It would be quite difficult for me to figure out I need to use *bus: virtio* without having someone else tell me I need to. The *virtctl create vm* interface on the other hand seems to be much easier to use. As referenced from the official documentation.

  At this point, I decided I wanted to try to launch a Debian VM.

  Using *virtctl create vm* however, it was not clear which container disks are generally available or how I can create one (*virtctl create vm –help* and the help for *–volume-containerdisk*). What I'd like is an explanation of what that flag means.

What I'm searching for now is how to create a Debian VM, looking at `https://quay.io/organization/kubevirt` where the example provided was I could not find anything called *debian*. Just trying *virtctl create vm | kubectl apply -f -* did not provide a working VM either, so I can't just modify that to work as I want.

The documentation referred to *quay.io/containerdisks*, which did have some more useful disks but not Debian (only Ubuntu, Fedora and Centos).

Thought: *containerDisks are not a good solution for any workload that requires persistent root disks across VM restarts, so a VM with a containerDisks is probably not what I want for my bog-standard VM.*

At this point, I thought I would just try to look at the standard KubeVirt documentation, hoping that they mention how you run a VM that isn't just a demo. What I found were two guides:

1. KubeVirt Lab 1 (`https://kubevirt.io/labs/kubernetes/lab1.html`)
   - Did not mention anything on how to run my VM.
2. KubeVirt Lab2 (`https://kubevirt.io/labs/kubernetes/lab2.html`)
   - This might be what I want but it is not clear what a CDI is nor what I should use it for, even after clicking through to `https://github.com/kubevirt/containerized-data-importer`.
   - The first step of the guide is to install an operator, which I don't think I want to do just to get started.
   - As an aside it is a bit worrying that they have written this:
     * If the importer pod completes in error, you may need to retry it or specify a different URL to the fedora cloud image. To retry, first delete the importer pod and the DataVolume, and then recreate the DataVolume.
     * That means I can't rely on just creating the resource and having it do what I want. I need to monitor it with my retry logic to be sure it works.
   - This guide specifies how I can provide my SSH keys via cloud-init, which is nice.
   - Trying to follow that guide gave me this error when I ran *kubectl describe datavolumes.cdi.kubevirt.io*

    * Normal ErrClaimNotValid 48s datavolume-import-controller PVC spec is missing accessMode and no storageClass to choose profile
    * While this doesn't seem like an unsurmountable problem I decided to stop here.

- **Osvald**

  Very easy! I like that it's as simple as creating a YAML file and then applying it. Much simpler than any other way I've tried for managing VMs.

How would you describe KubeVirt's approach to connecting to a VM? (SSH and VNC)

- **Linus**

  Quite, I would have loved to easily be able to integrate the SSH proxy command into my SSH config. Also, I find *virtctl console* nicer than VNC.

- **Osvald**

  Very simple! Simply doing *virtctl ssh cirros@my-vm* and then typing in the password and then I was in!

Descrbie any potential issues you stumbled upon when performing this task. If any, were they related to KubeVirt, its documentation or something else?

- **Linus** I could not figure out how to create a Debian image. The documentation was very lacking as it seemed geared towards how to operate a KubeVirt cluster, not how to use one.

  From the documentation, it also seems that KubeVirt is plagued by the same issues a lot of the Kubernetes ecosystem is: a lot of strange abstractions that aren't well explained (CDI, DataVolume, containerDisks etc.).

- **Osvald**

  Getting VNC to work on Mac was a bit tricky, but I downloaded Tiger VNC and saved the script from (`https://github.com/kubevirt/kubevirt/issues/812#issuecomment-391832335`) as */usr/local/bin/remote-viewer*.

Summary

- **Karl**

  Everything was smooth sailing (except when I didn't read the instructions properly and missed the provided password). This process for managing VMs is much more direct, transparent and reproducible than using a UI-based tool.

- **Sai**

  Personally, from having worked with older virtualization platforms and Kubernetes, I'd prefer to use KubeVirt as my virtualization platform. It is primarily because I'm already storing all my Kubernetes manifest in Git and have the CI setup for that, so why not also create virtual machines in a similar way.

D.3 Task 2/3

How would you describe the live migration operation compared to what you have used before?

- **Linus**

  Not very, it was unintuitive that the migration succeeded even though nothing was migrated, even though it should have been (I used a *nodeSelector* that didn't exist, so it should not have been running on any node).

  I also found it unintuitive that I needed to create the migration resource after everything was correctly labeled. I would have expected that just updating the *nodeSelector* would move the VM.

- **Osvald**

  Very simple! I once again really like that you just edit YAML files and apply labels to conveniently move VMs between worker nodes. Much simpler than our current way of doing it in OpenNebula.

Descrbie any potential issues you stumbled upon when performing this task. If any, were they related to KubeVirt, its documentation or something else?

- **Linus**

  The documentation mentions that I should look at the status of the VMI, which I did not think to do (regarding the intuitive question above).

Had I read the documentation I would at least have known that the migration decided to do nothing when I used a non-existent *nodeSelector*.

Once again the documentation deals a lot with how to operate the migration infrastructure and not anything on how one would do a migration operation. Granted it is listed under the *Operations* section of the documentation but I found nothing when searching under the *Virtual Machines* section where I would have expected it to be.

- **Osvald**

  I had to delete the entry from */.ssh/kubevirt_known_hosts* after each time I restarted the VM because *virtctl ssh* complained about a key mismatch.

Summary

- **Linus**

  I found this unintuitive as if it couldn't migrate (nothing matching the *nodeSelector*) I would have expected the migration to fail, not succeed.

  Once again this is also a bit strange, that I have to remove the migration resource and recreate it for something to happen. That means I can't just create a resource and trust KubeVirt will manage it but will need to have logic around the migration.

D.4 Task 3/3

How would you describe the debugging process compared to what you used before?

- **Linus**

  Not very, I did not manage to solve all the issues.

  The password issue was easy as not being able to log in was very obvious with *virtctl console*. The SSH issue I solved by accident, the *known-hosts: key mismatch* made me look into the *.ssh/authorized_keys* file and updated that with *virtctl console*. It didn't solve the key mismatch issue but it did allow me to continue with local SSH. I did not manage to solve the key mismatch issue as there is very little in the way of debugging *virtctl ssh*.

- **Osvald**

  Easy! Using *virtctl vnc* I could change the password and then try logging in.

How easy was it to fix the issue?

- **Linus**

  Two of the issues were easy to fix, I appreciated that *virtctl restart* worked as I did not think cloud-init would update the configs.

- **Osvald**

  Very easy, I just generated a new password.

How would you say the debugging process differs between KubeVirt and a traditional VM management system?

- **Linus**

  I'm not sure what a traditional VM management system refers to. It is quite similar to the OpenNebula command lines. I appreciated that *virtctl console* was so easy to use.

- **Osvald**

  I've used IPMI which is similar to the VNC session in this case.

Descrbie any potential issues you stumbled upon when performing this task. If any, were they related to KubeVirt, its documentation or something else?

- **Linus**

  I could not (easily) find documentation about how *virtctl ssh* worked so I could not figure out one of the issues. There seems to be no documentation for *virtctl*. Is it the same *virtctl* as the one from OpenShift? The fact that *kubectl ssh* broke without even a verbose option is worrying.

- **Osvald**

  The most difficult thing was to get the VNC viewer working on Mac, but after that, it was smooth sailing.

Summary

- **Linus**

  A "key mismatch" error is difficult to debug because there are no good debug options for *virtctl ssh*.

- **Karl**

  Debugging the issue was easy because the manifest uses standard constructs that are easily googleable.

D.5 Summary

How would you describe KubeVirt with previous VM management systems?

- **Linus**

  I like the *virtctl* tool (when it worked) and like the idea of declaring my VMs. However, the system is very complicated with a lot of parts that it isn't clear how they fit together.

  The documentation is also very lacking. The fact that you manage the system with two different tools, both *virtctl* and *kubectl* is unnecessary and confusing. I find it telling that I didn't even manage to run a Debian VM.

- **Osvald**

  It seems to be much easier to deploy and manage machines! I want to start using this in production for our build pipelines and internal clusters.

How would you describe KubeVirt's workflow? Meaning the steps required to do a task. Is it possible to use without resorting too much to the documentation?

- **Linus**

  Not at all. I would not have been able to use it without the documentation. I could barely use it with the documentation.

- **Osvald**

  Well, I had the example files you provided which were simple. But the CLI tools also gave good help.

How would you describe KubeVirt as a whole?

- **Linus**

  The idea is good but the implementation is severely lacking. I could see the potential and it seemed very promising but from this experience, KubeVirt did not deliver.

  One could probably build something sensible with it but I feel the barrier of entry is very high.

- **Osvald**

  Great! I want more!

- **Karl**

  For developers with previous Kubernetes experience, it looks to be a very intuitive way to handle virtual machines. In an environment where Kubernetes clusters are already part of the workflow, it is obvious how aspects such as automation and access control can be transferred to apply also to the VM infrastructure.

  Since I was just trying it out, I haven't had any real issues that I need to debug yet.

- **Sai**

  I've been using KubeVirt for several months and I like it. Specifically, the cloud native approach where everything is API accessible, which makes it easier to build tooling around and the community has done a great job of that as expected. Due to the fact it's APIs first, it opens up many approaches to manage virtual machines. Personally, from having worked with older virtualization platforms and Kubernetes, I'd prefer to use KubeVirt as my virtualization platform.

  I'd say I like this relatively new take on how to deal with virtual machines. At the same time, it may not be for everyone. In the end, it depends on what problems you are trying to solve and the need.